

Python and Computational Thinking

A Crash Course for Scientists

Frithjof Herb

*A practical guide for researchers who need to analyze data,
automate experiments, and create reproducible workflows—
without getting a computer science degree in the process.*

Draft Version 2.1

January 29, 2026

Contents

Preface	xi
How to Use This Book	xiii
I. Getting Started	1
1. Setting Up Your Environment	3
1.1. Why Scientists Program	3
1.2. Computer Hygiene	3
1.2.1. Folder Organisation	4
1.2.2. File Naming	4
1.2.3. Backup	4
1.3. Paths	4
1.4. Installing Python	5
1.4.1. Windows	5
1.4.2. macOS	5
1.4.3. Linux	6
1.5. The Terminal: Talking to Your Computer	6
1.6. Your First Python Session	7
1.7. Virtual Environments	8
1.7.1. The Problem	8
1.7.2. The Solution	8
1.7.3. Creating a Virtual Environment	8
1.7.4. Activating the Environment	9
1.7.5. Installing Packages	9
1.7.6. Saving and Reproducing Environments	10
1.7.7. Deactivating	10
1.8. Three Ways to Run Python Code	10
1.8.1. Interactive REPL	10
1.8.2. Python Scripts	11
1.8.3. Jupyter Notebooks	11
1.9. Choosing an Editor	12
1.9.1. Start Simple: Basic Text Editors	12
1.9.2. Upgrading Later: Integrated Development Environments	12
1.10. Chapter Summary	13
II. Python Fundamentals	15
2. Python Basics	17
2.1. Python as a Calculator	17
2.1.1. More Operations	17
2.1.2. Order of Operations	18
2.2. Variables and Assignment	18
2.2.1. Naming Rules	18

2.2.2. Meaningful Names	19
2.3. Data Types	19
2.3.1. Integers (int)	20
2.3.2. Floating-Point Numbers (float)	20
2.3.3. Strings (str)	20
2.3.4. Booleans (bool)	21
2.3.5. None	21
2.4. Type Checking	21
2.4.1. Type Conversion	21
2.5. Operators and Expressions	22
2.5.1. Comparison Operators	22
2.5.2. Logical Operators	23
2.5.3. String Operators	23
2.5.4. Compound Assignment	23
2.6. Comments	24
2.7. Formatted Output with f-strings	24
2.7.1. Formatting Numbers	25
2.8. Getting User Input	26
2.9. Writing Your First Script	26
2.10. Chapter Summary	27
3. Container Types	29
3.1. Lists	29
3.1.1. Indexing	29
3.1.2. Slicing	30
3.1.3. List Length	30
3.1.4. Modifying Lists	30
3.1.5. List Operations	31
3.1.6. Useful List Methods	32
3.2. Tuples	32
3.2.1. Why Use Tuples?	33
3.2.2. Tuple Packing and Unpacking	33
3.3. Dictionaries	34
3.3.1. Adding and Modifying	34
3.3.2. Accessing Values Safely	34
3.3.3. Dictionary Methods	35
3.3.4. Nested Dictionaries	35
3.4. Sets	36
3.4.1. Set Operations	36
3.4.2. Membership Testing	36
3.4.3. Modifying Sets	36
3.5. Choosing the Right Container	37
3.6. Converting Between Types	38
3.7. Nested Structures	38
3.8. Variables Are Labels, Not Boxes	39
3.9. Chapter Summary	40
4. Control Flow	43
4.1. Making Decisions with if	43
4.1.1. Indentation Matters	43
4.1.2. if-else	43
4.1.3. if-elif-else	44

4.1.4. Nested Conditions	44
4.1.5. Truthiness	45
4.2. Repeating with for Loops	45
4.2.1. Looping Over Ranges	45
4.2.2. Looping with Index: enumerate()	46
4.2.3. Looping Over Dictionaries	46
4.2.4. Looping Over Multiple Lists: zip()	47
4.3. Repeating with while Loops	47
4.4. Controlling Loops: break and continue	48
4.4.1. break: Exit the Loop Early	48
4.4.2. continue: Skip to Next Iteration	48
4.5. List Comprehensions	49
4.5.1. Adding Conditions	49
4.5.2. Practical Examples	49
4.6. Dictionary and Set Comprehensions	49
4.7. Structural Pattern Matching	50
4.7.1. Basic Matching	50
4.7.2. Matching Structures	51
4.8. Chapter Summary	52
5. Functions	53
5.1. Why Functions?	53
5.2. Defining Functions	53
5.2.1. Parameters and Arguments	54
5.2.2. Return Values	54
5.2.3. Multiple Return Values	54
5.3. Default Parameters	55
5.3.1. The Mutable Default Argument Trap	55
5.4. Keyword Arguments	56
5.5. Flexible Arguments: *args and **kwargs	57
5.5.1. *args: Variable Positional Arguments	57
5.5.2. **kwargs: Variable Keyword Arguments	57
5.5.3. Why This Matters	57
5.6. Docstrings	58
5.7. Scope: Where Variables Live	58
5.8. Functions as Values	59
5.9. Lambda Functions	60
5.10. Built-in Functions Worth Knowing	60
5.11. A Complete Example	61
5.12. Chapter Summary	61
6. Working with Files	63
6.1. Opening and Closing Files	63
6.2. The with Statement	63
6.3. Reading Files	63
6.3.1. Read Entire File	63
6.3.2. Read Lines	64
6.3.3. Read Line by Line (Memory Efficient)	64
6.4. Writing Files	64
6.4.1. Write Text	64
6.4.2. Write Multiple Lines	65
6.5. Working with Paths	65

6.6. CSV Files	66
6.6.1. Reading CSV with the csv Module	66
6.6.2. CSV with Headers: DictReader	66
6.6.3. Writing CSV	66
6.7. Handling Different Delimiters	67
6.8. JSON Files	68
6.8.1. Reading JSON	68
6.8.2. Writing JSON	68
6.9. Handling Encoding	68
6.10. Error Handling for Files	69
6.11. A Complete Example	69
6.12. Chapter Summary	70
7. Errors and Exceptions	71
7.1. Reading Error Messages	71
7.2. Common Error Types	71
7.2.1. SyntaxError	71
7.2.2. NameError	71
7.2.3. TypeError	72
7.2.4. ValueError	72
7.2.5. IndexError	72
7.2.6. KeyError	72
7.2.7. FileNotFoundError	72
7.2.8. AttributeError	72
7.3. Handling Exceptions with try-except	73
7.3.1. Catching Multiple Exception Types	73
7.3.2. Accessing Exception Information	73
7.3.3. Catching All Exceptions	73
7.4. else and finally	74
7.4.1. else: Code That Runs If No Exception	74
7.4.2. finally: Code That Always Runs	74
7.5. Raising Exceptions	75
7.5.1. When to Raise Exceptions	75
7.6. Assertions	76
7.7. Debugging Strategies	76
7.7.1. Read the Error Message	76
7.7.2. Add Print Statements	76
7.7.3. Simplify	77
7.7.4. Check Types	77
7.7.5. Use a Debugger	77
7.8. Defensive Programming	77
7.8.1. Validate Input Early	77
7.8.2. Use Default Values	78
7.8.3. Fail Fast	78
7.9. Automated Testing with pytest	78
7.9.1. Writing Your First Test (and the Float Trap)	78
7.9.2. Testing for Expected Errors	79
7.10. Chapter Summary	80
8. Object-Oriented Programming	81
8.1. What Are Objects?	81

8.2. Defining a Class	81
8.2.1. Creating Objects	82
8.3. The self Parameter	82
8.4. Attributes and Methods	82
8.4.1. Instance Attributes	82
8.4.2. Class Attributes	83
8.4.3. Methods	83
8.5. Special Methods	83
8.5.1. <code>__str__</code> and <code>__repr__</code>	83
8.5.2. Comparison Methods	84
8.5.3. Arithmetic Methods	84
8.6. Encapsulation	85
8.7. Inheritance	85
8.8. Decorators and Dataclasses	86
8.8.1. What Are Decorators?	86
8.8.2. The <code>@dataclass</code> Decorator	86
8.8.3. Dataclasses vs Regular Classes	88
8.9. When to Use Classes	88
8.10. Chapter Summary	89
 III. Scientific Python	 91
 9. NumPy: Numerical Computing	 93
9.1. Why NumPy?	93
9.2. Creating Arrays	93
9.2.1. From Lists	93
9.2.2. With Functions	94
9.3. Array Properties	94
9.4. Indexing and Slicing	95
9.4.1. 1D Arrays	95
9.4.2. 2D Arrays	95
9.4.3. Boolean Indexing	95
9.5. Array Operations	96
9.5.1. Element-wise Operations	96
9.5.2. Mathematical Functions	96
9.5.3. Aggregation Functions	96
9.6. Broadcasting	97
9.7. Reshaping Arrays	98
9.8. Stacking and Splitting	98
9.9. Linear Algebra	99
9.10. Saving and Loading Data	99
9.11. Common Patterns	100
9.11.1. Normalize Data	100
9.11.2. Moving Average	100
9.11.3. Finding Peaks	100
9.12. Chapter Summary	100
 10. Pandas: Data Analysis	 103
10.1. Why Pandas?	103
10.2. Series	103

10.3. DataFrames	103
10.3.1. Reading Data from Files	104
10.3.2. Quick Look at Data	104
10.4. Selecting Data	104
10.4.1. Selecting Columns	104
10.4.2. Selecting Rows	104
10.4.3. Selecting Rows and Columns	105
10.5. Modifying Data	105
10.5.1. Adding Columns	105
10.5.2. Modifying Values	105
10.5.3. Renaming	105
10.5.4. Dropping	106
10.6. Handling Missing Data	106
10.7. Aggregation and Grouping	106
10.7.1. Basic Statistics	106
10.7.2. Group By	107
10.8. Sorting	107
10.9. Merging DataFrames	108
10.10Pivot Tables	108
10.11Applying Functions	108
10.12Saving Data	109
10.13Working with Dates and Times	109
10.13.1.Python’s datetime Module	109
10.13.2.Date Time in Pandas	110
10.14A Complete Analysis Example	112
10.15Chapter Summary	113

11. Matplotlib: Visualization 115

11.1. The Basics	115
11.1.1. Adding Labels and Title	115
11.1.2. Saving Figures	116
11.2. Line Plots	116
11.2.1. Multiple Lines	116
11.2.2. Line Styles and Colors	117
11.3. Scatter Plots	118
11.3.1. Customizing Markers	119
11.3.2. Color by Value	119
11.4. Bar Charts	120
11.4.1. Grouped Bars	120
11.4.2. Error Bars	121
11.5. Histograms	122
11.5.1. Customizing Histograms	122
11.6. Subplots	123
11.6.1. Sharing Axes	124
11.7. The Object-Oriented Interface	124
11.8. Customization	124
11.8.1. Axis Limits and Ticks	124
11.8.2. Log Scales	124
11.8.3. Grid	125
11.8.4. Annotations	125
11.9. Style and Aesthetics	125
11.9.1. Built-in Styles	125

11.9.2. Custom RC Parameters	125
11.9.3. Publication-Ready Settings	125
11.10 Specialized Plots	125
11.10.1. Contour Plots	125
11.10.2. Heatmaps	126
11.10.3. Box Plots	127
11.11 A Complete Figure Example	127
11.12 Chapter Summary	129
12. SciPy: Scientific Computing	131
12.1. Statistics	131
12.1.1. Descriptive Statistics	131
12.1.2. Distributions	131
12.1.3. Statistical Tests	131
12.1.4. Correlation	132
12.2. Curve Fitting	132
12.2.1. Linear Regression	132
12.2.2. Nonlinear Curve Fitting	133
12.3. Optimization	135
12.3.1. Minimization	135
12.3.2. Root Finding	136
12.4. Integration	136
12.4.1. Definite Integrals	136
12.4.2. Differential Equations	136
12.5. Interpolation	138
12.6. Signal Processing	139
12.6.1. Smoothing	139
12.6.2. Peak Finding	140
12.6.3. Fourier Transform	140
12.7. Special Functions	141
12.8. Physical Constants	141
12.9. Chapter Summary	142
IV. Putting It Together	143
13. Practical Projects	145
13.1. Project 1: Calibration Curve Analysis	145
13.2. Project 2: Kinetics Data Analysis	147
13.3. Project 3: Batch Processing Spectra	149
13.4. Project 4: Interactive Data Dashboard	152
13.5. Tips for Real Projects	154
13.5.1. Project Organization	154
13.5.2. Reproducibility Checklist	154
13.5.3. Common Patterns	154
V. Closing Remarks	157
13.6. Where to Go From Here	159
13.6.1. Packages to Explore	159
13.6.2. Domain-Specific Libraries	159
13.6.3. Improve Your Workflow	159

13.6.4. Practice 159

13.6.5. AI in the Scientific Workflow 159

13.6.6. Where to Get Help 161

Preface

Who This Book Is For

This book is for scientists who need to program but never had formal training. You might be a chemistry grad student drowning in spectrometer data, a biologist tired of clicking through spreadsheets, or a physicist who inherited a folder of mysterious scripts from a graduated colleague. You know your science—you just need the programming skills to match.

This is not a computer science textbook. We won't discuss algorithm complexity or debate programming paradigms. Instead, we focus on what you need: reading data files, doing calculations, making figures, and automating the tedious parts of your work.

What You'll Learn

By the end of this book, you'll be able to:

- Set up a proper Python environment that won't break your computer
- Write scripts that read, process, and analyze your experimental data
- Create publication-quality figures
- Automate repetitive tasks that currently eat your afternoons
- Understand enough programming concepts to learn more on your own

We use chemistry examples throughout—titrations, spectra, kinetics—but the skills transfer to any scientific field. If you work with numbers and data, this book is for you.

How to Approach This Book

Programming is a skill, like pipetting or playing piano. You can't learn it by reading—you have to do it. Every code example in this book is meant to be typed out and run. Copy-pasting works, but typing builds muscle memory, confidence, and forces you to notice details.

A reasonable pace is one chapter per week: read through, try all the examples, then tackle the exercises. Rushing won't help. Neither will reading ahead without practicing. When you hit something confusing, that's normal. Read it again, try it in Python, and it will click.

A Note on Errors

You will see error messages. Many of them. This is not failure—it's how programming works. Every programmer, from beginner to expert, spends significant time reading error messages and figuring out what went wrong. The goal isn't to avoid errors; it's to get comfortable reading them and fixing the underlying problem.

When Python shows you an error, it's trying to help. The message tells you what went wrong and where. Learning to read these messages is as important as learning to write code.

Why Python?

Python has become the default language for scientific computing, and for good reason. It's readable, has excellent libraries for data analysis and visualization, and has a massive community of scientists

who share code and answer questions online. If you search for how to do almost anything in Python, you'll find answers.

Could you use R, MATLAB, or Julia? Sure. They're all fine languages. But Python's combination of general-purpose programming and scientific tools makes it the most practical choice for most researchers. Learn Python well, and you'll have a skill that serves you for your entire career.

How to Use This Book

Read in Order (At First)

The chapters build on each other. Chapter 2 assumes you completed Chapter 1. Skipping ahead usually leads to confusion and backtracking. Once you’ve worked through the whole book, it becomes a useful reference for jumping to specific topics. But for first-time learners: start at the beginning.

Special Boxes

Throughout the book, you’ll find colored boxes that highlight different types of information:

Warning

Warnings alert you to common mistakes and pitfalls. Pay attention to these—they’ll save you debugging time.

Tip

Tips offer shortcuts, best practices, or alternative approaches that experienced programmers use.

Note

Notes provide additional context or clarification that might help your understanding but isn’t strictly necessary.

OS Differences

OS Differences explain when something works differently on Windows, macOS, or Linux. Most of Python is identical across systems, but installation and file paths sometimes differ.

Lab Example

Lab Examples connect programming concepts to real scientific scenarios. These show how you might actually use what you’re learning in your research.

Things to Ponder

Things to Ponder are reflection questions at the end of sections. You don’t need to answer them in writing, but thinking about them helps solidify concepts.

Code Examples

Code appears in monospace font with syntax highlighting:

```
1 # This is Python code
2 temperature = 298.15 # Kelvin
3 print(f"Temperature: {temperature} K")
```

When we show interactive Python sessions (the REPL), you'll see `>>>` prompts:

```
1 >>> 2 + 2
2 4
3 >>> "hello".upper()
4 'HELLO'
```

Lines starting with `>>>` are what you type. Lines without are Python's output.

Typing vs. Copying

Type the code examples yourself. Yes, it's slower than copy-pasting. That's the point. Typing forces you to read every character, notice the syntax, and build muscle memory. It also means you'll make typos, see error messages, and learn to fix them—all valuable skills.

Data Files

Some examples use data files (CSV files, text files with measurements, etc.). These are available in the `data/` folder that accompanies this book. When an example references a file like `titration_data.csv`, you'll find it there.

Getting Unstuck

When something doesn't work:

1. Read the error message carefully. Python tells you what's wrong.
2. Check for typos. Case matters. Indentation matters. Quotes must match.
3. Re-read the relevant section. Something may have not registered the first time.
4. Search online. Copy-paste the error message into a search engine. Someone else has had the same problem.
5. Take a break. Fresh eyes find bugs faster.

Debugging is part of programming. Getting stuck doesn't mean you're bad at this—it means you're learning.

Part I.

Getting Started

Chapter 1.

Setting Up Your Environment

Before writing any code, you need a working Python installation and a sensible way to organise your files. This chapter covers both. It's tempting to skip ahead to "real" programming, but a broken setup will frustrate you later. Invest an hour now; save countless hours tracking down environment problems.

1.1. Why Scientists Program

You already know how to analyse data. You've used Excel, maybe Origin or Igor Pro. You can make graphs and calculate statistics. So why learn to program?

Reproducibility. When you analyse data by clicking through menus, the process lives only in your memory. Three months later, a reviewer asks how you normalised those spectra. Good luck remembering which buttons you clicked. A *script*—a text file containing instructions for the computer—documents every step. Run it again and you get exactly the same result.

Automation. You have 200 data files from a week of measurements. Processing them one by one takes hours. A script processes them in seconds—and doesn't make mistakes at file 187 because it's tired.

Flexibility. Commercial software does what its developers anticipated. Your analysis might need something unusual: a custom baseline correction, a niche fitting function, or a way to merge data from three different instruments. With programming, you build exactly what you need.

Figures. Publication-quality figures require control over every detail: axis labels, font sizes, colour schemes, subplot layouts. Matplotlib gives you that control. Once you write the code for a figure style you like, every future paper uses the same polished look.

Lab Example

Imagine you collect UV-Vis spectra weekly to monitor a reaction. Each measurement produces a file. By the end of the project, you have 50 files. A Python script can:

- Read all 50 files automatically
- Subtract baseline from each spectrum
- Calculate the absorbance at your wavelength of interest
- Plot the results vs. time
- Export the processed data and figure

When your advisor asks "What if we use a different baseline method?"—you change one line and re-run.

1.2. Computer Hygiene

Before installing anything, let's establish some habits that prevent future headaches.

1.2.1. Folder Organisation

Create a dedicated folder for your programming work. Don't scatter scripts across your Desktop, Downloads, and random project folders. Something like:

```

1 Documents/
2   python_projects/
3     learning/           # exercises from this book
4     lab_analysis/       # your research scripts
5     utilities/          # small helper scripts

```

Within each project, keep a consistent structure:

```

1 titration_project/
2   data/                # raw data files (never modify these)
3   output/              # processed data, figures
4   scripts/             # your Python code
5   README.txt           # notes about what this project does

```

Warning

Never put spaces in folder or file names used with code. Use underscores instead: `titration_data.csv`, not `titration data.csv`. Spaces cause problems with many programming tools.

1.2.2. File Naming

Good file names are:

- Descriptive: `analyze_kinetics.py`, not `script1.py`
- Dated when relevant: `calibration_2024_03_15.csv`
- Consistent: pick a convention and stick with it

1.2.3. Backup

Your code represents hours of work. Back it up. Options:

- Cloud sync (Dropbox, OneDrive, Google Drive): automatic, easy
- Git: a system that tracks every change you make to files, letting you undo mistakes and see your project's history. Best for code, but has a steeper learning curve.
- External drive: simple but requires discipline

At minimum, use cloud sync for your `python_projects` folder. We'll discuss Git later—it's worth learning, but not today.

1.3. Paths

A *path* is the address of a file on your computer. There are two types:

- **Absolute path:** Full address from the root of the filesystem
 - Windows: `C:\Users\Alice\Documents\data.txt`
 - macOS/Linux: `/home/alice/Documents/data.txt`
- **Relative path:** Address relative to your current working directory
 - `data.txt` (file in current directory)
 - `data/measurements.txt` (in a subfolder)
 - `../other_project/data.txt` (`..` means parent directory)

1.4. Installing Python

Python is free. You download it, install it, and use it forever without paying anyone. Several distributions exist; we'll use the standard one from python.org.

OS Differences

Installation differs slightly by operating system. Follow the section for your system, then continue to the next section about the terminal.

1.4.1. Windows

1. Go to <https://www.python.org/downloads/>
2. Click the big yellow “Download Python” button (for this book, get version 3.10 or later, though generally not the latest)
3. Run the installer
4. **Important:** Check the box that says “Add Python to PATH”
5. Click “Install Now”
6. When finished, click “Disable path length limit” if that option appears

Note

What is “PATH”? When you type a command like `python`, your computer needs to know where to find the Python program. The PATH is a list of folders the computer searches through. “Add Python to PATH” means: “remember where Python is installed so I can run it from anywhere.” Without this, you'd have to type the full location of Python every time.

To verify the installation:

1. Open PowerShell (search for it in the Start menu)
2. Type `python --version` and press Enter
3. You should see something like `Python 3.13.4`, select at least version 3.10 or later.

Warning

Windows sometimes has an old “python” command that opens the Microsoft Store. If `python --version` opens the Store instead of showing a version number, try `python3 --version` instead. The “Add Python to PATH” checkbox during installation usually prevents this.

1.4.2. macOS

macOS comes with an old Python 2, which we don't want. Install Python 3:

1. Go to <https://www.python.org/downloads/>
2. Download the macOS installer (version 3.10 or later)
3. Open the downloaded `.pkg` file
4. Follow the installation wizard

To verify:

1. Open Terminal (find it in Applications > Utilities, or search with Spotlight)
2. Type `python3 --version` and press Enter
3. You should see the version number

Note

On macOS, always use `python3` and `pip3` instead of `python` and `pip`. The shorter commands often point to the old system Python 2.

1.4.3. Linux

Most Linux distributions include Python 3. Check first:

```
1 python3 --version
```

If Python 3.10 or later is installed, you're ready to continue. If not, or if you simply want a newer version:

Ubuntu/Debian:

```
1 apt update
2 apt install python3 python3-pip python3-venv
```

Fedora:

```
1 dnf install python3 python3-pip
```

Note**Administrative Privileges Required**

These commands modify system files and require administrative access. You will likely need to prepend `sudo` (e.g., `sudo apt update`) to execute them.

I intentionally omit `sudo` from the examples. It is good practice to type `sudo` manually, as it forces a moment of pause to verify exactly what command you are about to run with root privileges. You should never blindly copy-paste shell commands—especially those involving superuser privileges—without first understanding what the command will do.

1.5. The Terminal: Talking to Your Computer

You're used to interacting with your computer by clicking icons and menus. But there's another way: typing commands directly. The program where you type these commands is called a *terminal* (also called a “command line” or “shell”). Think of it as having a text conversation with your computer instead of pointing at things.

Why use the terminal? Because many programming tools—including Python—are designed to be run by typing commands. It's also faster once you learn the basics, and it's the universal way programmers work across all operating systems.

Each operating system has its own built in terminal program:

- **Windows:** Command Prompt or PowerShell (both pre-installed; search in Start menu). PowerShell is the newer one.
- **macOS:** Terminal (in Applications > Utilities, or search with Spotlight)
- **Linux:** Terminal (varies by distribution; usually Ctrl+Alt+T opens it)

You'll also hear about Anaconda Prompt. Anaconda is an alternative Python distribution popular in data science. It's fine, but adds complexity. We'll use standard Python to keep things clean.

Open your terminal now. You'll see a *prompt*—some text followed by a cursor, waiting for you to type something:

```

1 PS C:\Users\yourname>           # Windows PowerShell
2 C:\Users\yourname>             # Windows Command Prompt
3 yourname@computer ~ %          # macOS Terminal
4 yourname@computer:~$           # Linux Terminal

```

The exact appearance varies, but they all work the same way: you type a *command* (an instruction) and press Enter. The computer does what you asked, shows you the result, and waits for the next command.

1.6. Your First Python Session

Let's run Python. In your launcher, type:

```

1 python           # Windows
2 python3          # macOS/Linux

```

You should see something like:

```

1 Python 3.11.4 (main, Jun  9 2023, 07:30:55)
2 [GCC 12.2.0] on linux
3 Type "help", "copyright", "credits" or "license" for more information.
4 >>>

```

The `>>>` is Python's prompt. It's waiting for you to type something. This interactive mode is called the REPL: Read-Eval-Print Loop. Python reads what you type, evaluates it, prints the result, then loops back to wait for more.

Try some calculations by entering an expression followed by the *enter key*:

```

1 >>> 2 + 2
2 4
3 >>> 7 * 6
4 42
5 >>> 10 / 3
6 3.3333333333333335

```

Python should respond immediately with the answer. Now try something more interesting (**note:** the quotes are important, you will get an error if you omit these. This is explained soon):

```

1 >>> print("Hello, scientist!")
2 Hello, scientist!

```

Here we're using `print()`, which is a *function*. A function is a named action—you give it some input (inside the parentheses), and it does something with it. The `print()` function displays whatever you give it on screen. The quotes around the text mark it as a *string*—a piece of text data, rather than a command or variable name. We'll learn much more about functions and strings soon.

To exit the REPL, simply use the exit function with no input:

```

1 >>> exit()

```

Or press Ctrl+D (macOS/Linux) or Ctrl+Z then Enter (Windows).

Things to Ponder

What happens if you type `print(Hello)` without the quotes? Try it and read the error message. Why does Python complain?

1.7. Virtual Environments

This section might seem bureaucratic, but it solves real problems. Paying attention here will save you many headaches in the future, while making a lot of public code more accessible to you.

1.7.1. The Problem

Python by itself can do basic things, but its real power comes from *packages* (also called *libraries*)—collections of code that other people have written and shared. Think of them like apps for your phone: Python is the operating system, and packages add new abilities.

While you *could* write your own statistical analysis tools from scratch, it is far more efficient to leverage the vast ecosystem of tools made available for free. For example, **NumPy** handles numerical computing, **Matplotlib** makes plots, and **Pandas** manages data tables. A major strength of Python is this community ecosystem. You’ll install and use many of them.

But here’s the problem: different projects might need different versions of the same package. Project A needs a package called NumPy at version 1.24 because it has a new and improved function; Project B needs NumPy version 1.21 because it uses a depreciated function that isn’t support the new version.

If you install packages globally (available everywhere on your computer), you can only have one version. Updating for Project A breaks Project B and vice-versa. This is called “dependency hell,” and it’s why scientists often say “I reinstalled Python and now nothing works.”

1.7.2. The Solution

Virtual environments. Each project gets its own isolated Python environment with its own packages. Project A’s environment has NumPy 1.24. Project B’s environment has NumPy 1.21. They don’t interfere with each other.

There are several external tools that manage environments (e.g., Conda, Poetry, Pipenv). While I highly recommend looking into *uv* (by Astral) once you are comfortable, this book focuses on Python’s standard, built-in tools. Mastering these first ensures you understand the core mechanics without needing to install extra software, and provides the foundation needed to effectively use power tools like *uv* later.

1.7.3. Creating a Virtual Environment

First, we need a place for your project to live. Instead of switching back to your file explorer and clicking around, we can do this directly from the terminal.

1. **Navigate** to where you keep your projects (e.g., Documents):

```
1 cd Documents
2
```

2. **Create** a new folder using the *make directory* command (**mkdir**):

```
1 mkdir learning
2
```

3. **Enter** the new folder:

```
1 cd learning
2
```

Tip

Don't type long folder names manually! Type the first few letters (e.g., `cd Doc`) and press **Tab**. The terminal will auto-complete the rest. This prevents typos and makes navigation significantly faster.

Create a virtual environment (we'll call it `venv`):

```
1 python -m venv .venv          # Windows
2 python3 -m venv .venv        # macOS/Linux
```

This tells `python` to use a module (`-m`) called `venv` to create a virtual environment in the folder `.venv`. This now contains a private Python installation.

1.7.4. Activating the Environment

Before using the environment, you must activate it:

OS Differences**Windows (PowerShell):**

```
1 .\venv\Scripts\Activate.ps1
```

Windows (Command Prompt):

```
1 .venv\Scripts\activate.bat
```

macOS/Linux:

```
1 source .venv/bin/activate
```

Note

Remember that our virtual environment folder is called `.venv` not `venv`.

When activated, your prompt changes to show the environment name:

```
1 (venv) PS C:\Users\yourname\Documents\python_projects\learning>
```

That `(venv)` prefix means you're in the virtual environment. Any packages you install now go into this environment only.

Warning

If you get an error about "execution policies" on Windows PowerShell, run this command first (as administrator):

```
1 Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Then try activating again.

1.7.5. Installing Packages

With the environment activated, use `pip` to install packages. `Pip` is Python's package manager—a tool that downloads packages from the internet and installs them for you. Think of it like an app store that you access by typing commands.

```
1 pip install numpy matplotlib pandas
```

This downloads NumPy, Matplotlib, and Pandas from the Python Package Index (a central repository of Python packages) and installs them into your virtual environment. To see what's installed:

```
1 pip list
```

1.7.6. Saving and Reproducing Environments

After installing packages, save the list to a file:

```
1 pip freeze > requirements.txt
```

This creates a `requirements.txt` file listing every package and its exact version. When setting up the project on another computer (or helping a collaborator), they can recreate your environment:

```
1 pip install -r requirements.txt
```

Lab Example

Your collaborator can't reproduce your analysis. "It works on my machine," you say. They're missing a package, or have a different version. Send them your `requirements.txt` and tell them to:

```
1 python3 -m venv venv
2 source venv/bin/activate
3 pip install -r requirements.txt
```

Now they have exactly your environment.

1.7.7. Deactivating

When you're done working, deactivate the environment:

```
1 deactivate
```

The `(venv)` prefix disappears. You're back to the system Python.

Tip

Always activate your project's virtual environment before working. Make it a habit, like putting on safety glasses before entering the lab. Many "my code stopped working" problems are just forgotten activation.

1.8. Three Ways to Run Python Code

You've seen the REPL. It's great for quick experiments, but not for serious work. Let's cover all three ways to run Python code.

1.8.1. Interactive REPL

You already know this one. Type `python` (or `python3`), and you're in an interactive session. Good for:

- Quick calculations
- Testing small code snippets
- Exploring how functions work

Not good for:

- Anything longer than a few lines
- Code you want to save and reuse

1.8.2. Python Scripts

A script is a text file containing Python code, saved with a `.py` extension. You write the code once, then run it whenever you want.

Create a file called `hello.py` with this content:

```
1 # hello.py - My first Python script
2 print("Hello from a script!")
3 print("This is more than one line.")
4 result = 6 * 7
5 print("Six times seven is", result)
```

Note

In the last line, we pass two items to `print()`, separated by a comma. Python automatically puts a space between them. This is a simple way to combine text with variable values. In Chapter 2, you'll learn about *f-strings*, a more powerful approach for formatting output.

Run it from the terminal:

```
1 python hello.py           # Windows
2 python3 hello.py         # macOS/Linux
```

Output:

```
1 Hello from a script!
2 This is more than one line.
3 Six times seven is 42
```

Scripts are how you'll write most of your code. They're saved, editable, and shareable.

1.8.3. Jupyter Notebooks

Jupyter notebooks mix code, output, and explanatory text in one document. They're popular for data exploration and creating reports.

Install Jupyter in your virtual environment:

```
1 pip install jupyter
```

Launch it:

```
1 jupyter notebook
```

A browser window opens with the Jupyter interface. You can create notebooks, write code in cells, and run them interactively.

Notebooks are excellent for:

- Exploratory data analysis
- Creating reports with embedded figures
- Teaching and documentation

Less ideal for:

- Large, complex programs
- Code that runs automatically (without human interaction)
- Version control with Git

We'll mostly use scripts in this book because they teach better programming habits. But know that notebooks exist—you'll encounter them in the wild.

1.9. Choosing an Editor

You need a text editor to write scripts. Not Microsoft Word—that's a word processor that adds invisible formatting codes. You need an editor that saves *plain text*: exactly what you type, nothing more.

1.9.1. Start Simple: Basic Text Editors

The best way to start is with a simple text editor. This might seem primitive, but it teaches you something important: Python scripts are just text files. You don't need special software to write Python—any program that saves plain text will work. This matters because it means your code isn't trapped in any particular tool.

- **Windows:** The built-in Notepad works, though Notepad++ is better (free download from <https://notepad-plus-plus.org/>). Notepad++ has “syntax highlighting”—it colours different parts of your code differently, making it easier to read.
- **macOS:** TextEdit works if you set it to plain text mode (Format > Make Plain Text). BBEdit (free version at <https://www.barebones.com/products/bbedit/>) is friendlier.
- **Linux:** gedit, kate, or your distribution's default text editor.

1.9.2. Upgrading Later: Integrated Development Environments

Tip

If you are totally new to Python, you can reduce the information overload by skipping this section for now—a good time to come back is after Chapter 5. Don't worry, I'll remind you!

I promise you're not missing out; the opposite is true. I recommend starting with text editors and never giving them up entirely. IDEs bring a lot to the table, but they take a lot as well.

If you already know your way around an IDE and disagree, then consider this tip a ponder block. Begin by asking yourself if the friction of a no frills text editor might actually help **weaponize your divine laziness**.

An IDE (Integrated Development Environment) is an editor designed specifically for programming. It provides helpful features: syntax highlighting, automatic error detection, tools to explore variables while your code runs, and more. When you're ready to upgrade from a basic text editor, here are some popular options.

Spyder: Designed specifically for scientists and data analysts using Python. If you've used RStudio, Spyder will feel familiar—it has a similar layout panels to show your script, an interactive console, a variable explorer, help files, a plot panel, and so on.

You could download it from <https://www.spyder-ide.org/> and install it using familiar pathways. Alternatively, check out the install guide <https://docs.spyder-ide.org/current/installation.html> for options such as installing using the build in package manager pip.

Visual Studio Code (VS Code): Made by Microsoft, free, and extremely popular in the broader software development community (though not required). It is a general-purpose programming editor used for many languages. Download from <https://code.visualstudio.com/>. After installing, you'll need to add the Python extension: click the Extensions icon (four squares) in the sidebar, search for “Python,” and install it.

No IDE: A plain old text editor like `nano`, `micro`, `vim`, or Notepad++ is perfectly serviceable. In fact, many scientists eventually work on remote supercomputers or cloud servers where graphical interfaces (like Spyder) aren't available. The ultimate IDE is your own mind. So, if you are comfortable editing plain text, you can work anywhere—on a powerful cluster, a colleague's laptop, or even on paper while commuting. The power lies in your abstractions made code, not the tool used to write it down. Challenge yourself to text-editor-only projects from time to time.

Note

VS Code is a generalist tool, whereas Spyder is a specialist. VS Code requires significant manual configuration to match Spyder's out-of-the-box scientific features. I recommend holding off on VS Code until you encounter a specific need that Spyder cannot fulfil.

1.10. Chapter Summary

You now have:

- Python installed and verified
- A terminal where you can type commands to your computer
- Understanding of virtual environments and packages (and why they matter)
- Knowledge of three ways to run Python: interactive mode (REPL), scripts, and notebooks
- A text editor for writing code (and knowledge of Spyder for later)

Before moving on, make sure you can:

1. Open your terminal and start Python
2. Create and activate a virtual environment
3. Install a package with pip
4. Create a `.py` file and run it from the terminal

If any of these don't work, debug now. Search for your error message online. The setup must work before you can learn programming.

Things to Ponder

Why do we use virtual environments instead of installing packages globally? What problem would arise if two of your projects needed different versions of NumPy?

Part II.

Python Fundamentals

Chapter 2.

Python Basics

With Python installed, let's learn the language. We call Python a “programming language” because, like English or German, it has vocabulary (words with specific meanings), grammar (rules for combining words), and syntax (rules for punctuation and structure). The difference is that Python is designed to be translated into instructions a computer can execute, while still being readable by humans. This is what makes programming accessible: you write something that looks vaguely like English, and the computer knows what to do.

This chapter covers the fundamental building blocks: how to store values, what types of data exist, and how to combine them into *expressions* (pieces of code that produce a value, like `2 + 3`). Everything else builds on this foundation.

Open Python in interactive mode (type `python` or `python3` in your terminal). We'll experiment in the REPL first, then move to scripts.

2.1. Python as a Calculator

At its simplest, Python is a calculator. Type an expression, get a result:

```
1  >>> 2 + 3
2  5
3  >>> 10 - 4
4  6
5  >>> 7 * 8
6  56
7  >>> 15 / 4
8  3.75
```

The four basic operations work as expected. A few notes on the syntax (the punctuation rules of the language):

- `*` is multiplication (not $\times$, which isn't on your keyboard)
- `/` is division (always gives a decimal result in Python 3)
- Spaces around operators are optional but recommended for readability

2.1.1. More Operations

Python has additional arithmetic operators:

```
1  >>> 15 // 4      # integer division (floor division)
2  3
3  >>> 15 % 4       # modulo (remainder after division)
4  3
5  >>> 2 ** 10      # exponentiation (2 to the power of 10)
6  1024
```

Integer division (`//`) discards the decimal part. Modulo (`%`) gives the remainder. Exponentiation (`**`) raises a number to a power.

Lab Example

Molarity calculation: You dissolved 5.85 grams of NaCl (molar mass 58.44 g/mol) in water and diluted to 500 mL. What's the concentration?

```
1 >>> mass_g = 5.85
2 >>> molar_mass = 58.44
3 >>> volume_L = 0.5
4 >>> concentration = mass_g / molar_mass / volume_L
5 >>> concentration
6 0.20020533881587955
```

About 0.2 M. We'll learn how to format this nicely soon.

2.1.2. Order of Operations

Python follows standard mathematical precedence: parentheses first, then exponentiation, then multiplication/division, then addition/subtraction.

```
1 >>> 2 + 3 * 4
2 14
3 >>> (2 + 3) * 4
4 20
5 >>> 2 ** 3 ** 2          # exponentiation is right-associative
6 512
7 >>> (2 ** 3) ** 2
8 64
```

When in doubt, use parentheses. They make your intent clear and prevent mistakes.

2.2. Variables and Assignment

Typing numbers directly gets tedious. Variables let you store values and refer to them by name.

```
1 >>> temperature = 298.15
2 >>> pressure = 101325
3 >>> temperature
4 298.15
5 >>> pressure
6 101325
```

The = sign is assignment: it stores the value on the right in the name on the left. This is not a mathematical equation—it's an instruction to put a value somewhere.

```
1 >>> x = 5
2 >>> x = x + 1          # take x, add 1, store result back in x
3 >>> x
4 6
```

That second line would be nonsense in mathematics, but in programming it's common: update a variable based on its current value.

2.2.1. Naming Rules

Variable names must:

- Start with a letter or underscore
- Contain only letters, numbers, and underscores

- Not be a Python keyword (like `if`, `for`, `class`)

Valid names: `temperature`, `sample_1`, `_private`, `CamelCase`, `my_class_of_thin`

Invalid names: `1st_sample` (starts with number), `my-variable` (contains hyphen), `class` (keyword)

Tip

Python convention: use `lowercase_with_underscores` for variable names. Reserve `CamelCase` for class names (covered later). This isn't required, but following conventions makes your code easier for others to read. While you can use python keywords in variable names, you should not use them as a variable name e.g. `class = "something"` (bad) vs `my_class = "something"` (fine). As you can see, with syntax highlighting enabled, you will at least get a warning that something is up, so don't worry about having to know all keywords in order to avoid them as variable names.

2.2.2. Meaningful Names

Compare:

```
1  >>> a = 5.85
2  >>> b = 58.44
3  >>> c = 0.5
4  >>> d = a / b / c
```

versus:

```
1  >>> mass_g = 5.85
2  >>> molar_mass = 58.44
3  >>> volume_L = 0.5
4  >>> concentration_M = mass_g / molar_mass / volume_L
```

Both calculate the same result, but the second version explains itself. Anyone reading it (including future you) understands what's happening.

Warning

Avoid single-letter variable names except in very short code blocks or when they match mathematical notation you're implementing. The characters you save aren't worth the confusion.

Warning

Avoid "Magic Numbers" In the example above, if we had written `conc = 5.85 / 58.44 / 0.5`, those numbers are called "Magic Numbers"—unexplained constants that appear out of nowhere.

To a programmer, a number without a name is dangerous. Is 0.5 the volume? The concentration? A correction factor? By assigning `volume_L = 0.5`, you give that number semantic meaning. You aren't just doing arithmetic; you are defining the physics of your simulation.

2.3. Data Types

Not all data is the same. Numbers behave differently from text, and Python treats them differently. The kind of data a value holds is called its *type*. Understanding types matters because operations that work on one type might not work on another—you can divide numbers, but you can't divide text.

2.3.1. Integers (int)

Whole numbers without decimal points:

```
1 >>> count = 42
2 >>> electrons = -1
3 >>> avogadro = 6022140760000000000000000
4 >>> type(count)
5 <class 'int'>
```

The `type()` function tells you what type a value is. You give it a value (or a variable containing a value), and it tells you the type. Python integers can be arbitrarily large—no overflow.

2.3.2. Floating-Point Numbers (float)

Numbers with decimal points:

```
1 >>> pi = 3.14159
2 >>> temperature = 298.15
3 >>> tiny = 1.23e-10      # scientific notation: 1.23 times 10^-10
4 >>> type(pi)
5 <class 'float'>
```

Scientific notation uses `e` for “times ten to the power of.” So `1.23e-10` is 1.23×10^{-10} .

Warning

Floats have limited precision. They are stored in binary, and some decimal numbers cannot be represented exactly, resulting in errors accruing over operations:

```
1 >>> 0.1 + 0.2
2 0.30000000000000004
```

This isn’t a Python bug—it’s how computers work. This matters because checking for exact equality will return `False` when you expect `True`:

```
1 >>> total = 0.1 + 0.2
2 >>> total == 0.3
3 False
4 >>> round(total, 5) == 0.3  # Safer way: round before comparing
5 True
```

2.3.3. Strings (str)

Text data is called a “string”—think of it as a string of characters, like beads on a necklace. Strings are enclosed in quotes:

```
1 >>> element = "Carbon"
2 >>> symbol = 'C'          # single or double quotes both work
3 >>> message = "It's alive!" # use double quotes to include apostrophe
4 >>> formula = 'H2O'
5 >>> type(element)
6 <class 'str'>
```

Strings can contain any characters. If your text contains single or double quotes, use the other kind of quote to delimit it, or escape with a backslash:

```
1 >>> quote = "She said \"hello\""
2 >>> quote
3 'She said "hello"'
```

Multiline strings use triple quotes:

```
1  >> long_text = """We have a
2  ... multiline
3  ... string"""
4  >> print(long_text)
5  We have a
6  multiline
7  string
```

2.3.4. Booleans (bool)

True or false values:

```
1  >> is_reactive = True
2  >> is_stable = False
3  >> type(is_reactive)
4  <class 'bool'>
```

Note the capitalization: `True` and `False`, not `true` or `TRUE`. Booleans are essential for making decisions in code (Covered further in Section 4).

2.3.5. None

A special type representing “no value” or “nothing”:

```
1  >> result = None
2  >> type(result)
3  <class 'NoneType'>
```

You’ll encounter `None` when a function doesn’t return anything, or when you want to initialize a variable without giving it a real value yet.

2.4. Type Checking

Sometimes you need to verify what type a value is:

```
1  >> x = 42
2  >> type(x)
3  <class 'int'>
4  >> type(x) == int
5  True
```

The `isinstance()` function is often better—it handles something called inheritance (Covered further once we get to Section 8.7):

```
1  >> isinstance(x, int)
2  True
3  >> isinstance(x, float)
4  False
5  >> isinstance(x, (int, float))    # check multiple types
6  True
```

2.4.1. Type Conversion

Convert between types using the type name as a function:

```

1  >>> int(3.7)           # float to int (truncates, doesn't round)
2  3
3  >>> float(42)          # int to float
4  42.0
5  >>> str(42)            # int to string
6  '42'
7  >>> int("42")          # string to int
8  42
9  >>> float("3.14")      # string to float
10 3.14

```

Warning

Conversion can fail if the value doesn't look like a number:

```

1  >>> int("hello")
2  ValueError: invalid literal for int() with base 10: 'hello'

```

Python cannot guess that "hello" is not a number. Similarly, `float("1.5 kg")` will fail because of the unit "kg". The string must contain *only* numbers.

Tip

The Helping Hand When Python crashes and a wall of red text appears, do not panic. Your instinct might be to flinch, interpreting the error as a strike against your competence.

I urge you to view it differently: see it not as a hand raised to strike, but as a hand reaching out to pull you up. Great programmers don't write code that never breaks to error is human. Instead, they are simply willing to take the hand that is offered. The traceback is a specific guide on how to resolve the misunderstanding between your intent and the computer's strict logic.

Things to Ponder

The Sound of Silence Consider the alternative. Imagine if there were no error messages. Imagine if you made a typo, or divided by zero, and the computer simply guessed what you meant or returned a random result without complaint.

You would be forced to manually verify every single intermediate calculation with a calculator before you could hope to trust the final output. That would be a true nightmare. The loud, abrasive error message is the only reason we can trust the silence of a successful run.

2.5. Operators and Expressions

You've seen arithmetic operators. Python has several other types.

2.5.1. Comparison Operators

These compare values and return booleans:

```

1  >>> 5 > 3
2  True
3  >>> 5 < 3
4  False
5  >>> 5 == 5           # equality (note: two equals signs)
6  True

```

```

7  >> 5 != 3      # not equal
8  True
9  >> 5 >= 5      # greater than or equal
10 True
11 >> 5 <= 4      # less than or equal
12 False

```

Warning

A common mistake: using `=` (assignment) when you mean `==` (comparison). They're very different:

```

1  >> x = 5      # assigns 5 to x
2  >> x == 5     # asks "is x equal to 5?"
3  True

```

2.5.2. Logical Operators

Combine boolean values:

```

1  >> True and False
2  False
3  >> True or False
4  True
5  >> not True
6  False

```

`and` is true only if both sides are true. `or` is true if at least one side is true. `not` inverts the value.

```

1  >> temperature = 298
2  >> pressure = 101325
3  >> is_standard_conditions = (temperature == 298) and (pressure ==
4  101325)
5  >> is_standard_conditions
6  True

```

2.5.3. String Operators

Strings have their own operators:

```

1  >> "Hello" + " " + "World"    # concatenation
2  'Hello World'
3  >> "Na" * 8                    # repetition
4  'NaNaNaNaNaNaNaNaNaN'
5  >> "H" in "Hello"             # membership test
6  True
7  >> "x" in "Hello"
8  False

```

2.5.4. Compound Assignment

Shortcuts for updating a variable:

```

1  >> x = 10
2  >> x += 5      # same as x = x + 5
3  >> x

```

```

4 15
5 >>> x -= 3      # same as x = x - 3
6 >>> x
7 12
8 >>> x *= 2      # same as x = x * 2
9 >>> x
10 24

```

These work with `+=`, `-=`, `*=`, `/=`, and others.

2.6. Comments

Comments are notes in your code that Python ignores. They explain what your code does (or why).

```

1 # This is a comment - Python ignores it
2 mass = 5.85 # grams of NaCl
3
4 # Calculate concentration in mol/L
5 # Using M = n/V where n = mass/molar_mass
6 molar_mass = 58.44 # g/mol for NaCl
7 volume = 0.5      # liters
8 concentration = mass / molar_mass / volume

```

Everything after `#` on a line is a comment.

Tip

Good comments explain *why*, not *what*. The code itself shows what's happening; comments should clarify the reasoning:

```

1 # Bad comment: adds 1 to x
2 x = x + 1
3
4 # Good comment: adjust for 1-based indexing in the output file
5 x = x + 1

```

Things to Ponder

Coding is Time Travel The primary audience for your code is not the computer; the computer will execute garbage instructions without complaint. The primary audience is *humans*—specifically, you, six months from now.

"Future You" will be tired, busy, and will have completely forgotten why you divided by 5.85. Writing clean code and helpful comments is an act of kindness to your future self. Start thinking about how to effectively communicate with the future. It's an acquired skill like many others, but it is a very valuable one.

2.7. Formatted Output with f-strings

The `print()` function displays output:

```

1 >>> print("Hello")
2 Hello
3 >>> print(42)
4 42

```

```
5 >> print("The answer is", 42)
6 The answer is 42
```

But what if you want to include variable values in a sentence? f-strings (formatted string literals) make this easy:

```
1 >> temperature = 298.15
2 >> print(f"Temperature: {temperature} K")
3 Temperature: 298.15 K
```

The `f` before the opening quote marks it as an f-string. Inside, curly braces `{}` contain expressions that get evaluated and inserted:

```
1 >> name = "Carbon"
2 >> atomic_number = 6
3 >> mass = 12.011
4 >> print(f"{name} has atomic number {atomic_number}")
5 Carbon has atomic number 6
```

2.7.1. Formatting Numbers

f-strings can format numbers with precision:

```
1 >> pi = 3.141592653589793
2 >> print(f"Pi is approximately {pi:.2f}")
3 Pi is approximately 3.14
4 >> print(f"Pi is approximately {pi:.4f}")
5 Pi is approximately 3.1416
```

The `:.2f` means “format as a float with 2 decimal places.” Common format specifiers:

- `:.3f` — 3 decimal places
- `:.2e` — scientific notation with 2 decimal places
- `:>10` — right-align in 10 characters
- `:`, — add thousands separators

```
1 >> avogadro = 6.022e23
2 >> print(f"Avogadro's number: {avogadro:.3e}")
3 Avogadro's number: 6.022e+23
4 >> population = 8000000000
5 >> print(f"World population: {population:,}")
6 World population: 8,000,000,000
```

Lab Example

Reporting a concentration with appropriate precision:

```
1 >> concentration = 0.20020533881587955
2 >> print(f"Concentration: {concentration:.3f} M")
3 Concentration: 0.200 M
4 >> print(f"Concentration: {concentration:.2e} mol/L")
5 Concentration: 2.00e-01 mol/L
```

2.8. Getting User Input

The `input()` function asks the user to type something:

```
1 >>> name = input("Enter your name: ")
2 Enter your name: Alice
3 >>> print(f"Hello, {name}!")
4 Hello, Alice!
```

Warning

`input()` always returns a string, even if the user types a number:

```
1 >>> age = input("Enter your age: ")
2 Enter your age: 25
3 >>> type(age)
4 <class 'str'>
5 >>> age + 1
6 TypeError: can only concatenate str (not "int") to str
```

Convert to the right type:

```
1 >>> age = int(input("Enter your age: "))
2 Enter your age: 25
3 >>> age + 1
4 26
```

For scientific work, you'll rarely use `input()`—data comes from files instead. But it's useful for quick interactive scripts, and some who read this will find use for it.

2.9. Writing Your First Script

Let's combine what we've learned into a script. Create a file called `molarity.py`. You could do this any number of ways; easiest is likely to open your editor of choice and save a file of this name in an appropriate location (recall hygiene rules from Chapter 0):

```
1 # molarity.py - Calculate solution concentration
2 # This script calculates molarity from mass, molar mass, and volume.
3
4 # Input values
5 mass_g = 5.85           # mass of solute in grams
6 molar_mass = 58.44      # molar mass in g/mol (NaCl)
7 volume_L = 0.5          # solution volume in liters
8
9 # Calculate moles and concentration
10 moles = mass_g / molar_mass
11 concentration_M = moles / volume_L
12
13 # Display results
14 print("Molarity Calculation")
15 print("=" * 30)
16 print(f"Mass of solute:      {mass_g:.2f} g")
17 print(f"Molar mass:          {molar_mass:.2f} g/mol")
18 print(f"Volume:                {volume_L:.3f} L")
19 print(f"Moles of solute:        {moles:.4f} mol")
20 print(f"Concentration:         {concentration_M:.3f} M")
```


Ensuring you've used `cd`, meaning "change directory", to navigate your terminal into the right path. Then run your script by simply entering:

```
1 python molarity.py
```

Output:

```
1 Molarity Calculation
2 =====
3 Mass of solute:      5.85 g
4 Molar mass:         58.44 g/mol
5 Volume:             0.500 L
6 Moles of solute:    0.1001 mol
7 Concentration:      0.200 M
```

Notice how the script:

- Starts with a comment explaining its purpose
- Uses descriptive variable names
- Separates input, calculation, and output sections
- Formats numbers appropriately for the context

This is good scientific programming style.

As a personal challenge, try to make an interactive molarity calculator by combining what we have covered so far, such as the use of `input()`.

2.10. Chapter Summary

You've learned the fundamentals:

- **Arithmetic:** `+`, `-`, `*`, `/`, `//`, `%`, `**`
- **Variables:** store values with meaningful names using `=`
- **Types:** `int`, `float`, `str`, `bool`, `None`
- **Type checking:** `type()`, `isinstance()`
- **Type conversion:** `int()`, `float()`, `str()`
- **Comparisons:** `==`, `!=`, `<`, `>`, `<=`, `>=`
- **Logical operators:** `and`, `or`, `not`
- **Comments:** `# this is ignored`
- **f-strings:** `f"Value: {x:.2f}"`
- **Output:** `print()`
- **Input:** `input()` (returns string)

Things to Ponder

1. Why does Python have both `/` and `//` for division? When would you use each?
2. What happens if you try to use a variable before assigning it a value?
3. Why is `0.1 + 0.2 != 0.3`? Is this a problem for scientific calculations?
4. What's the difference between `print(x)` and just typing `x` in the REPL?

Chapter 3.

Container Types

So far, each variable holds one value. But scientific data comes in groups: a series of measurements, a collection of samples, a table of properties. Python's container types hold multiple values in a single variable. This chapter covers the four main ones: lists, tuples, dictionaries, and sets.

3.1. Lists

A list is an ordered collection of items. Create one with square brackets:

```
1  >>> temperatures = [20.1, 21.3, 19.8, 22.0, 20.5]
2  >>> temperatures
3  [20.1, 21.3, 19.8, 22.0, 20.5]
4  >>> elements = ["H", "He", "Li", "Be", "B"]
5  >>> elements
6  ['H', 'He', 'Li', 'Be', 'B']
```

Lists can contain any type of data, and items don't have to be the same type (though they usually are):

```
1  >>> mixed = [1, "hello", 3.14, True]
2  >>> mixed
3  [1, 'hello', 3.14, True]
```

3.1.1. Indexing

Access individual items by position using square brackets. The position number is called the *index*. Python uses zero-based indexing—the first item is at index 0, not index 1. This is a common convention in programming (though not universal), so get used to it:

```
1  >>> elements = ["H", "He", "Li", "Be", "B"]
2  >>> elements[0]      # first item
3  'H'
4  >>> elements[1]      # second item
5  'He'
6  >>> elements[4]      # fifth item (last in this list)
7  'B'
8  >>> elements[-1]     # last item (negative indexing)
9  'B'
10 >>> elements[-2]     # second to last
11 'Be'
```

Warning

Accessing an index that doesn't exist causes an error:

```
1 >>> elements[10]
2 IndexError: list index out of range
```

Always check your list length if you're unsure.

3.1.2. Slicing

Extract a sublist using slice notation `[start:stop]`:

```
1 >>> numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2 >>> numbers[2:5]      # items from index 2 up to (not including) 5
3 [2, 3, 4]
4 >>> numbers[:4]       # from beginning to index 4
5 [0, 1, 2, 3]
6 >>> numbers[6:]       # from index 6 to end
7 [6, 7, 8, 9]
8 >>> numbers[::2]      # every second item (step of 2)
9 [0, 2, 4, 6, 8]
10 >>> numbers[::-1]    # reversed
11 [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

The pattern is `[start:stop:step]`. Any part can be omitted:

- Omit `start`: Defaults to begin at 0
- Omit `stop`: Defaults to go to the end
- Omit `step`: Defaults to use step of 1

Note

The `stop` index is exclusive—the slice includes items up to but not including that index. This feels odd at first but makes arithmetic cleaner: `numbers[0:5]` contains exactly 5 items.

Things to Ponder

If we have some list `numbers`, we get every value back if we use `numbers[0:-1:1]`. Why does `numbers[::-1]` reverse this list?

3.1.3. List Length

The `len()` function returns the number of items:

```
1 >>> elements = ["H", "He", "Li", "Be", "B"]
2 >>> len(elements)
3 5
4 >>> len([])      # empty list
5 0
```

3.1.4. Modifying Lists

Lists are *mutable*, meaning you can change them after creation. (“Mutable” comes from the same root as “mutation”—things that can change.) You can modify individual items, add new ones, or remove existing ones.

Change an item:

```

1  >> temps = [20.0, 21.0, 22.0]
2  >> temps[1] = 21.5
3  >> temps
4  [20.0, 21.5, 22.0]

```

Add items:

Here we'll use *methods*—functions that belong to a specific value. You call them using a dot: `temps.append(23.0)` means “call the `append` function that belongs to `temps`.” Methods are actions that the value knows how to perform on itself.

```

1  >> temps.append(23.0)           # add to end
2  >> temps
3  [20.0, 21.5, 22.0, 23.0]
4  >> temps.insert(0, 19.0)       # insert at position 0
5  >> temps
6  [19.0, 20.0, 21.5, 22.0, 23.0]
7  >> temps.extend([24.0, 25.0])  # add multiple items
8  >> temps
9  [19.0, 20.0, 21.5, 22.0, 23.0, 24.0, 25.0]

```

Remove items:

```

1  >> temps.pop()                 # remove and return last item
2  25.0
3  >> temps
4  [19.0, 20.0, 21.5, 22.0, 23.0, 24.0]
5  >> temps.pop(0)               # remove and return item at index 0
6  19.0
7  >> temps
8  [20.0, 21.5, 22.0, 23.0, 24.0]
9  >> temps.remove(21.5)         # remove first occurrence of value
10 >> temps
11 [20.0, 22.0, 23.0, 24.0]

```

Warning

`append()` and `extend()` look similar but differ:

```

1  >> a = [1, 2, 3]
2  >> a.append([4, 5])           # adds the list as one item
3  >> a
4  [1, 2, 3, [4, 5]]
5  >> b = [1, 2, 3]
6  >> b.extend([4, 5])          # adds each item individually
7  >> b
8  [1, 2, 3, 4, 5]

```

3.1.5. List Operations

```

1  >> [1, 2, 3] + [4, 5, 6]     # concatenation
2  [1, 2, 3, 4, 5, 6]
3  >> [1, 2] * 3                 # repetition
4  [1, 2, 1, 2, 1, 2]
5  >> 3 in [1, 2, 3, 4, 5]      # membership test
6  True
7  >> 10 in [1, 2, 3, 4, 5]

```

```
8 False
```

3.1.6. Useful List Methods

```
1 >>> numbers = [3, 1, 4, 1, 5, 9, 2, 6]
2 >>> numbers.count(1)           # how many 1s?
3 2
4 >>> numbers.index(5)           # position of first 5
5 4
6 >>> numbers.sort()             # sort in place
7 >>> numbers
8 [1, 1, 2, 3, 4, 5, 6, 9]
9 >>> numbers.reverse()          # reverse in place
10 >>> numbers
11 [9, 6, 5, 4, 3, 2, 1, 1]
```

For a sorted copy without modifying the original, use `sorted()`:

```
1 >>> original = [3, 1, 4, 1, 5]
2 >>> sorted_copy = sorted(original)
3 >>> sorted_copy
4 [1, 1, 3, 4, 5]
5 >>> original                     # unchanged
6 [3, 1, 4, 1, 5]
```

Lab Example

Processing a list of absorbance measurements:

```
1 >>> absorbances = [0.452, 0.487, 0.523, 0.478, 0.501]
2 >>> print(f"Number of measurements: {len(absorbances)}")
3 Number of measurements: 5
4 >>> print(f"First reading: {absorbances[0]}")
5 First reading: 0.452
6 >>> print(f"Last reading: {absorbances[-1]}")
7 Last reading: 0.501
8 >>> print(f"Min: {min(absorbances)}, Max: {max(absorbances)}")
9 Min: 0.452, Max: 0.523
10 >>> print(f"Average: {sum(absorbances) / len(absorbances):.3f}")
11 Average: 0.488
```

The built-in functions `min()`, `max()`, and `sum()` work on lists of numbers.

3.2. Tuples

A tuple is like a list, but *immutable*—once created, you can’t change it. (“Immutable” is the opposite of mutable: it cannot be mutated or changed.) This might seem like a limitation, but it’s often useful. Create tuples with parentheses:

```
1 >>> coordinates = (3.5, 2.1, 7.8)
2 >>> coordinates
3 (3.5, 2.1, 7.8)
4 >>> rgb = (255, 128, 0)
5 >>> rgb
6 (255, 128, 0)
```

Access items the same way as lists:

```
1  >>> coordinates[0]
2  3.5
3  >>> coordinates[1:]
4  (2.1, 7.8)
5  >>> len(coordinates)
6  3
```

But you cannot modify them:

```
1  >>> coordinates[0] = 4.0
2  TypeError: 'tuple' object does not support item assignment
```

3.2.1. Why Use Tuples?

If tuples are just restricted lists, why bother? Several reasons:

1. **Intent:** A tuple signals that the data shouldn't change. Coordinates of a point, RGB values of a color—these are fixed groupings.
2. **Dictionary keys:** Tuples can be dictionary keys; lists cannot (we'll see why soon).
3. **Performance:** Tuples are slightly faster and use less memory (rarely matters in practice).
4. **Unpacking:** Tuples are often used for returning multiple values from functions.

3.2.2. Tuple Packing and Unpacking

Python lets you assign multiple variables at once:

```
1  >>> point = (3.5, 2.1, 7.8)
2  >>> x, y, z = point          # unpacking
3  >>> x
4  3.5
5  >>> y
6  2.1
7  >>> z
8  7.8
```

This also works without explicit parentheses:

```
1  >>> a, b = 1, 2              # tuple packing on right, unpacking on
    left
2  >>> a
3  1
4  >>> b
5  2
6  >>> a, b = b, a             # swap values!
7  >>> a
8  2
9  >>> b
10 1
```

Note

A single-element tuple needs a trailing comma:

```
1  >> not_a_tuple = (42)           # just a number in parentheses
2  >> type(not_a_tuple)
3  <class 'int'>
4  >> actual_tuple = (42,)        # trailing comma makes it a tuple
5  >> type(actual_tuple)
6  <class 'tuple'>
```

3.3. Dictionaries

A dictionary stores key-value pairs. Instead of accessing items by position, you access them by name:

```
1  >> element = {"name": "Carbon", "symbol": "C", "atomic_number": 6}
2  >> element["name"]
3  'Carbon'
4  >> element["atomic_number"]
5  6
```

Create dictionaries with curly braces. Each entry has a key, a colon, and a value. Keys are typically strings, but can be any immutable type.

3.3.1. Adding and Modifying

```
1  >> element["mass"] = 12.011     # add new key-value pair
2  >> element
3  {'name': 'Carbon', 'symbol': 'C', 'atomic_number': 6, 'mass': 12.011}
4  >> element["mass"] = 12.0      # modify existing value
5  >> element["mass"]
6  12.0
```

3.3.2. Accessing Values Safely

Accessing a missing key raises an error:

```
1  >> element["electrons"]
2  KeyError: 'electrons'
```

Use `.get()` to provide a default instead:

```
1  >> element.get("electrons")     # returns None if missing
2  >> element.get("electrons", "unknown") # returns "unknown" if missing
3  'unknown'
4  >> element.get("name", "unknown") # returns actual value if present
5  'Carbon'
```

Check if a key exists:

```
1  >> "name" in element
2  True
3  >> "electrons" in element
4  False
```


3.3.3. Dictionary Methods

```
1 >>> element.keys()           # all keys
2 dict_keys(['name', 'symbol', 'atomic_number', 'mass'])
3 >>> element.values()         # all values
4 dict_values(['Carbon', 'C', 6, 12.0])
5 >>> element.items()          # key-value pairs as tuples
6 dict_items([('name', 'Carbon'), ('symbol', 'C'), ...])
```

Remove entries:

```
1 >>> element.pop("mass")      # remove and return value
2 12.0
3 >>> element
4 {'name': 'Carbon', 'symbol': 'C', 'atomic_number': 6}
5 >>> del element["symbol"]    # remove without returning
6 >>> element
7 {'name': 'Carbon', 'atomic_number': 6}
```

3.3.4. Nested Dictionaries

Values can be any type, including other dictionaries:

```
1 >>> periodic_table = {
2 ...     "H": {"name": "Hydrogen", "mass": 1.008, "number": 1},
3 ...     "He": {"name": "Helium", "mass": 4.003, "number": 2},
4 ...     "Li": {"name": "Lithium", "mass": 6.941, "number": 3}
5 ... }
6 >>> periodic_table["H"]["mass"]
7 1.008
8 >>> periodic_table["Li"]["name"]
9 'Lithium'
```

Lab Example

Storing experimental conditions:

```
1 >>> experiment = {
2 ...     "sample_id": "RXN-2024-001",
3 ...     "temperature_K": 298.15,
4 ...     "pressure_atm": 1.0,
5 ...     "solvent": "water",
6 ...     "concentrations_M": {
7 ...         "NaCl": 0.1,
8 ...         "HCl": 0.01
9 ...     },
10 ...     "notes": "Standard conditions"
11 ... }
12 >>> print(f"Sample: {experiment['sample_id']}")
13 Sample: RXN-2024-001
14 >>> print(f"NaCl conc: {experiment['concentrations_M']['NaCl']} M")
15 NaCl conc: 0.1 M
```

Dictionaries are excellent for structured data where items have meaningful names.

A set is an unordered collection of unique elements. Sets automatically remove duplicates:

The order may vary—sets are unordered.

Sets support mathematical set operations:

3.4.2. Membership Testing

This is much faster than lists for large collections.

```
1 >> elements = {"H", "C", "N"}
2 >> elements.add("O")           # add one element
3 >> elements
4 {'O', 'C', 'N', 'H'}
5 >> elements.update(["S", "P"]) # add multiple
6 >> elements
7 {'S', 'O', 'C', 'P', 'N', 'H'}
8 >> elements.remove("S")        # remove (error if missing)
9 >> elements.discard("Fe")      # remove (no error if missing)
```

Note

To create an empty set, use `set()`, not `{}`:

```
1 >>> empty_dict = {}
2 >>> type(empty_dict)
3 <class 'dict'>
4 >>> empty_set = set()
5 >>> type(empty_set)
6 <class 'set'>
```

Curly braces without colons create an empty dictionary, not a set.

Lab Example

Finding common elements between two samples:

```
1 >>> sample_a = {"Fe", "Cu", "Zn", "Pb", "Cd"}
2 >>> sample_b = {"Cu", "Ag", "Au", "Pb"}
3 >>> common = sample_a & sample_b
4 >>> print(f"Elements in both samples: {common}")
5 Elements in both samples: {'Cu', 'Pb'}
6 >>> only_in_a = sample_a - sample_b
7 >>> print(f"Only in sample A: {only_in_a}")
8 Only in sample A: {'Fe', 'Zn', 'Cd'}
```

3.5. Choosing the Right Container

With four container types, how do you choose?

Characteristic	List	Tuple	Dict	Set
Ordered	Yes	Yes	No*	No
Mutable	Yes	No	Yes	Yes
Duplicates allowed	Yes	Yes	Keys: No	No
Access by index	Yes	Yes	No	No
Access by key	No	No	Yes	No

Table 3.1.: Container type comparison. *Dicts preserve insertion order since Python 3.7, but aren't indexed by position.

Use a list when:

- Order matters
- You need to add, remove, or change items
- Items are accessed by position
- Example: time series of measurements

Use a tuple when:

- Order matters but data shouldn't change
- You're grouping related fixed values
- Example: (x, y, z) coordinates, RGB colors

Use a dictionary when:

- Items have meaningful names
- You need to look up values by key
- Data is structured with labeled fields
- Example: element properties, experiment metadata

Use a set when:

- You need unique elements only
- Order doesn't matter
- You're doing membership tests or set operations
- Example: elements present in a sample, unique identifiers

3.6. Converting Between Types

Convert between container types using their constructors:

```

1  >>> list((1, 2, 3))           # tuple to list
2  [1, 2, 3]
3  >>> tuple([1, 2, 3])         # list to tuple
4  (1, 2, 3)
5  >>> set([1, 2, 2, 3, 3, 3])   # list to set (removes duplicates)
6  {1, 2, 3}
7  >>> list({1, 2, 3})          # set to list
8  [1, 2, 3]
9  >>> list({"a": 1, "b": 2})    # dict to list (gets keys)
10 ['a', 'b']
11 >>> list({"a": 1, "b": 2}.values()) # dict values to list
12 [1, 2]
```

A common pattern: remove duplicates from a list while preserving some order:

```

1  >>> items = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
2  >>> unique = list(dict.fromkeys(items)) # preserves first occurrence
3  >>> unique
4  [3, 1, 4, 5, 9, 2, 6]
```

3.7. Nested Structures

Containers can hold other containers, creating complex data structures:

```

1  >>> # List of dictionaries (common for tabular data)
2  >>> samples = [
3  ...     {"id": "A1", "concentration": 0.1, "absorbance": 0.23},
4  ...     {"id": "A2", "concentration": 0.2, "absorbance": 0.45},
5  ...     {"id": "A3", "concentration": 0.3, "absorbance": 0.68},
6  ... ]
7  >>> samples[1]["absorbance"]
8  0.45
9
10 >>> # Dictionary with list values
11 >>> spectra = {
12 ...     "wavelengths": [400, 450, 500, 550, 600],
13 ...     "absorbances": [0.12, 0.34, 0.56, 0.43, 0.21]
14 ... }
15 >>> spectra["wavelengths"][2]
16 500
```

Lab Example

A complete experiment record:

```

1  >>> kinetics_run = {
2  ...     "reaction": "A + B -> C",
3  ...     "temperature_K": 298,
4  ...     "initial_concentrations": {"A": 0.1, "B": 0.2},
5  ...     "time_points_s": [0, 60, 120, 180, 240, 300],
6  ...     "concentration_A": [0.1, 0.082, 0.067, 0.055, 0.045, 0.037],
7  ...     "replicates": 3,
8  ...     "notes": ["Used fresh reagents", "Room temp stable"]
9  ... }
10 >>> # Access nested data
11 >>> initial_A = kinetics_run["initial_concentrations"]["A"]
12 >>> final_A = kinetics_run["concentration_A"][-1]
13 >>> print(f"[A] dropped from {initial_A} to {final_A} M")
14 [A] dropped from 0.1 to 0.037 M

```

3.8. Variables Are Labels, Not Boxes

Here's a mental model that will prevent many bugs: variables in Python are *labels* attached to objects, not boxes that contain values. When you write `x = 5`, you're not putting 5 into a box called `x`—you're attaching the label `x` to the number 5.

This distinction matters when you assign one variable to another:

```

1  >>> a = [1, 2, 3]
2  >>> b = a           # b is now another label for the SAME list
3  >>> b.append(4)
4  >>> a
5  [1, 2, 3, 4]        # a changed too!

```

This surprises many beginners. The line `b = a` doesn't copy the list—it creates a second label pointing to the same list in memory. Modifying through `b` affects `a` because they're the same object.

You can check if two variables refer to the same object using `is`:

```

1  >>> a = [1, 2, 3]
2  >>> b = a
3  >>> b is a
4  True                # same object
5  >>> c = [1, 2, 3]
6  >>> c is a
7  False               # different object (even though values are equal)
8  >>> c == a
9  True                # equal values, different objects

```

The `==` operator checks if values are equal. The `is` operator checks if they're the exact same object in memory.

Note

This aliasing behavior applies to *mutable* objects—things that can be changed after creation, like lists and dictionaries. For *immutable* objects like numbers, strings, and tuples, you can't modify them in place, so aliasing rarely causes problems. When you “change” a number, you're really creating a new number and re-attaching the label.

Making Actual Copies

To create an independent copy, use the `copy` module:

```
1 >>> import copy
2 >>> original = [1, 2, 3]
3 >>> shallow = copy.copy(original)
4 >>> shallow.append(4)
5 >>> original
6 [1, 2, 3]                # original unchanged
```

For nested structures (lists containing lists), you need `deepcopy`:

```
1 >>> original = [[1, 2], [3, 4]]
2 >>> shallow = copy.copy(original)
3 >>> deep = copy.deepcopy(original)
4
5 >>> shallow[0].append(99)
6 >>> original
7 [[1, 2, 99], [3, 4]]    # shallow copy shares inner lists!
8
9 >>> deep[1].append(100)
10 >>> original
11 [[1, 2, 99], [3, 4]]   # deep copy is fully independent
```

Lab Example

This aliasing trap commonly appears when processing experimental data:

```
1 # Dangerous: both variables point to same data
2 raw_data = [0.12, 0.25, 0.31, 0.42]
3 processed = raw_data          # NOT a copy!
4 processed[0] = 0              # "fix" first point
5 print(raw_data)               # [0, 0.25, 0.31, 0.42] - raw data
6                               # corrupted!
7
8 # Safe: make an explicit copy
9 import copy
10 raw_data = [0.12, 0.25, 0.31, 0.42]
11 processed = copy.copy(raw_data)
12 processed[0] = 0
13 print(raw_data)               # [0.12, 0.25, 0.31, 0.42] - preserved
```

Always copy data before modifying it if you need to preserve the original.

3.9. Chapter Summary

You've learned Python's four main container types:

- **Lists:** ordered, mutable sequences accessed by index
- **Tuples:** ordered, immutable sequences for fixed groupings
- **Dictionaries:** key-value pairs for structured data
- **Sets:** unordered collections of unique elements

Key operations:

- **Indexing:** `items[0]`, `items[-1]`

- **Slicing:** `items[1:4]`, `items[::2]`
- **Length:** `len(items)`
- **Membership:** `x in items`
- **List methods:** `append()`, `extend()`, `pop()`, `sort()`
- **Dict access:** `d["key"]`, `d.get("key", default)`
- **Set operations:** `|`, `&`, `-` (union, intersection, difference)

Things to Ponder

1. Why can't lists be dictionary keys, but tuples can?
2. You have a list of 1000 sample IDs and need to check if a new ID already exists. Would a list or set be faster for this? Why?
3. When would you use a list of dictionaries versus a dictionary of lists?
4. What happens if you try to access `my_list[len(my_list)]`? Why?

Chapter 4.

Control Flow

So far, Python executes your code line by line, top to bottom. But real programs need to make decisions and repeat actions. This chapter introduces *control flow*—the tools that let your code take different paths and loop through repetitive tasks.

4.1. Making Decisions with if

The `if` statement lets your code choose what to do based on a condition:

```
1 temperature = 373.15
2
3 if temperature > 373:
4     print("Water is boiling!")
```

The structure is: `if` followed by a condition (something that evaluates to `True` or `False`), then a colon. The code to run when the condition is true is *indented* on the next line(s).

4.1.1. Indentation Matters

In Python, indentation isn't just for readability—it's part of the syntax. The indented lines after `if` are called a *block*. Everything indented at the same level belongs to that block:

```
1 if temperature > 373:
2     print("Water is boiling!")
3     print("Be careful!")
4     print("This is also part of the if block")
5
6 print("This runs regardless - it's not indented")
```

Warning

Inconsistent indentation causes errors. Python doesn't care whether you use spaces or tabs, but you must be consistent. The standard convention is 4 spaces per indentation level. Most editors can be configured to insert 4 spaces when you press Tab.

4.1.2. if-else

What if you want to do something different when the condition is false?

```
1 pH = 6.5
2
3 if pH < 7:
4     print("Solution is acidic")
5 else:
6     print("Solution is basic or neutral")
```

The `else` block runs when the `if` condition is false.

4.1.3. if-elif-else

For multiple conditions, use `elif` (short for “else if”):

```
1 pH = 7.0
2
3 if pH < 7:
4     print("Acidic")
5 elif pH > 7:
6     print("Basic")
7 else:
8     print("Neutral")
```

Python checks conditions from top to bottom and runs the first block whose condition is true. If none are true, it runs `else` (if present).

```
1 score = 85
2
3 if score >= 90:
4     grade = "A"
5 elif score >= 80:
6     grade = "B"
7 elif score >= 70:
8     grade = "C"
9 elif score >= 60:
10    grade = "D"
11 else:
12    grade = "F"
13
14 print(f"Grade: {grade}")
```

Lab Example

Classifying a reaction based on Gibbs free energy:

```
1 delta_G = -15.3 # kJ/mol
2
3 if delta_G < 0:
4     print("Reaction is spontaneous (exergonic)")
5 elif delta_G > 0:
6     print("Reaction is non-spontaneous (endergonic)")
7 else:
8     print("Reaction is at equilibrium")
```

4.1.4. Nested Conditions

You can put `if` statements inside other `if` statements:

```
1 temperature = 25
2 pressure = 1.0
3
4 if temperature > 0:
5     if pressure > 0.5:
6         print("Conditions might support liquid water")
7     else:
8         print("Pressure too low for liquid")
9 else:
10    print("Too cold for liquid water")
```

Nesting works, but deep nesting becomes hard to read. Often you can use **and/or** instead:

```
1 if temperature > 0 and pressure > 0.5:
2     print("Conditions might support liquid water")
```

4.1.5. Truthiness

Python treats some values as “truthy” (act like **True**) and others as “falsy” (act like **False**):

- **Falsy:** **False**, **None**, 0, 0.0, "" (empty string), [] (empty list), {} (empty dict)
- **Truthy:** Everything else

This lets you write concise conditions:

```
1 data = [1.2, 3.4, 5.6]
2
3 if data: # True if list is not empty
4     print(f"Processing {len(data)} values")
5 else:
6     print("No data to process")
```

4.2. Repeating with for Loops

A **for** loop repeats code for each item in a sequence:

```
1 elements = ["H", "He", "Li", "Be", "B"]
2
3 for element in elements:
4     print(element)
```

Output:

```
1 H
2 He
3 Li
4 Be
5 B
```

The variable `element` takes on each value from the list, one at a time. The indented block runs once for each value.

4.2.1. Looping Over Ranges

The **range()** function generates a sequence of numbers:

```
1 for i in range(5):
2     print(i)
```

Output:

```
1 0
2 1
3 2
4 3
5 4
```

range(5) produces 0, 1, 2, 3, 4—five numbers starting from 0. Like slicing, the endpoint is exclusive.

range() can take start, stop, and step arguments:

```

1  >> list(range(2, 10))      # start at 2, stop before 10
2  [2, 3, 4, 5, 6, 7, 8, 9]
3  >> list(range(0, 10, 2))  # step by 2
4  [0, 2, 4, 6, 8]
5  >> list(range(10, 0, -1)) # count backwards
6  [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

```

Lab Example

Calculating reaction progress at time intervals:

```

1  k = 0.05 # rate constant (1/s)
2  C0 = 1.0 # initial concentration (M)
3
4  print("Time (s)   Concentration (M)")
5  print("-" * 30)
6
7  for t in range(0, 101, 20): # 0, 20, 40, 60, 80, 100
8      C = C0 * (2.718 ** (-k * t)) # first-order decay
9      print(f"{t:>5}      {C:.4f}")

```

4.2.2. Looping with Index: enumerate()

Sometimes you need both the item and its position. The `enumerate()` function provides both:

```

1  elements = ["H", "He", "Li", "Be", "B"]
2
3  for index, element in enumerate(elements):
4      print(f"{index}: {element}")

```

Output:

```

1  0: H
2  1: He
3  2: Li
4  3: Be
5  4: B

```

Start counting from 1 instead of 0:

```

1  for index, element in enumerate(elements, start=1):
2      print(f"{index}: {element}")

```

4.2.3. Looping Over Dictionaries

Looping over a dictionary gives you its keys:

```

1  atomic_masses = {"H": 1.008, "He": 4.003, "Li": 6.941}
2
3  for element in atomic_masses:
4      print(element) # prints H, He, Li

```

To get both keys and values, use `.items()`:

```

1  for element, mass in atomic_masses.items():
2      print(f"{element}: {mass} u")

```

To loop over just values:

```
1 for mass in atomic_masses.values():
2     print(mass)
```

4.2.4. Looping Over Multiple Lists: zip()

The `zip()` function pairs up items from multiple sequences:

```
1 wavelengths = [400, 450, 500, 550, 600]
2 absorbances = [0.12, 0.34, 0.56, 0.43, 0.21]
3
4 for wavelength, absorbance in zip(wavelengths, absorbances):
5     print(f"{wavelength} nm: A = {absorbance}")
```

Warning

If the sequences have different lengths, `zip()` stops at the shortest one. No error is raised—data from the longer sequence is silently ignored. Be careful!

4.3. Repeating with while Loops

A `while` loop repeats as long as a condition is true:

```
1 count = 0
2
3 while count < 5:
4     print(count)
5     count += 1
```

This prints 0, 1, 2, 3, 4—then stops because `count < 5` becomes false.

Warning

If the condition never becomes false, the loop runs forever (an “infinite loop”). Make sure something in the loop changes the condition. If you accidentally create an infinite loop, press Ctrl+C to stop it.

`while` loops are useful when you don’t know in advance how many iterations you need:

```
1 # Find when concentration drops below threshold
2 C = 1.0
3 k = 0.1
4 threshold = 0.1
5 t = 0
6
7 while C > threshold:
8     t += 1
9     C = C * (1 - k)
10
11 print(f"Concentration drops below {threshold} at t = {t}")
```

Lab Example

Simulating radioactive decay until activity drops below detection limit:

```

1 N = 1000000 # initial number of atoms
2 half_life = 5.27 # years (Co-60)
3 decay_constant = 0.693 / half_life
4 detection_limit = 100
5 t = 0
6 dt = 0.1 # time step in years
7
8 while N > detection_limit:
9     N = N * (2.718 ** (-decay_constant * dt))
10    t += dt
11
12 print(f"Below detection after {t:.1f} years")

```

4.4. Controlling Loops: break and continue

4.4.1. break: Exit the Loop Early

The **break** statement immediately exits the loop:

```

1 for i in range(100):
2     if i * i > 50:
3         print(f"First square > 50: {i}^2 = {i*i}")
4         break

```

Once **break** executes, the loop ends entirely.

4.4.2. continue: Skip to Next Iteration

The **continue** statement skips the rest of the current iteration and moves to the next:

```

1 for i in range(10):
2     if i % 2 == 0: # if i is even
3         continue # skip even numbers
4     print(i)      # only prints odd numbers

```

Lab Example

Processing data but skipping invalid readings:

```

1 readings = [1.2, 3.4, -999, 5.6, -999, 7.8] # -999 indicates error
2
3 valid_readings = []
4 for reading in readings:
5     if reading < 0: # skip error values
6         continue
7     valid_readings.append(reading)
8
9 print(f"Valid readings: {valid_readings}")

```

4.5. List Comprehensions

A *list comprehension* is a compact way to create a list from another sequence. Instead of:

```
1 squares = []
2 for x in range(10):
3     squares.append(x ** 2)
```

You can write:

```
1 squares = [x ** 2 for x in range(10)]
```

The syntax is: [expression for item in sequence]

4.5.1. Adding Conditions

Filter items with an **if** clause:

```
1 # Only squares of even numbers
2 even_squares = [x ** 2 for x in range(10) if x % 2 == 0]
3 # Result: [0, 4, 16, 36, 64]
```

4.5.2. Practical Examples

```
1 # Convert temperatures from Celsius to Kelvin
2 celsius = [20, 25, 30, 35, 40]
3 kelvin = [c + 273.15 for c in celsius]
4
5 # Extract concentrations above a threshold
6 all_conc = [0.01, 0.15, 0.08, 0.22, 0.05]
7 high_conc = [c for c in all_conc if c > 0.1]
8
9 # Process strings
10 elements = ["hydrogen", "helium", "lithium"]
11 symbols = [e[0].upper() for e in elements] # ['H', 'H', 'L']
```

Tip

List comprehensions are elegant but can become unreadable if too complex. If you need nested loops or multiple conditions, a regular **for** loop is often clearer.

4.6. Dictionary and Set Comprehensions

The same syntax works for dictionaries and sets:

```
1 # Dictionary comprehension
2 squares_dict = {x: x**2 for x in range(5)}
3 # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
4
5 # Set comprehension
6 unique_lengths = {len(word) for word in ["cat", "dog", "elephant"]}
7 # {3, 8}
```

Lab Example

Creating a lookup table for molar masses:

```
1 elements = ["H", "He", "Li", "Be", "B"]
2 masses = [1.008, 4.003, 6.941, 9.012, 10.81]
3
4 molar_mass = {elem: mass for elem, mass in zip(elements, masses)}
5 print(molar_mass["Li"]) # 6.941
```

4.7. Structural Pattern Matching

Python 3.10 introduced a powerful new way to make decisions called *structural pattern matching*. While `if-elif-else` is great for checking conditions (like `x > 5`), the `match` statement is designed for comparing a value against specific patterns or structures.

The basic syntax looks like this:

```
1 match variable:
2     case pattern1:
3         # do something
4     case pattern2:
5         # do something else
6     case _:
7         # default (catch-all)
```

This is often cleaner than a long chain of `elif` statements when you are checking for specific values.

4.7.1. Basic Matching

Here is how you might classify an atomic orbital based on its angular momentum quantum number (l):

```
1 l = 2
2
3 match l:
4     case 0:
5         orbital = "s"
6     case 1:
7         orbital = "p"
8     case 2:
9         orbital = "d"
10    case 3:
11        orbital = "f"
12    case _:
13        orbital = "theoretical/undefined"
14
15 print(f"Orbital type: {orbital}")
```

The `case _:` block acts exactly like `else:`—it catches anything that didn't match the previous cases. The underscore (`_`) is a wildcard that means "anything."

Warning

The `match` statement was added in Python 3.10. If you are using an older version (like Python 3.8 or 3.9), this code will cause a `SyntaxError`. You must upgrade your Python installation or stick to `if-elif-else` for older environments.

4.7.2. Matching Structures

The real power of `match` isn't just checking values, but checking the *structure* of data, such as lists or tuples. It can even unpack variables inside the case statement.

Imagine an instrument returns a status code. Sometimes it returns just a string (e.g., `"IDLE"`), but if there is an error, it returns a tuple with an error code (e.g., `("ERROR", 404)`).

```

1  # Try changing this to ("ERROR", 500) or "IDLE"
2  status = ("ERROR", 404)
3
4  match status:
5      case "IDLE":
6          print("Instrument is ready.")
7      case "RUNNING":
8          print("Experiment in progress...")
9      case ("ERROR", code):
10         # This matches ANY tuple starting with "ERROR"
11         # and captures the second item into the variable 'code'
12         print(f"Instrument malfunction! Error code: {code}")
13     case _:
14         print("Unknown status received.")

```

In the third case, Python checks if `status` is a tuple with two items where the first is `"ERROR"`. If it matches, it automatically assigns the second item to the variable `code`. This is much more readable than writing `if isinstance(status, tuple) and status[0] == "ERROR":`.

Lab Example

Parsing simple commands for a syringe pump:

```

1  command = ["INJECT", 50]  # Command and volume in uL
2
3  match command:
4      case ["START"]:
5          print("Pump started.")
6      case ["STOP"]:
7          print("Pump stopped.")
8      case ["INJECT", volume]:
9          print(f"Injecting {volume} uL")
10     case ["WITHDRAW", volume]:
11         print(f"Withdrawing {volume} uL")
12     case _:
13         print("Invalid command format.")

```

Tip**Simulating match-case in older Python**

If you are working on a system with Python 3.9 or older, you cannot use the `match` statement. Instead of writing a long chain of `if-elif-else` statements, experienced Python programmers often use a **dictionary lookup**.

This is cleaner and often faster. Here is the orbital example rewritten using a dictionary:

```

1  l = 2
2
3  # Define the mapping
4  orbital_map = {0: "s", 1: "p", 2: "d", 3: "f"}
5
6  # Use .get() to handle the default case (like 'case _')
7  orbital = orbital_map.get(l, "theoretical/undefined")

```

This works perfectly for matching simple values. However, it cannot do the advanced *structural* matching (like checking for tuples or unpacking variables) that the modern `match` statement provides.

4.8. Chapter Summary

You've learned to control how your code executes:

- **Conditionals:** `if`, `elif`, `else` for making decisions
- **for loops:** Iterate over sequences with `for` item `in` sequence
- **range():** Generate number sequences for looping
- **enumerate():** Get both index and value
- **zip():** Pair up items from multiple sequences
- **while loops:** Repeat while a condition is true
- **break/continue:** Control loop execution
- **Comprehensions:** Compact syntax for creating lists, dicts, sets
- **Structural Pattern Matching:** Use structured inputs to guide decisions.

Things to Ponder

1. When would you use a `while` loop instead of a `for` loop?
2. What's the difference between `break` and `continue`?
3. Rewrite this using a list comprehension:

```

1  result = []
2  for x in range(20):
3      if x % 3 == 0:
4          result.append(x * 2)
5

```

4. What happens if you forget to increment the counter in a `while` loop?
5. What happens if a counter increments past a discrete stop condition in a `while` loop (e.g. `while` counter `!=` 5 vs `while` counter `<` 5)?

Chapter 5.

Functions

You've been using functions like `print()`, `len()`, and `type()` throughout this book. Now it's time to write your own. Functions let you package code into reusable pieces, making your programs shorter, clearer, and easier to maintain.

5.1. Why Functions?

Imagine you need to calculate molarity in multiple places in your code:

```
1  # First calculation
2  mass1 = 5.85
3  molar_mass1 = 58.44
4  volume1 = 0.5
5  molarity1 = mass1 / molar_mass1 / volume1
6
7  # Second calculation
8  mass2 = 11.7
9  molar_mass2 = 58.44
10 volume2 = 1.0
11 molarity2 = mass2 / molar_mass2 / volume2
12
13 # Third calculation...
```

This is repetitive. If you later realize you made an error in the formula, you'd need to fix it in every location. Functions solve this by letting you define the calculation once and use it many times.

5.2. Defining Functions

Create a function with the `def` keyword:

```
1  def greet():
2      print("Hello, scientist!")
```

The structure is:

- `def` keyword
- Function name (follows same rules as variable names)
- Parentheses (may contain parameters)
- Colon
- Indented block of code (the function body)

To use ("call") the function:

```
1  greet()  # prints "Hello, scientist!"
2  greet()  # prints it again
```

5.2.1. Parameters and Arguments

Functions become powerful when they accept inputs called *parameters*:

```
1 def greet(name):
2     print(f"Hello, {name}!")
3
4 greet("Alice")    # prints "Hello, Alice!"
5 greet("Bob")      # prints "Hello, Bob!"
```

In the definition, `name` is a *parameter*—a placeholder. When you call the function, you provide an *argument*—the actual value. (Many people use these terms interchangeably, but technically parameters are in definitions, arguments are in calls.)

Multiple parameters are separated by commas:

```
1 def calculate_molarity(mass, molar_mass, volume):
2     molarity = mass / molar_mass / volume
3     print(f"Molarity: {molarity:.4f} M")
4
5 calculate_molarity(5.85, 58.44, 0.5)
6 calculate_molarity(11.7, 58.44, 1.0)
```

5.2.2. Return Values

Printing is nice, but usually you want to get a value back from a function. Use `return`:

```
1 def calculate_molarity(mass, molar_mass, volume):
2     molarity = mass / molar_mass / volume
3     return molarity
4
5 # Now you can use the result
6 conc = calculate_molarity(5.85, 58.44, 0.5)
7 print(f"Concentration: {conc:.4f} M")
8
9 # Or use it directly in expressions
10 if calculate_molarity(11.7, 58.44, 1.0) > 0.1:
11     print("Concentration is above threshold")
```

Note

A function without a `return` statement returns `None`. If you try to use the result, you'll get `None`—a common source of confusion for beginners.

5.2.3. Multiple Return Values

Python functions can return multiple values using tuples:

```
1 def analyze_data(values):
2     minimum = min(values)
3     maximum = max(values)
4     average = sum(values) / len(values)
5     return minimum, maximum, average
6
7 data = [1.2, 3.4, 2.1, 5.6, 4.3]
8 low, high, avg = analyze_data(data)
9 print(f"Range: {low} to {high}, Average: {avg:.2f}")
```

Lab Example

A function to calculate reaction quotient Q:

```

1 def reaction_quotient(products, reactants):
2     """
3     Calculate Q for a reaction.
4     products: list of [concentration, coefficient] pairs
5     reactants: list of [concentration, coefficient] pairs
6     """
7     Q_num = 1.0
8     for conc, coeff in products:
9         Q_num *= conc ** coeff
10
11     Q_den = 1.0
12     for conc, coeff in reactants:
13         Q_den *= conc ** coeff
14
15     return Q_num / Q_den
16
17 # For: 2NO2 <-> N2O4
18 # [N2O4] = 0.5 M, [NO2] = 0.1 M
19 Q = reaction_quotient(
20     products=[[0.5, 1]],          # N2O4 with coefficient 1
21     reactants=[[0.1, 2]]         # NO2 with coefficient 2
22 )
23 print(f"Q = {Q}")

```

5.3. Default Parameters

You can give parameters default values:

```

1 def calculate_molarity(mass, molar_mass, volume=1.0):
2     return mass / molar_mass / volume
3
4 # volume defaults to 1.0 if not specified
5 conc1 = calculate_molarity(5.85, 58.44)      # volume = 1.0
6 conc2 = calculate_molarity(5.85, 58.44, 0.5) # volume = 0.5

```

Warning

Parameters with defaults must come after parameters without defaults:

```

1 def func(a, b=1, c):      # ERROR! c has no default but comes after b
2     pass
3
4 def func(a, c, b=1):      # OK - default parameter is last
5     pass

```

5.3.1. The Mutable Default Argument Trap

One of Python's most notorious gotchas involves mutable default arguments. Consider this function:

```

1 def add_reading(reading, readings=[]):
2     readings.append(reading)

```

```

3     return readings
4
5 # Seems fine at first...
6 print(add_reading(1.5))    # [1.5]
7 print(add_reading(2.3))    # [1.5, 2.3] -- wait, what?
8 print(add_reading(3.1))    # [1.5, 2.3, 3.1] -- it remembers!

```

The list accumulates values across calls! This happens because the default value `[]` is created *once* when the function is defined, not each time it's called. Every call shares the same list object.

Recall from Chapter 2 (Section ??) that variables are labels pointing to objects. The default `[]` creates one list object, and every call that uses the default gets that same object.

Tip

The fix is to use `None` as the default and create the list inside the function:

```

1 def add_reading(reading, readings=None):
2     if readings is None:
3         readings = []    # create a NEW list each time
4         readings.append(reading)
5     return readings
6
7 print(add_reading(1.5))    # [1.5]
8 print(add_reading(2.3))    # [2.3] -- fresh list each time

```

This pattern applies to any mutable default: lists, dictionaries, sets, or custom objects.

5.4. Keyword Arguments

When calling functions, you can specify arguments by name:

```

1 def describe_element(name, symbol, atomic_number):
2     print(f"{name} ({symbol}): Z = {atomic_number}")
3
4 # Positional arguments (order matters)
5 describe_element("Carbon", "C", 6)
6
7 # Keyword arguments (order doesn't matter)
8 describe_element(atomic_number=6, name="Carbon", symbol="C")
9
10 # Mixed (positional first, then keyword)
11 describe_element("Carbon", atomic_number=6, symbol="C")

```

Keyword arguments make function calls more readable, especially with many parameters:

```

1 # What do these numbers mean?
2 result = analyze_spectrum(data, 400, 700, 0.1, True, False)
3
4 # Much clearer:
5 result = analyze_spectrum(
6     data,
7     wavelength_min=400,
8     wavelength_max=700,
9     resolution=0.1,
10    normalize=True,
11    subtract_baseline=False
12 )

```

5.5. Flexible Arguments: **args* and ***kwargs*

When reading library documentation, you'll often see function signatures like:

```
1 def plot(x, y, *args, **kwargs):
2     ...
```

What do the asterisks mean?

5.5.1. **args*: Variable Positional Arguments

A parameter prefixed with *** collects any extra positional arguments into a tuple:

```
1 def calculate_mean(*values):
2     """Calculate mean of any number of values."""
3     return sum(values) / len(values)
4
5 print(calculate_mean(1, 2, 3))           # 2.0
6 print(calculate_mean(10, 20, 30, 40))    # 25.0
```

The name *args* is a convention—you could use any name, but **args* is standard.

5.5.2. ***kwargs*: Variable Keyword Arguments

A parameter prefixed with **** collects extra keyword arguments into a dictionary:

```
1 def create_sample(name, **properties):
2     """Create a sample with arbitrary properties."""
3     print(f"Sample: {name}")
4     for key, value in properties.items():
5         print(f"  {key}: {value}")
6
7 create_sample("S1", concentration=0.5, temperature=25, pH=7.2)
8 # Output:
9 # Sample: S1
10 #   concentration: 0.5
11 #   temperature: 25
12 #   pH: 7.2
```

5.5.3. Why This Matters

You'll rarely need to define functions with **args* and ***kwargs* yourself, but you'll *call* many library functions that use them. Understanding this syntax helps you:

- Read documentation for libraries like Matplotlib or Pandas
- Pass optional parameters through wrapper functions
- Understand error messages about unexpected arguments

```
1 # Matplotlib's plot() accepts many optional keyword arguments
2 import matplotlib.pyplot as plt
3 plt.plot(x, y, color='red', linewidth=2, marker='o')
4 # These extra arguments are collected by **kwargs
```

5.6. Docstrings

A *docstring* is a string at the beginning of a function that documents what it does:

```
1 def calculate_molarity(mass, molar_mass, volume):
2     """
3     Calculate the molarity of a solution.
4
5     Parameters:
6         mass: Mass of solute in grams
7         molar_mass: Molar mass of solute in g/mol
8         volume: Volume of solution in liters
9
10    Returns:
11        Molarity in mol/L
12    """
13    return mass / molar_mass / volume
```

Docstrings help others (and future you) understand your code. Python’s `help()` function displays them:

```
1 >>> help(calculate_molarity)
```

Tip

Write docstrings for any function that isn’t immediately obvious. It takes a minute now and saves hours of confusion later.

5.7. Scope: Where Variables Live

Variables created inside a function exist only inside that function. This is called *scope*:

```
1 def my_function():
2     x = 10 # x exists only inside this function
3     print(x)
4
5 my_function() # prints 10
6 print(x)     # ERROR! x is not defined here
```

Variables defined outside functions are “global” and can be *read* from anywhere without special syntax:

```
1 multiplier = 2 # global variable
2
3 def double(x):
4     return x * multiplier # reading global variable works fine
5
6 print(double(5)) # prints 10
```

However, *modifying* a global variable from inside a function requires the `global` keyword:

```
1 counter = 0
2
3 def increment_broken():
4     counter = counter + 1 # ERROR! Python thinks counter is local
5
6 def increment_works():
7     global counter        # tell Python we mean the global variable
```



```

8     counter = counter + 1
9
10    increment_works()
11    print(counter)    # prints 1

```

Without `global`, Python assumes you're creating a new local variable. The error occurs because you're trying to read `counter` (on the right side of `=`) before assigning it locally.

Warning

Modifying global variables is generally discouraged—it makes code harder to understand and debug. Instead, pass values as parameters and return results:

```

1  # Bad - modifies global state
2  counter = 0
3  def increment():
4      global counter
5      counter += 1
6
7  # Better - explicit input and output
8  def increment(counter):
9      return counter + 1
10
11 counter = increment(counter)

```

Functions that don't modify global state are easier to test, reuse, and reason about.

5.8. Functions as Values

In Python, functions are values—you can store them in variables, put them in lists, and pass them to other functions:

```

1  def square(x):
2      return x ** 2
3
4  def cube(x):
5      return x ** 3
6
7  # Store function in a variable
8  operation = square
9  print(operation(4))    # prints 16
10
11 operation = cube
12 print(operation(4))    # prints 64
13
14 # Put functions in a list
15 operations = [square, cube]
16 for op in operations:
17     print(op(3))    # prints 9, then 27

```

This is useful for applying different operations to data:

```

1  def apply_to_all(data, function):
2      """Apply a function to every element in a list."""
3      return [function(x) for x in data]
4
5  numbers = [1, 2, 3, 4, 5]
6  squared = apply_to_all(numbers, square)    # [1, 4, 9, 16, 25]

```

5.9. Lambda Functions

For simple, one-line functions, Python offers `lambda`—a way to create small anonymous (unnamed) functions:

```
1 # Regular function
2 def square(x):
3     return x ** 2
4
5 # Equivalent lambda
6 square = lambda x: x ** 2
```

Lambdas are useful when you need a quick function for a single use:

```
1 numbers = [1, 2, 3, 4, 5]
2
3 # Using lambda with map
4 squared = list(map(lambda x: x ** 2, numbers))
5
6 # Using lambda for sorting
7 elements = [("C", 6), ("H", 1), ("O", 8), ("N", 7)]
8 sorted_by_number = sorted(elements, key=lambda x: x[1])
9 # [('H', 1), ('C', 6), ('N', 7), ('O', 8)]
```

Tip

If a lambda is complex enough that it's hard to read, use a regular function instead. Lambdas are for simple operations only.

5.10. Built-in Functions Worth Knowing

Python includes many useful built-in functions. Here are some important ones:

```
1 # Math operations on sequences
2 numbers = [3, 1, 4, 1, 5, 9, 2, 6]
3 sum(numbers)           # 31
4 min(numbers)           # 1
5 max(numbers)           # 9
6 len(numbers)           # 8
7
8 # Rounding
9 round(3.14159, 2)      # 3.14
10 abs(-5)                # 5
11
12 # Type conversions
13 int("42")              # 42
14 float("3.14")          # 3.14
15 str(42)                # "42"
16 list("abc")            # ['a', 'b', 'c']
17
18 # Checking types
19 isinstance(42, int)     # True
20 isinstance("hi", (int, str)) # True (either type)
21
22 # Working with sequences
23 sorted([3, 1, 4])       # [1, 3, 4]
24 reversed([1, 2, 3])    # an iterator which returns: 3, 2, 1
```

```

25 list(zip([1,2], ['a','b'])) # [(1,'a'), (2,'b')]
26 list(enumerate(['a','b'])) # [(0,'a'), (1,'b')]
27
28 # All/any
29 all([True, True, False]) # False (not all True)
30 any([True, False, False]) # True (at least one True)

```

5.11. A Complete Example

Let's build a set of functions for basic spectroscopy calculations:

```

1
2 import math # load the math library, this gives us access to logarithms
   by name.
3
4 def absorbance_to_transmittance(A):
5     """Convert absorbance to percent transmittance."""
6     return 100 * (10 ** (-A))
7
8 def transmittance_to_absorbance(T):
9     """Convert percent transmittance to absorbance. This function
   assumes transmittance is supplied as a percentage."""
10    return 2-math.log10(T)
11
12 def beers_law_concentration(A, epsilon, path_length=1.0):
13     """
14     Calculate concentration using Beer's Law:  $A = \epsilon \cdot c \cdot l$ 
15
16     Parameters:
17         A: Absorbance (unitless)
18         epsilon: Molar absorptivity (L/(mol*cm))
19         path_length: Path length in cm (default 1.0)
20
21     Returns:
22         Concentration in mol/L
23     """
24     return A / (epsilon * path_length)
25
26 def beers_law_absorbance(concentration, epsilon, path_length=1.0):
27     """Calculate expected absorbance using Beer's Law."""
28     return epsilon * concentration * path_length
29
30 # Using the functions
31 A_measured = 0.523
32 epsilon_450nm = 18500 # L/(mol*cm) for beta-carotene at 450 nm
33
34 conc = beers_law_concentration(A_measured, epsilon_450nm)
35 print(f"Concentration: {conc:.2e} M")
36
37 T = absorbance_to_transmittance(A_measured)
38 print(f"Transmittance: {T:.1f}%")

```

5.12. Chapter Summary

Functions are fundamental to good programming:

- **Define:** `def function_name(parameters):`
- **Return values:** `return result`
- **Default parameters:** `def func(a, b=1):`
- **Keyword arguments:** `func(a=1, b=2)`
- **Docstrings:** Document what functions do
- **Scope:** Variables inside functions are local
- **Lambda:** `lambda x: x ** 2` for simple functions

Things to Ponder

1. What's the difference between `print()` and `return`?
2. Why should you avoid modifying global variables in functions?
3. When would you use a lambda instead of a regular function?
4. Write a function that takes a list of numbers and returns both the mean and standard deviation.

Tip

Ready to Upgrade? Now that you are writing more complex scripts with functions and variable scope, you might find a simple text editor limiting. If you skipped Section 1.9.2 (IDEs) in the introduction, you are now ready to fully benefit from those tools. Go check it out!

Chapter 6.

Working with Files

Scientific data lives in files: CSV tables from instruments, text files of measurements, configuration files for experiments. This chapter teaches you to read data from files and write results back out—essential skills for any scientist who programs. While libraries like **Pandas** (discussed later) provide methods for reading common file types, understanding how to read and write arbitrary files is a powerful skill.

6.1. Opening and Closing Files

The `open()` function opens a file and returns a *file object* you can read from or write to:

```
1 # Open a file for reading
2 file = open("data.txt", "r")
3 content = file.read()
4 file.close()
```

The second argument specifies the *mode*:

- `"r"` — Read (default). File must exist.
- `"w"` — Write. Creates file or overwrites existing.
- `"a"` — Append. Adds to end of existing file.
- `"x"` — Exclusive creation. Fails if file exists.

Warning

Always close files when you're done. Forgetting to close can cause data loss or prevent other programs from accessing the file. The `with` statement (next section) handles this automatically.

6.2. The with Statement

The `with` statement automatically closes files, even if errors occur:

```
1 with open("data.txt", "r") as file:
2     content = file.read()
3     # file is automatically closed when this block ends
4
5 # file is now closed
```

Always use `with` when working with files. It's safer and cleaner.

6.3. Reading Files

6.3.1. Read Entire File

```
1 with open("data.txt", "r") as file:
2     content = file.read() # entire file as one string
3     print(content)
```

6.3.2. Read Lines

```
1 with open("data.txt", "r") as file:
2     lines = file.readlines()  # list of lines
3
4 for line in lines:
5     print(line)
```

Note

Each line includes the newline character (`\n`) at the end. Use `line.strip()` to remove it.

6.3.3. Read Line by Line (Memory Efficient)

For large files, iterate directly over the file object:

```
1 with open("large_data.txt", "r") as file:
2     for line in file:
3         process(line.strip())
```

This reads one line at a time, not the whole file into memory.

Lab Example

Reading a simple data file with one measurement per line:

```
1 # File: measurements.txt
2 # 23.4
3 # 25.1
4 # 24.8
5 # 26.2
6
7 measurements = []
8 with open("measurements.txt", "r") as file:
9     for line in file:
10         line = line.strip()  # remove whitespace
11         if line:  # skip empty lines
12             value = float(line)  # convert to number
13             measurements.append(value)
14
15 print(f"Read {len(measurements)} values")
16 print(f"Average: {sum(measurements)/len(measurements):.2f}")
```

6.4. Writing Files

6.4.1. Write Text

```
1 with open("output.txt", "w") as file:
2     file.write("First line\n")
3     file.write("Second line\n")
```

Warning

Mode "w" overwrites the file completely. If the file existed, its old content is gone. Use "a" to append instead.

6.4.2. Write Multiple Lines

```

1 lines = ["Line 1", "Line 2", "Line 3"]
2
3 with open("output.txt", "w") as file:
4     for line in lines:
5         file.write(line + "\n")
6
7 # Or use writelines (doesn't add newlines automatically!)
8 with open("output.txt", "w") as file:
9     file.writelines(line + "\n" for line in lines)

```

Lab Example

Saving processed data:

```

1 times = [0, 10, 20, 30, 40, 50]
2 concentrations = [1.0, 0.82, 0.67, 0.55, 0.45, 0.37]
3
4 with open("kinetics_results.txt", "w") as file:
5     file.write("Time (s)\tConcentration (M)\n")
6     file.write("-" * 30 + "\n")
7     for t, c in zip(times, concentrations):
8         file.write(f"{t}\t{c:.4f}\n")

```

6.5. Working with Paths

Python's `pathlib` module provides a clean way to work with paths (recall section 1.3):

```

1 from pathlib import Path
2
3 # Create a path
4 data_dir = Path("data")
5 file_path = data_dir / "measurements.txt" # / joins paths
6
7 # Check if file exists
8 if file_path.exists():
9     print("File found!")
10
11 # Get file information
12 print(file_path.name)           # measurements.txt
13 print(file_path.stem)           # measurements
14 print(file_path.suffix)         # .txt
15 print(file_path.parent)         # data
16
17 # List files in a directory
18 for file in Path("data").glob("*.csv"):
19     print(file)

```

Tip

Using `pathlib` instead of raw strings makes your code work on both Windows (which uses backslashes) and macOS/Linux (which use forward slashes). The `/` operator builds the correct path for your system.

6.6. CSV Files

CSV (Comma-Separated Values) is the most common format for tabular data. Each line is a row, with columns separated by commas:

```
1 wavelength,absorbance
2 400,0.12
3 450,0.34
4 500,0.56
5 550,0.43
```

6.6.1. Reading CSV with the `csv` Module

```
1 import csv
2
3 with open("spectrum.csv", "r") as file:
4     reader = csv.reader(file)
5     header = next(reader) # skip header row
6
7     for row in reader:
8         wavelength = float(row[0])
9         absorbance = float(row[1])
10        print(f"{wavelength} nm: A = {absorbance}")
```

6.6.2. CSV with Headers: `DictReader`

`DictReader` gives you each row as a dictionary with column names as keys:

```
1 import csv
2
3 with open("spectrum.csv", "r") as file:
4     reader = csv.DictReader(file)
5
6     for row in reader:
7         print(row["wavelength"], row["absorbance"])
```

This is more readable and less error-prone than using indices.

6.6.3. Writing CSV

```
1 import csv
2
3 wavelengths = [400, 450, 500, 550, 600]
4 absorbances = [0.12, 0.34, 0.56, 0.43, 0.21]
5
6 with open("output.csv", "w", newline="") as file:
7     writer = csv.writer(file)
8     writer.writerow(["wavelength", "absorbance"]) # header
```



```
9
10     for w, a in zip(wavelengths, absorbances):
11         writer.writerow([w, a])
```

Note

The `newline=""` argument prevents blank lines between rows on Windows.

Lab Example

Processing a CSV of titration data:

```
1 import csv
2
3 volumes = []
4 pH_values = []
5
6 with open("titration.csv", "r") as file:
7     reader = csv.DictReader(file)
8     for row in reader:
9         volumes.append(float(row["volume_mL"]))
10        pH_values.append(float(row["pH"]))
11
12 # Find equivalence point (maximum pH change)
13 max_change = 0
14 equiv_volume = 0
15 for i in range(1, len(pH_values)):
16     change = pH_values[i] - pH_values[i-1]
17     if change > max_change:
18         max_change = change
19         equiv_volume = volumes[i]
20
21 print(f"Equivalence point at approximately {equiv_volume} mL")
```

6.7. Handling Different Delimiters

Not all “CSV” files use commas. Tab-separated and semicolon-separated are common:

```
1 import csv
2
3 # Tab-separated
4 with open("data.tsv", "r") as file:
5     reader = csv.reader(file, delimiter="\t")
6     for row in reader:
7         print(row)
8
9 # Semicolon-separated (common in European data)
10 with open("data.csv", "r") as file:
11     reader = csv.reader(file, delimiter=";")
12     for row in reader:
13         print(row)
```

6.8. JSON Files

JSON (JavaScript Object Notation, JavaScript is another type of programming language) is a format for structured data, commonly used for configuration files and web APIs (API means application programming interface, a way for programs to interact with things). It looks similar to Python dictionaries:

```
1 {  
2     "experiment": "kinetics_001",  
3     "temperature": 298.15,  
4     "concentrations": [0.1, 0.2, 0.3],  
5     "parameters": {  
6         "rate_constant": 0.05,  
7         "order": 1  
8     }  
9 }
```

6.8.1. Reading JSON

```
1 import json  
2  
3 with open("config.json", "r") as file:  
4     data = json.load(file)  
5  
6 print(data["experiment"])           # kinetics_001  
7 print(data["parameters"]["rate_constant"]) # 0.05
```

6.8.2. Writing JSON

```
1 import json  
2  
3 results = {  
4     "experiment": "kinetics_001",  
5     "rate_constant": 0.0523,  
6     "r_squared": 0.9987,  
7     "observations": [1.0, 0.82, 0.67, 0.55, 0.45]  
8 }  
9  
10 with open("results.json", "w") as file:  
11     json.dump(results, file, indent=2) # indent for readability
```

6.9. Handling Encoding

Text files use different *encodings* to represent characters. UTF-8 is the modern standard, but you may encounter files in other encodings:

```
1 # Specify encoding explicitly  
2 with open("data.txt", "r", encoding="utf-8") as file:  
3     content = file.read()  
4  
5 # For files with special characters that won't open  
6 with open("old_data.txt", "r", encoding="latin-1") as file:  
7     content = file.read()
```

Warning

If you get a `UnicodeDecodeError`, the file probably uses a different encoding than Python expects. Common alternatives to try: `"latin-1"`, `"cp1252"` (Windows), or `"iso-8859-1"`. This happens because computers don't store text, only numbers. Each character gets an index number and different encoding schemes index differently, humans rarely agree on anything and so we have many encoding schemes in use. Thankfully, these days almost everyone agrees on the UTF encoding schema.

6.10. Error Handling for Files

Files can fail to open for many reasons: wrong path, permissions, file in use. Handle these gracefully:

```
1 from pathlib import Path
2
3 filepath = Path("data.txt")
4
5 if not filepath.exists():
6     print(f"Error: {filepath} not found")
7 else:
8     with open(filepath, "r") as file:
9         content = file.read()
```

Or use exception handling (covered in detail in the next chapter):

```
1 try:
2     with open("data.txt", "r") as file:
3         content = file.read()
4 except FileNotFoundError:
5     print("File not found!")
6 except PermissionError:
7     print("Permission denied!")
```

6.11. A Complete Example

Processing a folder of spectroscopy data files:

```
1 import csv
2 from pathlib import Path
3
4 def process_spectrum(filepath):
5     """Read a spectrum CSV and return wavelengths, absorbances."""
6     wavelengths = []
7     absorbances = []
8
9     with open(filepath, "r") as file:
10         reader = csv.DictReader(file)
11         for row in reader:
12             wavelengths.append(float(row["wavelength"]))
13             absorbances.append(float(row["absorbance"]))
14
15     return wavelengths, absorbances
16
17 def find_peak(wavelengths, absorbances):
18     """Find wavelength of maximum absorbance."""
19     max_abs = max(absorbances)
```

```

20     max_idx = absorbances.index(max_abs)
21     return wavelengths[max_idx], max_abs
22
23 # Process all CSV files in data folder
24 data_folder = Path("spectra")
25 results = []
26
27 for filepath in data_folder.glob("*.csv"):
28     wavelengths, absorbances = process_spectrum(filepath)
29     peak_wl, peak_abs = find_peak(wavelengths, absorbances)
30     results.append({
31         "filename": filepath.name,
32         "peak_wavelength": peak_wl,
33         "peak_absorbance": peak_abs
34     })
35     print(f"{filepath.name}: peak at {peak_wl} nm (A = {peak_abs:.3f})")
36
37 # Save summary
38 with open("summary.csv", "w", newline="") as file:
39     writer = csv.DictWriter(file,
40         fieldnames=["filename", "peak_wavelength", "peak_absorbance"])
41     writer.writeheader()
42     writer.writerows(results)

```

6.12. Chapter Summary

File operations are central to scientific programming:

- **Opening:** `with open(path, mode) as file:`
- **Modes:** `"r"` read, `"w"` write, `"a"` append
- **Reading:** `file.read()`, `file.readlines()`, iterate line by line
- **Writing:** `file.write(text)`
- **Paths:** Use `pathlib.Path` for cross-platform compatibility
- **CSV:** Use the `csv` module, especially `DictReader`
- **JSON:** Use `json.load()` and `json.dump()`

Things to Ponder

1. What's the difference between `"w"` and `"a"` modes?
2. Why should you always use `with` when opening files?
3. How would you read a CSV file that uses semicolons instead of commas?
4. What happens if you try to open a file that doesn't exist in `"r"` mode?

Chapter 7.

Errors and Exceptions

Things go wrong. Files are missing, users enter invalid data, calculations produce impossible results. Good programs anticipate problems and handle them gracefully instead of crashing. This chapter teaches you to read and write error messages, handle exceptions, and write robust code.

This is where you distinguish yourself from large language models. While AI can generate code, it lacks your physical intuition. It takes human experience to really understand the specific constraints of a system and anticipate *why* a calculation might fail.

7.1. Reading Error Messages

When Python encounters a problem, it raises an *exception* and shows an error message called a *traceback*:

```
1  Traceback (most recent call last):
2    File "analysis.py", line 15, in <module>
3      result = calculate_concentration(data)
4    File "analysis.py", line 8, in calculate_concentration
5      concentration = mass / molar_mass / volume
6  ZeroDivisionError: float division by zero
```

Read tracebacks from bottom to top:

1. **Last line:** The type of error and a description
2. **Lines above:** The chain of function calls that led to the error
3. **Line numbers:** Where to look in your code

In this example: `ZeroDivisionError` occurred at line 8, which was called from line 15.

7.2. Common Error Types

7.2.1. SyntaxError

Your code violates Python's grammar rules. Python can't even run the code.

```
1  # Missing colon
2  if x > 5
3      print("big")
4
5  # Mismatched parentheses
6  print("Hello "
```

7.2.2. NameError

You used a variable that doesn't exist.

```
1  >>> print(temperature)
2  NameError: name 'temperature' is not defined
```

Usually a typo or you forgot to assign a value.

7.2.3. TypeError

You used an operation on an inappropriate type.

```
1  >>> "5" + 3
2  TypeError: can only concatenate str (not "int") to str
3
4  >>> len(42)
5  TypeError: object of type 'int' has no len()
```

7.2.4. ValueError

The type is correct, but the value is inappropriate.

```
1  >>> int("hello")
2  ValueError: invalid literal for int() with base 10: 'hello'
3
4  >>> import math
5  >>> math.sqrt(-1)
6  ValueError: math domain error
```

7.2.5. IndexError

You accessed a list index that doesn't exist.

```
1  >>> data = [1, 2, 3]
2  >>> data[10]
3  IndexError: list index out of range
```

7.2.6. KeyError

You accessed a dictionary key that doesn't exist.

```
1  >>> element = {"name": "Carbon", "symbol": "C"}
2  >>> element["mass"]
3  KeyError: 'mass'
```

7.2.7. FileNotFoundError

You tried to open a file that doesn't exist.

```
1  >>> open("nonexistent.txt")
2  FileNotFoundError: [Errno 2] No such file or directory: 'nonexistent.txt'
```

7.2.8. AttributeError

You tried to access a method or property that doesn't exist on an object.

```
1  >>> x = 42
2  >>> x.append(5)
3  AttributeError: 'int' object has no attribute 'append'
```

7.3. Handling Exceptions with try-except

Instead of letting errors crash your program, you can catch and handle them:

```
1 try:
2     value = int(input("Enter a number: "))
3     print(f"You entered: {value}")
4 except ValueError:
5     print("That's not a valid number!")
```

The `try` block contains code that might fail. If an exception occurs, Python jumps to the matching `except` block instead of crashing.

7.3.1. Catching Multiple Exception Types

```
1 try:
2     result = data[index] / divisor
3 except IndexError:
4     print("Index out of range")
5 except ZeroDivisionError:
6     print("Cannot divide by zero")
```

Or catch several types the same way:

```
1 try:
2     value = float(user_input)
3 except (ValueError, TypeError):
4     print("Invalid input")
```

7.3.2. Accessing Exception Information

Use `as` to get details about the exception:

```
1 try:
2     with open("data.txt") as file:
3         content = file.read()
4 except FileNotFoundError as e:
5     print(f"Could not open file: {e}")
```

7.3.3. Catching All Exceptions

You can catch any exception with bare `except` or `except Exception`:

```
1 try:
2     risky_operation()
3 except Exception as e:
4     print(f"Something went wrong: {e}")
```

Warning

Catching all exceptions can hide bugs. Use specific exception types when possible. Bare `except` : is especially dangerous because it catches even keyboard interrupts (Ctrl+C), making your program hard to stop.

7.4. else and finally

7.4.1. else: Code That Runs If No Exception

```
1 try:
2     value = int(input("Enter a number: "))
3 except ValueError:
4     print("Invalid input")
5 else:
6     print(f"You entered: {value}")
7     # This only runs if no exception occurred
```

7.4.2. finally: Code That Always Runs

The **finally** block runs whether or not an exception occurred—useful for cleanup operations. However, you must be careful: if an exception occurs early, variables you expect to exist might not be defined.

```
1 # Buggy version - if open() fails, 'file' doesn't exist!
2 try:
3     file = open("data.txt")
4     process(file)
5 finally:
6     file.close() # NameError if open() failed!
```

The safe pattern initializes the variable first:

```
1 file = None
2 try:
3     file = open("data.txt")
4     process(file)
5 except FileNotFoundError:
6     print("File not found")
7 finally:
8     if file:
9         file.close()
10    print("Cleanup complete")
```

Tip

For files specifically, the **with** statement (Chapter 6) handles this automatically and is always preferred:

```
1 with open("data.txt") as file:
2     process(file)
3 # File is automatically closed, even if an exception occurred
```

The **try/finally** pattern shown above is mainly useful for resources that don't support **with**, or when you need custom cleanup logic.

Lab Example

A robust data loading function:

```

1 def load_measurements(filepath):
2     """Load measurements from a file, handling common errors."""
3     measurements = []
4
5     try:
6         with open(filepath, "r") as file:
7             for line_num, line in enumerate(file, start=1):
8                 try:
9                     value = float(line.strip())
10                    measurements.append(value)
11                except ValueError:
12                    print(f"Warning: Invalid value on line {line_num}
13                        })
14
15            except FileNotFoundError:
16                print(f"Error: File '{filepath}' not found")
17                return None
18            except PermissionError:
19                print(f"Error: Cannot read '{filepath}' (permission denied)"
20                )
21                return None
22
23            if not measurements:
24                print("Warning: No valid measurements found")
25                return None
26
27            return measurements

```

7.5. Raising Exceptions

You can raise your own exceptions when something goes wrong:

```

1 def calculate_molarity(mass, molar_mass, volume):
2     if volume <= 0:
3         raise ValueError("Volume must be positive")
4     if molar_mass <= 0:
5         raise ValueError("Molar mass must be positive")
6
7     return mass / molar_mass / volume

```

When the function is called with bad values:

```

1 >>> calculate_molarity(5.0, 58.44, -1)
2 ValueError: Volume must be positive

```

7.5.1. When to Raise Exceptions

Raise exceptions when:

- Invalid input is passed to a function
- A required condition isn't met
- Something unexpected happens that the caller should know about

```
1 def get_element(symbol):
2     elements = {"H": "Hydrogen", "He": "Helium", "Li": "Lithium"}
3
4     if symbol not in elements:
5         raise KeyError(f"Unknown element symbol: {symbol}")
6
7     return elements[symbol]
```

7.6. Assertions

`assert` is a way to check that something is true, raising `AssertionError` if not:

```
1 def calculate_ph(h_concentration):
2     assert h_concentration > 0, "H+ concentration must be positive"
3     import math # usually imports are at file start, it's inside a
4                 # function here to show you can, but generally you shouldn't import
5                 # inside a function
6     return -math.log10(h_concentration)
```

Assertions are useful for:

- Catching programmer errors during development
- Documenting assumptions in your code
- Sanity checks that should never fail

Warning

Don't use `assert` for validating user input or handling expected errors. Assertions can be disabled with `python -O`, so they're not guaranteed to run. Use regular `if` statements and `raise` for real validation.

7.7. Debugging Strategies

When your code doesn't work, here's how to find the problem:

7.7.1. Read the Error Message

Start with the traceback. What line caused the error? What type of error? What does the message say?

7.7.2. Add Print Statements

```
1 def mystery_function(data):
2     print(f"Input data: {data}")
3     result = process(data)
4     print(f"After process: {result}")
5     final = transform(result)
6     print(f"After transform: {final}")
7     return final
```

This helps you see where things go wrong.

7.7.3. Simplify

If a complex expression fails, break it into steps:

```
1 # Hard to debug
2 result = data[get_index(params)].split(",")[column_map[name]]
3
4 # Easier to debug
5 index = get_index(params)
6 print(f"index: {index}")
7 row = data[index]
8 print(f"row: {row}")
9 parts = row.split(",")
10 print(f"parts: {parts}")
11 col_index = column_map[name]
12 print(f"col_index: {col_index}")
13 result = parts[col_index]
```

7.7.4. Check Types

When you get `TypeError`, print the types of your variables:

```
1 print(type(x), type(y))
2 print(repr(x), repr(y)) # repr shows more detail
```

7.7.5. Use a Debugger

Spyder and VS Code have built-in debuggers that let you:

- Pause execution at specific lines (breakpoints)
- Step through code line by line
- Inspect variable values at any point

Learning to use a debugger is worth the investment.

7.8. Defensive Programming

Write code that anticipates problems:

7.8.1. Validate Input Early

```
1 def analyze_spectrum(wavelengths, absorbances):
2     # Check inputs at the start
3     if len(wavelengths) != len(absorbances):
4         raise ValueError("Wavelengths and absorbances must have same
5         length")
6
7     if len(wavelengths) == 0:
8         raise ValueError("Cannot analyze empty spectrum")
9
10    if any(a < 0 for a in absorbances):
11        raise ValueError("Absorbances cannot be negative")
12
13    # Now proceed with analysis...
```

7.8.2. Use Default Values

```

1 # Instead of crashing on missing key
2 value = data.get("key", default_value)
3
4 # Instead of crashing on empty list
5 average = sum(values) / len(values) if values else 0

```

7.8.3. Fail Fast

If something is wrong, fail immediately with a clear message rather than continuing and causing confusing errors later:

```

1 def load_config(filepath):
2     if not filepath.exists():
3         raise FileNotFoundError(f"Config file not found: {filepath}")
4
5     # Continue only if file exists...

```

7.9. Automated Testing with pytest

In Section 7.6, we used `assert` to check if conditions were true. While useful, scattering assertions inside your analysis scripts makes them messy. Worse, you have to run the script manually to check if they pass.

Unit testing is the practice of writing small, separate pieces of code that check if your "real" code is working correctly. The standard tool for this in Python is `pytest`.

To get started, install it (making sure your virtual environment is active):

```

1 pip install pytest

```

7.9.1. Writing Your First Test (and the Float Trap)

`pytest` relies on conventions. It looks for files starting with `test_` and functions starting with `test_`.

Let's test a simple dilution function. Create a file called `test_dilution.py`:

```

1 # The function we want to test
2 def calculate_c2(c1, v1, v2):
3     """Calculate final concentration: c1*v1 = c2*v2"""
4     return (c1 * v1) / v2
5
6 # A simple test case
7 def test_simple_dilution():
8     # 1.0 M * 0.5 L / 1.0 L should be 0.5 M
9     assert calculate_c2(1.0, 0.5, 1.0) == 0.5

```

Run this in your terminal by typing `pytest`. It should pass with green text.

However, scientific calculations often involve decimals. Recall Section 2.3.2, where we discussed that computers cannot store all floating-point numbers exactly (e.g., $0.1 + 0.2$ is slightly larger than 0.3). This causes *flaky tests*.

Consider this test case:

```

1 def test_float_precision_problem():
2     # 0.1 M * 3.0 L / 1.0 L should be 0.3 M
3     result = calculate_c2(0.1, 3.0, 1.0)
4

```

```

5     # This assertion will FAIL!
6     # Actual result is 0.30000000000000004
7     assert result == 0.3

```

If you run this, `pytest` will report a failure because the numbers are strictly not equal.

Things to Ponder

Before reading on, how would you solve this? You cannot change how Python stores math. How can you write a test that checks if two numbers are "close enough" to be considered equal?

Tip

Handling Float Comparison

The manual way to solve this is to check if the difference is smaller than a tiny tolerance:

```
1 assert abs(result - 0.3) < 1e-6
```

However, `pytest` has a built-in tool for this: `pytest.approx()`.

```

1 from pytest import approx
2 assert result == approx(0.3)

```

This is the standard way to test numerical code in Python.

7.9.2. Testing for Expected Errors

A robust function should not just return correct values; it should also reject nonsense inputs. For example, a volume cannot be negative.

If you write code to raise an error (defensive programming), you must test that the error actually occurs. You cannot use a standard `assert` for this, because if the error occurs, the test crashes.

Instead, use `pytest.raises` as a "context manager" to catch the crash:

```

1 import pytest
2
3 # Improved function with error checking
4 def calculate_c2_safe(c1, v1, v2):
5     if v2 <= 0:
6         raise ValueError("Final volume must be positive")
7     return (c1 * v1) / v2
8
9 def test_zero_volume_error():
10     # We assert that the code inside the block RAISES ValueError.
11     # If the code runs successfully (no error), the test FAILS.
12     with pytest.raises(ValueError):
13         calculate_c2_safe(1.0, 0.5, 0)
14
15 def test_negative_volume_error():
16     with pytest.raises(ValueError):
17         calculate_c2_safe(1.0, 0.5, -10.0)

```

This pattern documents your assumptions. Anyone reading `test_zero_volume_error` knows immediately that your function is strictly designed to reject invalid volumes.

7.10. Chapter Summary

Error handling makes your code robust, while testing ensures it stays correct:

- **Read tracebacks** bottom to top
- **Common errors:** `SyntaxError`, `NameError`, `TypeError`, `ValueError`, `IndexError`, `KeyError`
- **try-except:** Catch and handle exceptions
- **raise:** Create your own exceptions
- **assert:** Check assumptions during development
- **Debug:** Print statements, simplify, check types
- **pytest:** Automate testing with simple functions in `test_*.py` files
- **Testing failures:** Use `pytest.raises` to verify your code catches invalid inputs

Things to Ponder

1. What's the difference between `SyntaxError` and other exceptions?
2. Why is catching all exceptions with bare `except:` dangerous?
3. When would you use `assert` vs. `raise`?
4. How would you handle a function that might receive either a filename string or an already-opened file object?
5. What benefits are there to writing unit tests, and is it worth the time? If your answer is no, think harder.

Chapter 8.

Object-Oriented Programming

You've been using objects throughout this book. Strings have methods like `.upper()`. Lists have methods like `.append()`. Now it's time to understand what objects are and how to create your own. Object-oriented programming (OOP) lets you bundle data and the functions that operate on that data into cohesive units.

8.1. What Are Objects?

An *object* combines data (called *attributes*) with functions that operate on that data (called *methods*). You've used many objects already:

```
1 text = "hello"           # text is a string object
2 text.upper()             # upper() is a method of string objects
3
4 numbers = [1, 2, 3]      # numbers is a list object
5 numbers.append(4)        # append() is a method of list objects
```

A *class* is the blueprint for creating objects. `str` is a class; `"hello"` is an object (or *instance*) of that class.

8.2. Defining a Class

Create a class with the `class` keyword:

```
1 class Element:
2     """Represents a chemical element."""
3
4     def __init__(self, symbol, name, atomic_number):
5         """Initialize the element with its properties."""
6         self.symbol = symbol
7         self.name = name
8         self.atomic_number = atomic_number
9
10    def describe(self):
11        """Print a description of the element."""
12        print(f"{self.name} ({self.symbol}): Z = {self.atomic_number}")
```

Let's break this down:

- `class Element:` defines a new class called `Element`
- `__init__` is a special method called when you create a new object (the *constructor*)
- `self` refers to the object being created or used
- `self.symbol` creates an attribute called `symbol` on the object
- `describe` is a regular method that operates on the object

8.2.1. Creating Objects

Create objects by calling the class like a function:

```
1 carbon = Element("C", "Carbon", 6)
2 oxygen = Element("O", "Oxygen", 8)
3
4 print(carbon.symbol)           # C
5 print(oxygen.atomic_number)    # 8
6
7 carbon.describe()              # Carbon (C): Z = 6
8 oxygen.describe()              # Oxygen (O): Z = 8
```

Each object has its own data. `carbon.symbol` is "C", while `oxygen.symbol` is "O".

8.3. The `self` Parameter

Every method's first parameter is `self`, which refers to the object the method is called on:

```
1 class Counter:
2     def __init__(self):
3         self.count = 0
4
5     def increment(self):
6         self.count += 1
7
8     def get_count(self):
9         return self.count
10
11 c = Counter()
12 c.increment()    # self refers to c
13 c.increment()
14 print(c.get_count())    # 2
```

When you call `c.increment()`, Python automatically passes `c` as the `self` parameter.

Note

The name `self` is a convention, not a requirement. You could use any name, but `self` is universal in Python—use it.

8.4. Attributes and Methods

8.4.1. Instance Attributes

Attributes defined on `self` belong to each individual object:

```
1 class Sample:
2     def __init__(self, name, concentration):
3         self.name = name
4         self.concentration = concentration
5
6 sample1 = Sample("A", 0.1)
7 sample2 = Sample("B", 0.2)
8
9 print(sample1.concentration)    # 0.1
10 print(sample2.concentration)    # 0.2
```


8.4.2. Class Attributes

Attributes defined on the class are shared by all objects:

```
1 class Element:
2     # Class attribute - shared by all instances
3     elements_created = 0
4
5     def __init__(self, symbol, name):
6         self.symbol = symbol # instance attribute
7         self.name = name
8         Element.elements_created += 1
9
10 e1 = Element("H", "Hydrogen")
11 e2 = Element("He", "Helium")
12 print(Element.elements_created) # 2
```

8.4.3. Methods

Methods are functions that belong to a class:

```
1 class Solution:
2     def __init__(self, solute, concentration, volume):
3         self.solute = solute
4         self.concentration = concentration # mol/L
5         self.volume = volume # L
6
7     def moles(self):
8         """Calculate moles of solute."""
9         return self.concentration * self.volume
10
11     def dilute(self, new_volume):
12         """Return new concentration after dilution."""
13         moles = self.moles()
14         return moles / new_volume
15
16     def describe(self):
17         print(f"{self.concentration} M {self.solute}, {self.volume} L")
18
19
20 stock = Solution("NaCl", 1.0, 0.5)
21 stock.describe() # 1.0 M NaCl, 0.5 L
22 print(f"Moles: {stock.moles()}") # Moles: 0.5
23
24 new_conc = stock.dilute(2.0)
25 print(f"After dilution to 2L: {new_conc} M")
```

8.5. Special Methods

Python uses methods with double underscores (“dunder” methods) for special behaviors:

8.5.1. `__str__` and `__repr__`

Control how objects are displayed:

```
1 class Element:
2     def __init__(self, symbol, name, atomic_number):
```

```

3     self.symbol = symbol
4     self.name = name
5     self.atomic_number = atomic_number
6
7     def __str__(self):
8         """Human-readable string (used by print)."""
9         return f"{self.name} ({self.symbol})"
10
11    def __repr__(self):
12        """Developer-readable string (used in REPL)."""
13        return f"Element('{self.symbol}', '{self.name}', {self.
atomic_number})"
14
15    carbon = Element("C", "Carbon", 6)
16    print(carbon)    # Carbon (C)

```

8.5.2. Comparison Methods

Make objects comparable:

```

1    class Element:
2        def __init__(self, symbol, atomic_number):
3            self.symbol = symbol
4            self.atomic_number = atomic_number
5
6        def __eq__(self, other):
7            """Equal if same atomic number."""
8            return self.atomic_number == other.atomic_number
9
10       def __lt__(self, other):
11           """Less than based on atomic number."""
12           return self.atomic_number < other.atomic_number
13
14       carbon = Element("C", 6)
15       oxygen = Element("O", 8)
16
17       print(carbon < oxygen)    # True
18       print(carbon == oxygen)  # False
19
20       elements = [oxygen, carbon]
21       print(sorted(elements))  # sorts by atomic number

```

8.5.3. Arithmetic Methods

Define what +, -, * do for your objects:

```

1    class Vector:
2        def __init__(self, x, y, z):
3            self.x = x
4            self.y = y
5            self.z = z
6
7        def __add__(self, other):
8            return Vector(self.x + other.x, self.y + other.y, self.z + other
.z)
9
10       def __mul__(self, scalar):

```

```

11         return Vector(self.x * scalar, self.y * scalar, self.z * scalar)
12
13     def __str__(self):
14         return f"({self.x}, {self.y}, {self.z})"
15
16 v1 = Vector(1, 2, 3)
17 v2 = Vector(4, 5, 6)
18 print(v1 + v2)      # (5, 7, 9)
19 print(v1 * 2)       # (2, 4, 6)

```

8.6. Encapsulation

Encapsulation means hiding internal details and providing a clean interface:

```

1 class Thermometer:
2     def __init__(self):
3         self._temperature_c = 0 # underscore suggests "private"
4
5     def set_celsius(self, temp):
6         if temp < -273.15:
7             raise ValueError("Temperature below absolute zero!")
8         self._temperature_c = temp
9
10    def get_celsius(self):
11        return self._temperature_c
12
13    def get_kelvin(self):
14        return self._temperature_c + 273.15
15
16    def get_fahrenheit(self):
17        return self._temperature_c * 9/5 + 32

```

The underscore prefix (`_temperature_c`) is a convention meaning “don’t access this directly.” Users should use the methods, which can validate input.

Note

Python doesn’t enforce privacy—you can still access `_temperature_c`. The underscore is a polite suggestion, not a lock.

8.7. Inheritance

Inheritance lets you create new classes based on existing ones:

```

1 class Compound:
2     """Base class for chemical compounds."""
3
4     def __init__(self, formula, molar_mass):
5         self.formula = formula
6         self.molar_mass = molar_mass
7
8     def describe(self):
9         print(f"{self.formula}: {self.molar_mass} g/mol")
10
11
12 class Acid(Compound):

```

```

13     """An acid compound with pKa."""
14
15     def __init__(self, formula, molar_mass, pKa):
16         super().__init__(formula, molar_mass) # call parent's __init__
17         self.pKa = pKa
18
19     def describe(self):
20         super().describe() # call parent's describe
21         print(f"    pKa = {self.pKa}")
22
23     def Ka(self):
24         return 10 ** (-self.pKa)
25
26
27 acetic = Acid("CH3COOH", 60.05, 4.76)
28 acetic.describe()
29 # CH3COOH: 60.05 g/mol
30 #    pKa = 4.76
31
32 print(f"Ka = {acetic.Ka():.2e}") # Ka = 1.74e-05

```

Acid inherits from Compound:

- It gets `formula` and `molar_mass` from `Compound`
- `super()` calls the parent class's methods
- It adds its own `pKa` attribute and `Ka()` method
- It can override methods (like `describe`)

8.8. Decorators and Dataclasses

When reading Python code, you'll encounter the `@` symbol above function or class definitions. This is *decorator* syntax—a way to modify or enhance functions and classes.

8.8.1. What Are Decorators?

A decorator is a function that takes another function (or class) and returns a modified version of it. The `@decorator` syntax is shorthand:

```

1 @some_decorator
2 def my_function():
3     pass
4
5 # This is equivalent to:
6 def my_function():
7     pass
8 my_function = some_decorator(my_function)

```

You don't need to write your own decorators, but you'll *use* many from libraries. Common examples include `@property`, `@staticmethod`, and `@classmethod` in classes, plus countless decorators in frameworks like `Flask` or `pytest`.

8.8.2. The `@dataclass` Decorator

Python's `dataclasses` module provides a decorator that automatically generates common methods for classes that primarily store data:

```
1 from dataclasses import dataclass
2
3 @dataclass
4 class Element:
5     symbol: str
6     name: str
7     atomic_number: int
8     atomic_mass: float
9
10 # The decorator automatically creates:
11 # - __init__ (so you can write Element("C", "Carbon", 6, 12.011))
12 # - __repr__ (nice string representation)
13 # - __eq__ (comparison by values)
14
15 carbon = Element("C", "Carbon", 6, 12.011)
16 print(carbon)
17 # Element(symbol='C', name='Carbon', atomic_number=6, atomic_mass
18 #         =12.011)
19
20 oxygen = Element("O", "Oxygen", 8, 15.999)
21 print(carbon == oxygen) # False
```

Compare this to the manual version, which requires much more code:

```
1 class Element:
2     def __init__(self, symbol, name, atomic_number, atomic_mass):
3         self.symbol = symbol
4         self.name = name
5         self.atomic_number = atomic_number
6         self.atomic_mass = atomic_mass
7
8     def __repr__(self):
9         return (f"Element(symbol='{self.symbol}', name='{self.name}', "
10                f"atomic_number={self.atomic_number}, "
11                f"atomic_mass={self.atomic_mass})")
12
13     def __eq__(self, other):
14         return (self.symbol == other.symbol and
15                self.name == other.name and
16                self.atomic_number == other.atomic_number and
17                self.atomic_mass == other.atomic_mass)
```

Lab Example

Dataclasses are perfect for scientific metadata:

```
1 from dataclasses import dataclass
2 from datetime import datetime
3
4 @dataclass
5 class Measurement:
6     sample_id: str
7     value: float
8     uncertainty: float
9     timestamp: datetime
10    instrument: str = "UV-Vis" # default value
11
12 reading = Measurement(
13     sample_id="S001",
14     value=0.452,
15     uncertainty=0.003,
16     timestamp=datetime.now()
17 )
18 print(reading)
```

8.8.3. Dataclasses vs Regular Classes

Use a **dataclass** when:

- The class primarily holds data
- You want automatic `__init__`, `__repr__`, `__eq__`
- Behavior is minimal or straightforward

Use a **regular class** when:

- You need complex initialization logic
- The class has significant custom behavior
- You need fine control over special methods

Tip

When in doubt, start with a dataclass. You can always convert to a regular class later if you need more control. Dataclasses reduce boilerplate and make your intent clear: “this class is primarily a data container.”

8.9. When to Use Classes

Classes are useful when:

- You have data that naturally belongs together (element properties, sample metadata)
- You need multiple instances with the same structure
- You have functions that always operate on the same data
- You want to model real-world entities (samples, instruments, experiments)

Don't use classes when:

- A simple function would do

- You only need one instance
- The data doesn't have associated behavior

Lab Example

A class for spectroscopy data:

```

1  class Spectrum:
2      def __init__(self, wavelengths, absorbances, name=""):
3          self.wavelengths = wavelengths
4          self.absorbances = absorbances
5          self.name = name
6
7      def peak_wavelength(self):
8          """Find wavelength of maximum absorbance."""
9          max_idx = self.absorbances.index(max(self.absorbances))
10         return self.wavelengths[max_idx]
11
12     def absorbance_at(self, wavelength):
13         """Get absorbance at specific wavelength (nearest point)."""
14         closest_idx = min(range(len(self.wavelengths)),
15                             key=lambda i: abs(self.wavelengths[i] -
wavelength))
16         return self.absorbances[closest_idx]
17
18     def normalize(self):
19         """Return new spectrum normalized to max = 1."""
20         max_abs = max(self.absorbances)
21         normalized = [a / max_abs for a in self.absorbances]
22         return Spectrum(self.wavelengths.copy(), normalized, self.
name)
23
24
25 # Using the class
26 wl = [400, 420, 440, 460, 480, 500]
27 ab = [0.1, 0.3, 0.8, 0.6, 0.4, 0.2]
28
29 spectrum = Spectrum(wl, ab, "Sample A")
30 print(f"Peak at {spectrum.peak_wavelength()} nm")
31 print(f"A at 450nm: {spectrum.absorbance_at(450):.2f}")
32
33 normalized = spectrum.normalize()
34 print(f"Normalized peak: {max(normalized.absorbances)}")

```

8.10. Chapter Summary

Object-oriented programming bundles data with behavior:

- **Class:** Blueprint for objects (`class MyClass:`)
- **Object:** Instance of a class (`obj = MyClass()`)
- **Attributes:** Data stored on objects (`self.data`)
- **Methods:** Functions belonging to objects
- **`__init__`:** Constructor, called when creating objects
- **`self`:** Reference to the current object
- **Inheritance:** Creating classes based on other classes

Things to Ponder

1. What's the difference between a class and an object?
2. Why does every method need `self` as its first parameter?
3. When would you use inheritance?
4. Design a class to represent a laboratory sample with properties like ID, collection date, and concentration. What methods might it have?

Part III.

Scientific Python

Chapter 9.

NumPy: Numerical Computing

Python lists are flexible but slow for numerical work. NumPy (Numerical Python) provides arrays—efficient containers for numerical data—and fast operations on them. NumPy is the foundation of scientific Python; virtually every scientific library builds on it. You can install it from your terminal, preferably with a virtual environment active, via:

```
1 pip install numpy
```

9.1. Why NumPy?

Compare adding two lists of numbers:

```
1 # Pure Python - slow
2 a = [1, 2, 3, 4, 5]
3 b = [10, 20, 30, 40, 50]
4 result = [x + y for x, y in zip(a, b)]
5
6 # NumPy - fast
7 import numpy as np
8 a = np.array([1, 2, 3, 4, 5])
9 b = np.array([10, 20, 30, 40, 50])
10 result = a + b # array([11, 22, 33, 44, 55])
```

NumPy is faster because:

- Arrays store data in contiguous memory
- Operations are implemented in optimized C code
- Operations work on entire arrays at once (“vectorization”)

For large datasets, NumPy can be 100x faster than pure Python.

9.2. Creating Arrays

Import NumPy with the conventional alias:

```
1 import numpy as np
```

9.2.1. From Lists

```
1 # 1D array
2 a = np.array([1, 2, 3, 4, 5])
3
4 # 2D array (matrix)
5 b = np.array([[1, 2, 3],
6               [4, 5, 6]])
7
8 # Note: all elements must be the same type
```

```

9 c = np.array([1, 2, 3.0]) # all become floats
10 print(c.dtype) # float64

```

9.2.2. With Functions

```

1 # Zeros and ones
2 np.zeros(5) # array([0., 0., 0., 0., 0.])
3 np.ones((2, 3)) # 2x3 array of ones
4 np.full((2, 2), 7) # 2x2 array filled with 7
5
6 # Ranges
7 np.arange(0, 10, 2) # array([0, 2, 4, 6, 8])
8 np.linspace(0, 1, 5) # array([0., 0.25, 0.5, 0.75, 1.])
9
10 # Identity matrix
11 np.eye(3) # 3x3 identity matrix
12
13 # Random numbers
14 np.random.rand(3) # 3 uniform random numbers [0, 1)
15 np.random.randn(3) # 3 standard normal random numbers
16 np.random.randint(0, 10, 5) # 5 random integers from 0-9

```

Tip

`np.linspace(start, stop, num)` creates exactly `num` evenly spaced points including both end-points. `np.arange(start, stop, step_size)` creates values from `start` to `end-step_size`, with values differing by `step_size`. Which is better really depends on usecase and needs.

9.3. Array Properties

```

1 a = np.array([[1, 2, 3],
2               [4, 5, 6]])
3
4 a.shape # (2, 3) - 2 rows, 3 columns
5 a.ndim # 2 - number of dimensions
6 a.size # 6 - total number of elements
7 a.dtype # dtype('int64') - data type

```

Common data types:

- `int32`, `int64`: Integers
- `float32`, `float64`: Floating point (64 is default)
- `bool`: Boolean
- `complex128`: Complex numbers

Specify type when creating:

```

1 a = np.array([1, 2, 3], dtype=np.float32)

```

Tip

The numbers (32, 64) indicate how many **bits** (1s and 0s) are used to store each value. A `float64` uses twice the memory of a `float32`, allowing for much higher precision and larger ranges. For example, an `int8` can only hold numbers from -128 to 127. If you try to store 500, it simply won't fit! Meanwhile, `int16` can store -32,768 to 32,767. While 64-bit floats are generally a safe default, the curious can find the technical details of how these are stored by searching for “[Floating-point arithmetic](#),” or following that link to the wikipedia page on the topic.

9.4. Indexing and Slicing

NumPy indexing extends Python's list indexing:

9.4.1. 1D Arrays

```

1 a = np.array([10, 20, 30, 40, 50])
2
3 a[0]          # 10 (first element)
4 a[-1]         # 50 (last element)
5 a[1:4]        # array([20, 30, 40])
6 a[::2]        # array([10, 30, 50]) - every other

```

9.4.2. 2D Arrays

```

1 b = np.array([[1, 2, 3],
2              [4, 5, 6],
3              [7, 8, 9]])
4
5 b[0, 0]        # 1 (row 0, col 0)
6 b[1, 2]        # 6 (row 1, col 2)
7 b[0]          # array([1, 2, 3]) - first row
8 b[:, 0]        # array([1, 4, 7]) - first column
9 b[0:2, 1:3]    # 2x2 subarray: [[2, 3], [5, 6]]

```

9.4.3. Boolean Indexing

Select elements that meet a condition:

```

1 a = np.array([1, 5, 3, 8, 2, 9])
2
3 a[a > 4]       # array([5, 8, 9])
4 a[a % 2 == 0]  # array([8, 2]) - even numbers
5
6 # Modify based on condition
7 a[a > 5] = 0    # set values > 5 to 0
8 print(a)       # array([1, 5, 3, 0, 2, 0])

```

Lab Example

Filtering spectroscopy data:

```
1 wavelengths = np.linspace(400, 700, 100)
2 absorbances = np.random.rand(100) * 0.5
3
4 # Find wavelengths where absorbance > 0.3
5 mask = absorbances > 0.3
6 high_abs_wavelengths = wavelengths[mask]
7 print(f"High absorbance at {len(high_abs_wavelengths)} wavelengths")
```

9.5. Array Operations

9.5.1. Element-wise Operations

Operations apply to every element:

```
1 a = np.array([1, 2, 3, 4])
2 b = np.array([10, 20, 30, 40])
3
4 a + b      # array([11, 22, 33, 44])
5 a - b      # array([-9, -18, -27, -36])
6 a * b      # array([10, 40, 90, 160])
7 a / b      # array([0.1, 0.1, 0.1, 0.1])
8 a ** 2     # array([1, 4, 9, 16])
9 np.sqrt(a) # array([1., 1.41..., 1.73..., 2.] )
```

9.5.2. Mathematical Functions

Vectorized functions are orders of magnitude faster for large volumes of high dimensional data than functions applied in loops would be. Scripts that can take hours using loops might take seconds with vectorized functions. NumPy provides vectorized versions of many common math functions, examples include:

```
1 x = np.array([0, np.pi/4, np.pi/2, np.pi])
2
3 np.sin(x)      # sine
4 np.cos(x)      # cosine
5 np.exp(x)      # e^x
6 np.log(x)      # natural log (watch out for log(0)!)
7 np.log10(x)    # base-10 log
8 np.abs(x)      # absolute value
```

Tip

It is a good idea to search up "numpy documentation" to have a look at what other functions and operations are available through numpy. In your programming journey, getting comfortable with finding and making use of documentation is among the most important skills you will have. It is well beyond the scope of this book to give an exhaustive insight into all NumPy has to offer.

9.5.3. Aggregation Functions

Summarize array data:

```

1 a = np.array([1, 5, 3, 8, 2])
2
3 np.sum(a)      # 19
4 np.mean(a)     # 3.8
5 np.std(a)      # standard deviation
6 np.var(a)      # variance
7 np.min(a)      # 1
8 np.max(a)      # 8
9 np.argmin(a)   # 0 (index of minimum)
10 np.argmax(a)  # 3 (index of maximum)

```

For 2D arrays, specify axis:

```

1 b = np.array([[1, 2, 3],
2              [4, 5, 6]])
3
4 np.sum(b)      # 21 (all elements)
5 np.sum(b, axis=0) # array([5, 7, 9]) - sum each column
6 np.sum(b, axis=1) # array([6, 15]) - sum each row

```

9.6. Broadcasting

NumPy can operate on arrays of different shapes through *broadcasting*:

```

1 a = np.array([1, 2, 3])
2 b = 10
3
4 a + b      # array([11, 12, 13]) - scalar broadcast to all elements
5 a * b      # array([10, 20, 30])

```

More complex broadcasting:

```

1 # 2D array + 1D array
2 matrix = np.array([[1, 2, 3],
3                   [4, 5, 6]])
4 row = np.array([10, 20, 30])
5
6 matrix + row
7 # array([[11, 22, 33],
8 #        [14, 25, 36]])

```

The 1D array is “broadcast” across each row of the 2D array.

Lab Example

Baseline subtraction:

```

1 # Spectrum with baseline
2 wavelengths = np.linspace(400, 700, 100)
3 raw_spectrum = np.random.rand(100) + 0.2 # signal + offset
4
5 # Subtract baseline (minimum value)
6 baseline = np.min(raw_spectrum)
7 corrected = raw_spectrum - baseline # broadcasts scalar

```

9.7. Reshaping Arrays

```

1 a = np.arange(12)  # array([0, 1, 2, ..., 11])
2
3 # Reshape to 3x4
4 b = a.reshape(3, 4)
5 # array([[ 0,  1,  2,  3],
6 #        [ 4,  5,  6,  7],
7 #        [ 8,  9, 10, 11]])
8
9 # Reshape to 2x6
10 c = a.reshape(2, 6)
11
12 # Flatten back to 1D
13 d = b.flatten()  # or b.ravel()
14
15 # Transpose (swap rows and columns)
16 e = b.T
17 # array([[ 0,  4,  8],
18 #        [ 1,  5,  9],
19 #        [ 2,  6, 10],
20 #        [ 3,  7, 11]])

```

Warning

`reshape()` requires the total number of elements to stay the same. You can't reshape a 12-element array into 3x5 (15 elements).

9.8. Stacking and Splitting

```

1 a = np.array([1, 2, 3])
2 b = np.array([4, 5, 6])
3
4 # Stack vertically (row by row)
5 np.vstack([a, b])
6 # array([[1, 2, 3],
7 #        [4, 5, 6]])
8
9 # Stack horizontally (column by column)
10 np.hstack([a, b])
11 # array([1, 2, 3, 4, 5, 6])
12
13 # Concatenate along an axis
14 np.concatenate([a, b])  # same as hstack for 1D

```

Splitting:

```

1 c = np.arange(12).reshape(3, 4)
2
3 # Split into 3 parts along rows
4 np.vsplit(c, 3)  # list of three 1x4 arrays
5
6 # Split into 2 parts along columns
7 np.hsplit(c, 2)  # list of two 3x2 arrays

```


9.9. Linear Algebra

NumPy provides linear algebra operations:

```

1 a = np.array([[1, 2],
2               [3, 4]])
3 b = np.array([[5, 6],
4               [7, 8]])
5
6 # Matrix multiplication
7 np.dot(a, b)      # or a @ b
8 # array([[19, 22],
9 #        [43, 50]])
10
11 # Transpose
12 a.T
13
14 # Determinant
15 np.linalg.det(a)  # -2.0
16
17 # Inverse
18 np.linalg.inv(a)
19
20 # Eigenvalues and eigenvectors
21 eigenvalues, eigenvectors = np.linalg.eig(a)
22
23 # Solve linear system Ax = b
24 A = np.array([[3, 1], [1, 2]])
25 b = np.array([9, 8])
26 x = np.linalg.solve(A, b)  # x such that A @ x = b

```

Lab Example

Linear regression using linear algebra:

```

1 # Data: concentration vs absorbance
2 conc = np.array([0.1, 0.2, 0.3, 0.4, 0.5])
3 absorbance = np.array([0.12, 0.25, 0.35, 0.48, 0.61])
4
5 # Set up design matrix for y = mx + c
6 # [1 x1]      [c]      [y1]
7 # [1 x2] @ [m] = [y2]
8 # ...      ...
9 A = np.column_stack([np.ones_like(conc), conc])
10 b = absorbance
11
12 # Solve least squares: (A^T A) x = A^T b
13 coeffs = np.linalg.lstsq(A, b, rcond=None)[0]
14 intercept, slope = coeffs
15
16 print(f"Beer's Law: A = {slope:.3f} * C + {intercept:.3f}")
17 print(f"Molar absorptivity (slope): {slope:.3f} L/(mol*cm)")

```

9.10. Saving and Loading Data

```

1 data = np.random.rand(100, 5)

```

```

2
3 # NumPy binary format (fast, preserves precision)
4 np.save("data.npy", data)
5 loaded = np.load("data.npy")
6
7 # Text format (human-readable)
8 np.savetxt("data.csv", data, delimiter=",")
9 loaded = np.loadtxt("data.csv", delimiter=",")
10
11 # Save multiple arrays
12 np.savez("multiple.npz", wavelengths=wl, absorbances=ab)
13 loaded = np.load("multiple.npz")
14 print(loaded["wavelengths"])

```

9.11. Common Patterns

9.11.1. Normalize Data

```

1 data = np.array([10, 20, 30, 40, 50])
2
3 # Scale to 0-1
4 normalized = (data - data.min()) / (data.max() - data.min())
5
6 # Z-score normalization
7 z_scores = (data - data.mean()) / data.std()

```

9.11.2. Moving Average

```

1 def moving_average(data, window):
2     return np.convolve(data, np.ones(window)/window, mode='valid')
3
4 smoothed = moving_average(noisy_data, window=5)

```

9.11.3. Finding Peaks

```

1 from scipy.signal import find_peaks # scipy is covered more in a later
   section
2
3 data = np.array([1, 3, 7, 1, 2, 6, 4, 2])
4 peaks, _ = find_peaks(data, height=5) # indices where peaks > 5
5 print(f"Peaks at indices: {peaks}")

```

9.12. Chapter Summary

NumPy is essential for scientific computing:

- **Arrays:** `np.array()`, `np.zeros()`, `np.linspace()`
- **Properties:** `.shape`, `.dtype`, `.size`
- **Indexing:** `a[0]`, `a[1:5]`, `a[a > 0]`
- **Operations:** Element-wise (+, *), functions (`np.sin`, `np.exp`)
- **Aggregation:** `np.sum()`, `np.mean()`, `np.std()`

- **Linear algebra:** `@` or `np.dot()`, `np.linalg.solve()`
- **Reshaping:** `.reshape()`, `.T`, `np.vstack()`

Things to Ponder

1. Why is `a * b` element-wise multiplication for arrays but `a @ b` is matrix multiplication?
2. How would you select all values in an array that are between 5 and 10?
3. What does `axis=0` vs `axis=1` mean for a 2D array?
4. Write code to normalize an array so its values sum to 1.

Chapter 10.

Pandas: Data Analysis

Real scientific data comes with labels, missing values, and mixed types. Pandas provides the `DataFrame`—a powerful table structure that handles these complexities. If NumPy is like a matrix, Pandas is like a spreadsheet with superpowers. Install pandas via:

```
1 pip install pandas
```

10.1. Why Pandas?

NumPy arrays are great for homogeneous numerical data. But experimental data often includes:

- Column names (wavelength, absorbance, sample_id)
- Mixed types (numbers, text, dates)
- Missing values
- Row labels (sample names, timestamps)

Pandas handles all of this with two main structures: `Series` (1D) and `DataFrame` (2D table).

```
1 import pandas as pd
2 import numpy as np
```

10.2. Series

A `Series` is a labeled one-dimensional array:

```
1 # From a list
2 concentrations = pd.Series([0.1, 0.2, 0.3, 0.4])
3 print(concentrations)
4 # 0    0.1
5 # 1    0.2
6 # 2    0.3
7 # 3    0.4
8 # dtype: float64
9
10 # With custom labels
11 conc = pd.Series([0.1, 0.2, 0.3], index=["low", "medium", "high"])
12 print(conc["medium"]) # 0.2
```

10.3. DataFrames

A `DataFrame` is a labeled two-dimensional table—the workhorse of Pandas:

```
1 # From a dictionary
2 data = {
3     "sample": ["A", "B", "C", "D"],
4     "concentration": [0.1, 0.2, 0.3, 0.4],
```

```

5     "absorbance": [0.12, 0.25, 0.35, 0.48]
6 }
7 df = pd.DataFrame(data)
8 print(df)
9 #   sample  concentration  absorbance
10 #  0      A              0.1         0.12
11 #  1      B              0.2         0.25
12 #  2      C              0.3         0.35
13 #  3      D              0.4         0.48

```

10.3.1. Reading Data from Files

```

1 # CSV (most common)
2 df = pd.read_csv("data.csv")
3
4 # Excel
5 df = pd.read_excel("data.xlsx", sheet_name="Sheet1")
6
7 # With options
8 df = pd.read_csv("data.csv",
9                  sep=";",          # semicolon delimiter
10                 skiprows=2,       # skip header rows
11                 na_values=["NA", "-999"]) # missing value codes

```

10.3.2. Quick Look at Data

```

1 df.head()          # first 5 rows
2 df.tail(3)         # last 3 rows
3 df.shape           # (rows, columns)
4 df.columns         # column names
5 df.dtypes          # data types
6 df.info()          # summary of structure
7 df.describe()      # statistics for numeric columns

```

10.4. Selecting Data

10.4.1. Selecting Columns

```

1 # Single column (returns Series)
2 df["concentration"]
3
4 # Multiple columns (returns DataFrame)
5 df[["sample", "concentration"]]
6
7 # As attribute (only for simple column names)
8 df.concentration

```

10.4.2. Selecting Rows

```

1 # By position
2 df.iloc[0]          # first row
3 df.iloc[0:3]        # first three rows

```

```

4 df.iloc[[0, 2]] # rows 0 and 2
5
6 # By label
7 df.loc[0]       # row with label 0
8 df.loc[0:2]     # rows with labels 0, 1, 2 (inclusive!)
9
10 # With conditions (boolean indexing)
11 df[df["concentration"] > 0.2]
12 df[df["sample"].isin(["A", "C"])]

```

10.4.3. Selecting Rows and Columns

```

1 # By position: df.iloc[rows, columns]
2 df.iloc[0:2, 1:3] # first 2 rows, columns 1-2
3
4 # By label: df.loc[rows, columns]
5 df.loc[0:2, "concentration":"absorbance"]
6
7 # Mixed
8 df.loc[df["concentration"] > 0.2, ["sample", "absorbance"]]

```

Tip

Use `.loc` for label-based indexing (including boolean conditions) and `.iloc` for position-based indexing. Mixing them up causes confusion.

10.5. Modifying Data

10.5.1. Adding Columns

```

1 # New column from calculation
2 df["transmittance"] = 10 ** (-df["absorbance"]) * 100
3
4 # New column from function
5 df["log_conc"] = np.log10(df["concentration"])

```

10.5.2. Modifying Values

```

1 # Set specific cells
2 df.loc[0, "concentration"] = 0.15
3
4 # Set based on condition
5 df.loc[df["absorbance"] > 0.4, "quality"] = "high"
6
7 # Apply function to column
8 df["sample"] = df["sample"].str.lower()

```

10.5.3. Renaming

```

1 df.rename(columns={"concentration": "conc_M"}, inplace=True)

```

10.5.4. Dropping

Apply the following then inspect your dataframe.

```
1 # Drop columns
2 df.drop(columns=["transmittance"], inplace=True)
3
4 # Drop rows
5 df.drop(index=[0, 1], inplace=True)
```

What has happened to the data?

10.6. Handling Missing Data

Missing values appear as NaN (Not a Number):

```
1 # Check for missing values
2 df.isna()           # boolean mask
3 df.isna().sum()     # count per column
4
5 # Drop rows with any missing values
6 df.dropna()
7
8 # Drop rows where specific columns are missing
9 df.dropna(subset=["absorbance"])
10
11 # Fill missing values
12 df.fillna(0)         # fill with 0
13 df.fillna(df.mean()) # fill with column means
14 df.fillna(method="ffill") # forward fill
```

Lab Example

Handling missing readings in experimental data:

```
1 # Data with missing values (-999 indicates error)
2 data = pd.read_csv("measurements.csv")
3 data.replace(-999, np.nan, inplace=True)
4
5 # Report missing data
6 missing = data.isna().sum()
7 print(f"Missing values:\n{missing}")
8
9 # Remove rows with missing absorbance
10 clean = data.dropna(subset=["absorbance"])
11 print(f"Kept {len(clean)} of {len(data)} rows")
```

10.7. Aggregation and Grouping

10.7.1. Basic Statistics

```
1 df["absorbance"].mean()
2 df["absorbance"].std()
3 df["absorbance"].min()
4 df["absorbance"].max()
5 df["concentration"].sum()
```



```
6 df["sample"].value_counts() # count unique values
```

10.7.2. Group By

Split data into groups and compute statistics:

```
1 # Sample data
2 experiments = pd.DataFrame({
3     "treatment": ["A", "A", "B", "B", "A", "B"],
4     "replicate": [1, 2, 1, 2, 3, 3],
5     "yield": [23, 25, 31, 33, 24, 35]
6 })
7
8 # Group and aggregate
9 grouped = experiments.groupby("treatment")
10 print(grouped["yield"].mean())
11 # treatment
12 # A      24.0
13 # B      33.0
14
15 # Multiple aggregations
16 experiments.groupby("treatment")["yield"].agg(["mean", "std", "count"])
```

Lab Example

Analyzing replicate measurements:

```
1 data = pd.DataFrame({
2     "sample": ["S1", "S1", "S1", "S2", "S2", "S2"],
3     "absorbance": [0.45, 0.47, 0.44, 0.82, 0.85, 0.81]
4 })
5
6 summary = data.groupby("sample")["absorbance"].agg(
7     mean="mean",
8     std="std",
9     n="count"
10 )
11 summary["sem"] = summary["std"] / np.sqrt(summary["n"])
12 print(summary)
```

10.8. Sorting

```
1 # Sort by column
2 df.sort_values("absorbance")
3 df.sort_values("absorbance", ascending=False)
4
5 # Sort by multiple columns
6 df.sort_values(["treatment", "concentration"])
7
8 # Sort index
9 df.sort_index()
```

10.9. Merging DataFrames

Combine data from multiple sources:

```

1  # Sample data
2  samples = pd.DataFrame({
3      "sample_id": ["A", "B", "C"],
4      "concentration": [0.1, 0.2, 0.3]
5  })
6
7  metadata = pd.DataFrame({
8      "sample_id": ["A", "B", "D"],
9      "temperature": [25, 30, 25]
10 })
11
12 # Merge on common column
13 merged = pd.merge(samples, metadata, on="sample_id", how="inner")
14 # Only keeps rows present in both (A, B)
15
16 # Different join types
17 pd.merge(samples, metadata, how="left")    # keep all from samples
18 pd.merge(samples, metadata, how="right")   # keep all from metadata
19 pd.merge(samples, metadata, how="outer")   # keep all from both

```

10.10. Pivot Tables

Reshape data for analysis:

```

1  data = pd.DataFrame({
2      "date": ["Mon", "Mon", "Tue", "Tue"],
3      "sample": ["A", "B", "A", "B"],
4      "value": [1.2, 2.3, 1.4, 2.5]
5  })
6
7  # Pivot: rows=dates, columns=samples, values=value
8  pivot = data.pivot(index="date", columns="sample", values="value")
9      #      A      B
10 # date
11 # Mon  1.2  2.3
12 # Tue  1.4  2.5

```

10.11. Applying Functions

```

1  # Apply to entire column
2  df["absorbance"].apply(lambda x: x * 1000)  # convert to mAU
3
4  # Apply to each row
5  def classify_sample(row):
6      if row["absorbance"] > 0.5:
7          return "high"
8      return "low"
9
10 df["category"] = df.apply(classify_sample, axis=1)
11
12 # Vectorized operations (faster)
13 df["category"] = np.where(df["absorbance"] > 0.5, "high", "low")

```

10.12. Saving Data

```

1  # To CSV
2  df.to_csv("results.csv", index=False)
3
4  # To Excel
5  df.to_excel("results.xlsx", sheet_name="Data", index=False)
6
7  # Multiple sheets
8  with pd.ExcelWriter("report.xlsx") as writer:
9      raw_data.to_excel(writer, sheet_name="Raw")
10     summary.to_excel(writer, sheet_name="Summary")

```

10.13. Working with Dates and Times

Scientific data is often timestamped: instrument readings, reaction monitoring, environmental measurements. Python and Pandas provide powerful tools for handling temporal data.

10.13.1. Python's datetime Module

The standard library datetime module handles dates and times:

```

1  from datetime import datetime, date, time, timedelta
2
3  # Current date and time
4  now = datetime.now()
5  print(now)    # 2024-03-15 14:30:45.123456
6
7  # Create specific dates/times
8  experiment_date = date(2024, 3, 15)
9  start_time = time(9, 30, 0)    # 9:30:00 AM
10 measurement = datetime(2024, 3, 15, 14, 30, 45)
11
12 # Access components
13 print(measurement.year)    # 2024
14 print(measurement.month)  # 3
15 print(measurement.hour)   # 14
16 print(measurement.weekday()) # 4 (Friday, where Monday=0)

```

Formatting and Parsing

Convert between datetime objects and strings using format codes:

```

1  from datetime import datetime
2
3  # Format datetime as string
4  dt = datetime(2024, 3, 15, 14, 30)
5  formatted = dt.strftime("%Y-%m-%d %H:%M")
6  print(formatted)    # "2024-03-15 14:30"
7
8  # Parse string to datetime
9  text = "15/03/2024 09:45"
10 parsed = datetime.strptime(text, "%d/%m/%Y %H:%M")
11 print(parsed)    # 2024-03-15 09:45:00

```

Common format codes: %Y (4-digit year), %m (month), %d (day), %H (24-hour), %M (minute), %S (second).

Time Differences

Use `timedelta` for durations and arithmetic:

```
1 from datetime import datetime, timedelta
2
3 start = datetime(2024, 3, 15, 9, 0)
4 end = datetime(2024, 3, 15, 17, 30)
5
6 duration = end - start
7 print(duration)           # 8:30:00
8 print(duration.total_seconds()) # 30600.0
9
10 # Add time to a datetime
11 next_reading = start + timedelta(hours=2, minutes=30)
12 print(next_reading)      # 2024-03-15 11:30:00
```

10.13.2. DateTime in Pandas

Pandas extends datetime handling for working with columns of timestamps.

Converting to DateTime

Use `pd.to_datetime()` to parse date columns:

```
1 import pandas as pd
2
3 # From various string formats
4 df = pd.DataFrame({
5     "date_str": ["2024-03-15", "2024-03-16", "2024-03-17"],
6     "value": [1.2, 1.5, 1.3]
7 })
8 df["date"] = pd.to_datetime(df["date_str"])
9
10 # Pandas often infers format automatically
11 dates = pd.to_datetime(["March 15, 2024", "March 16, 2024"])
12
13 # Specify format for speed or unusual formats
14 df["date"] = pd.to_datetime(df["date_str"], format="%Y-%m-%d")
```

The .dt Accessor

Access datetime components using `.dt`:

```
1 df["date"] = pd.to_datetime(df["date_str"])
2
3 df["year"] = df["date"].dt.year
4 df["month"] = df["date"].dt.month
5 df["day_of_week"] = df["date"].dt.dayofweek # Monday=0
6 df["hour"] = df["date"].dt.hour
7
8 # Useful for filtering
9 morning_data = df[df["date"].dt.hour < 12]
10 march_data = df[df["date"].dt.month == 3]
```

DateTime as Index

Setting timestamps as the index enables powerful time series operations:

```
1 # Create time series data
2 data = pd.DataFrame({
3     "timestamp": pd.date_range("2024-03-15", periods=24, freq="h"),
4     "temperature": [20.1, 20.3, 20.0, 19.8, 19.5, 19.2,
5                     19.0, 19.5, 21.0, 23.5, 25.2, 26.8,
6                     27.5, 28.0, 27.8, 27.0, 25.5, 24.0,
7                     22.5, 21.5, 21.0, 20.5, 20.2, 20.0]
8 })
9 data = data.set_index("timestamp")
10
11 # Select by date/time
12 march_15_afternoon = data.loc["2024-03-15 12:00":"2024-03-15 18:00"]
13
14 # Resample: convert hourly to 6-hour averages
15 six_hourly = data.resample("6h").mean()
16 print(six_hourly)
```

Resampling Time Series

Resampling converts data between time frequencies:

```
1 # Common frequency codes:
2 # "D" = daily, "h" = hourly, "min" = minute
3 # "W" = weekly, "ME" = month end, "YE" = year end
4
5 # Downsample: reduce frequency (aggregate)
6 daily_mean = hourly_data.resample("D").mean()
7 daily_max = hourly_data.resample("D").max()
8
9 # Multiple aggregations
10 daily_stats = hourly_data.resample("D").agg(["mean", "min", "max"])
```

Lab Example

Analyzing timestamped instrument readings:

```

1 import pandas as pd
2
3 # Load data with timestamps
4 data = pd.read_csv("instrument_log.csv")
5 data["timestamp"] = pd.to_datetime(data["timestamp"])
6 data = data.set_index("timestamp")
7
8 # Find readings from specific time window
9 morning = data.between_time("08:00", "12:00")
10
11 # Calculate hourly statistics
12 hourly = data["reading"].resample("h").agg(["mean", "std", "count"])
13 print(hourly.head())
14
15 # Find days with high variance (potential issues)
16 daily_std = data["reading"].resample("D").std()
17 problem_days = daily_std[daily_std > daily_std.mean() + 2*daily_std.
    std()]
18 print(f"High-variance days: {len(problem_days)}")

```

10.14. A Complete Analysis Example

```

1 import pandas as pd
2 import numpy as np
3
4 # Load calibration data
5 calib = pd.read_csv("calibration.csv")
6 print("Calibration data:")
7 print(calib.head())
8
9 # Check for missing values
10 if calib.isna().any().any():
11     print("Warning: missing values found")
12     calib = calib.dropna()
13
14 # Calculate statistics by standard
15 stats = calib.groupby("standard_id")["absorbance"].agg(
16     ["mean", "std", "count"]
17 )
18
19 # Load unknown samples
20 unknowns = pd.read_csv("unknowns.csv")
21
22 # Calculate concentration from calibration curve
23 # (assuming linear:  $A = m \cdot C + b$ , pre-calculated)
24 slope = 1.5
25 intercept = 0.02
26 unknowns["concentration"] = (unknowns["absorbance"] - intercept) / slope
27
28 # Add quality flags
29 unknowns["in_range"] = (
30     (unknowns["concentration"] >= calib["concentration"].min()) &

```

```
31     (unknowns["concentration"] <= calib["concentration"].max())
32 )
33
34 # Save results
35 unknowns.to_csv("results.csv", index=False)
36 print(f"\nProcessed {len(unknowns)} samples")
37 print(f"Samples in calibration range: {unknowns['in_range'].sum()}")
```

Tip

As for numpy, you are highly encourage to find and peruse the documentation for pandas in order to find out more about it's features. For example, it offers plotting functionality, which can be useful but is not essential to cover as we will cover plotting in detail next.

10.15. Chapter Summary

Pandas makes data analysis manageable:

- **DataFrame:** Labeled 2D table with mixed types
- **Reading:** `pd.read_csv()`, `pd.read_excel()`
- **Selection:** `df["col"]`, `df.loc[]`, `df.iloc[]`
- **Missing data:** `df.isna()`, `df.dropna()`, `df.fillna()`
- **Grouping:** `df.groupby("col").agg()`
- **Merging:** `pd.merge(df1, df2, on="key")`
- **Saving:** `df.to_csv()`, `df.to_excel()`

Things to Ponder

1. What's the difference between `df.loc[0]` and `df.iloc[0]`?
2. How would you find rows where column A is greater than 5 AND column B equals "yes"?
3. When would you use `groupby()` vs. just calculating `.mean()` on a column?
4. How do the different merge types (inner, left, right, outer) differ?

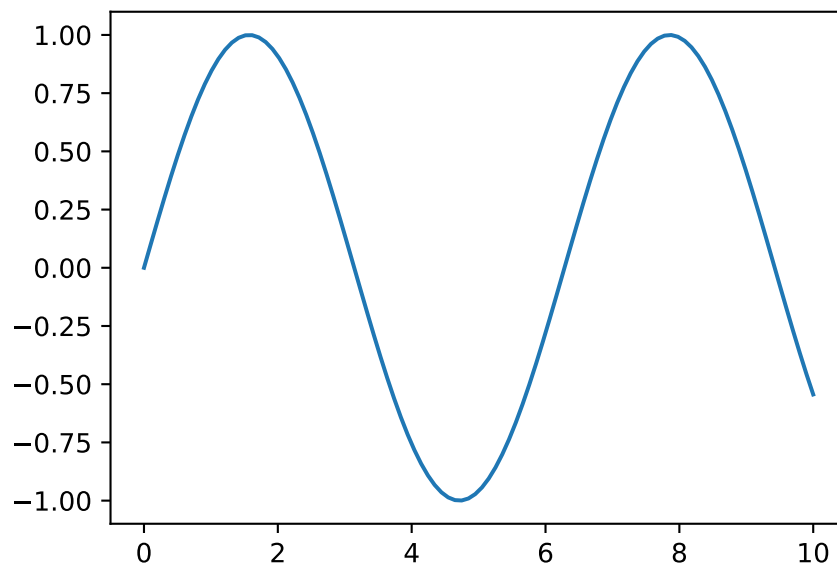
Chapter 11.

Matplotlib: Visualization

Clear figures are essential for understanding data and communicating results. Matplotlib is Python's foundational plotting library—flexible, powerful, and capable of producing publication-quality graphics.

11.1. The Basics

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Create data
5 x = np.linspace(0, 10, 100)
6 y = np.sin(x)
7
8 # Create plot
9 plt.plot(x, y)
10 plt.show()
```

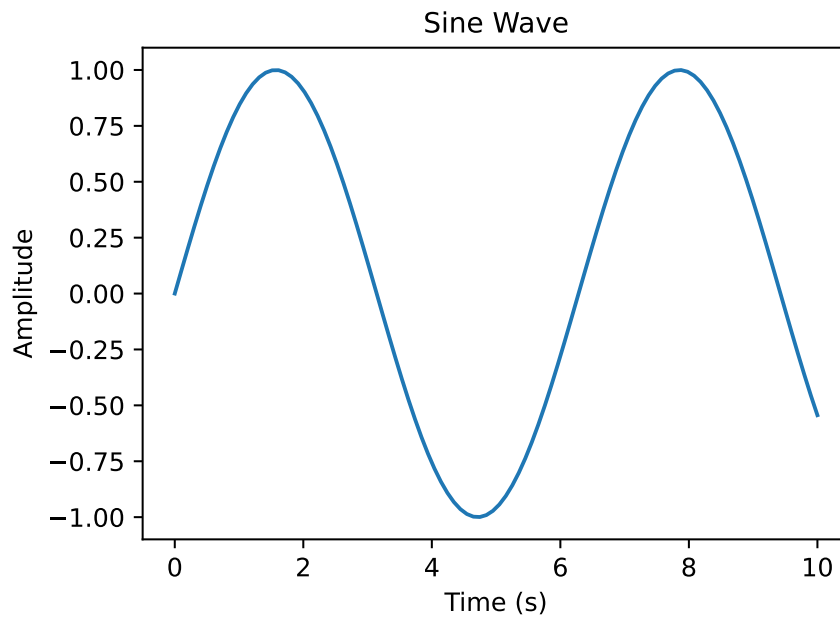


This creates a simple line plot. `plt.show()` displays the figure in a window or inline (in Jupyter notebooks). In spyder there is a "plot" pane which will keep a history of that session's plots for perusing.

11.1.1. Adding Labels and Title

```
1 plt.plot(x, y)
2 plt.xlabel("Time (s)")
3 plt.ylabel("Amplitude")
```

```
4 plt.title("Sine Wave")
5 plt.show()
```



11.1.2. Saving Figures

```
1 plt.plot(x, y)
2 plt.xlabel("Time (s)")
3 plt.ylabel("Amplitude")
4 plt.savefig("sine_wave.png", dpi=300) # 300 DPI for print quality
5 plt.savefig("sine_wave.pdf")         # vector format for publications
```

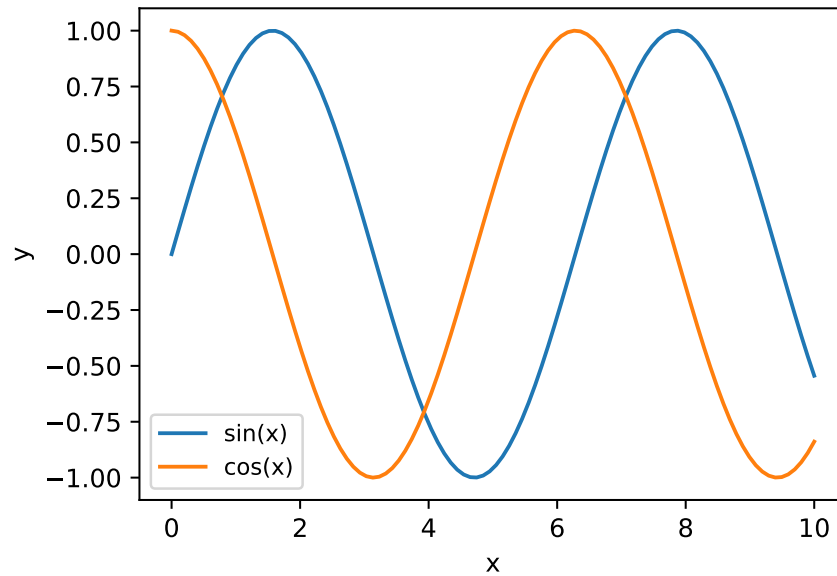
Tip

Save as PDF or SVG for publications—these are vector formats that scale without pixelation. For more information, read up on the difference between raster and vector graphics.

11.2. Line Plots

11.2.1. Multiple Lines

```
1 x = np.linspace(0, 10, 100)
2
3 plt.plot(x, np.sin(x), label="sin(x)")
4 plt.plot(x, np.cos(x), label="cos(x)")
5 plt.xlabel("x")
6 plt.ylabel("y")
7 plt.legend()
8 plt.show()
```



11.2.2. Line Styles and Colors

```
1 plt.plot(x, y, color="red")
2 plt.plot(x, y, color="#1f77b4")      # hex color
3 plt.plot(x, y, linestyle="--")       # dashed
4 plt.plot(x, y, linestyle=":")        # dotted
5 plt.plot(x, y, linewidth=2)          # thicker line
6
7 # Combined format string
8 plt.plot(x, y, "r--")                 # red dashed
9 plt.plot(x, y, "b.-")                 # blue dots with line
10 plt.plot(x, y, "go")                 # green circles (no line)
```

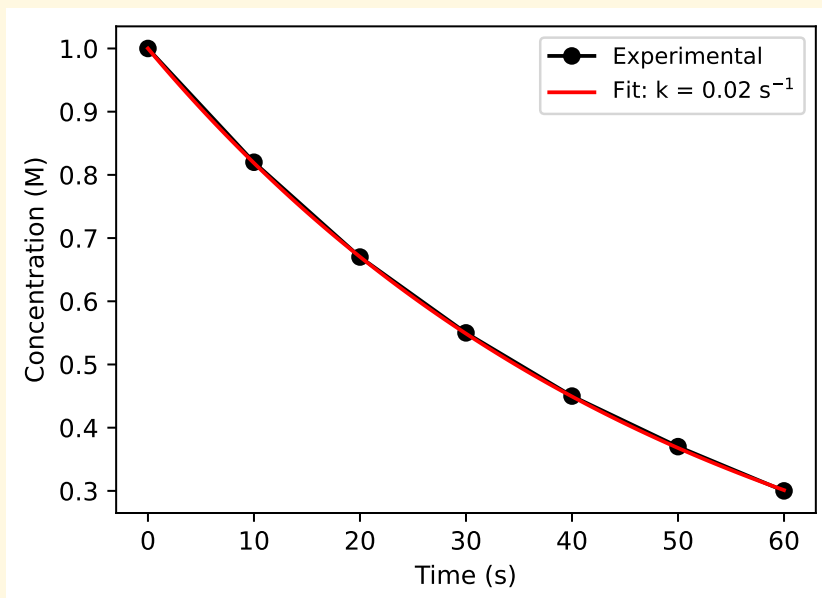
Lab Example

Plotting kinetics data:

```

1 time = np.array([0, 10, 20, 30, 40, 50, 60])
2 concentration = np.array([1.0, 0.82, 0.67, 0.55, 0.45, 0.37, 0.30])
3
4 plt.plot(time, concentration, "ko-", label="Experimental")
5
6 # Add fitted curve
7 k = 0.02
8 t_fit = np.linspace(0, 60, 100)
9 c_fit = 1.0 * np.exp(-k * t_fit)
10 plt.plot(t_fit, c_fit, "r-", label=f"Fit: k = {k} s-1")
11
12 plt.xlabel("Time (s)")
13 plt.ylabel("Concentration (M)")
14 plt.legend()
15 plt.savefig("kinetics.png", dpi=300)

```

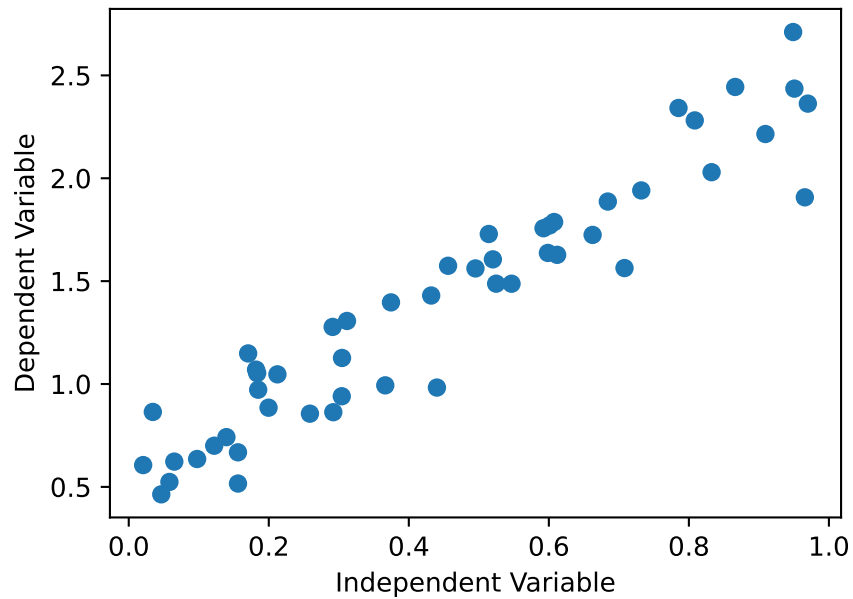


11.3. Scatter Plots

```

1 x = np.random.rand(50)
2 y = 2 * x + 0.5 + np.random.randn(50) * 0.2
3
4 plt.scatter(x, y)
5 plt.xlabel("Independent Variable")
6 plt.ylabel("Dependent Variable")
7 plt.show()

```

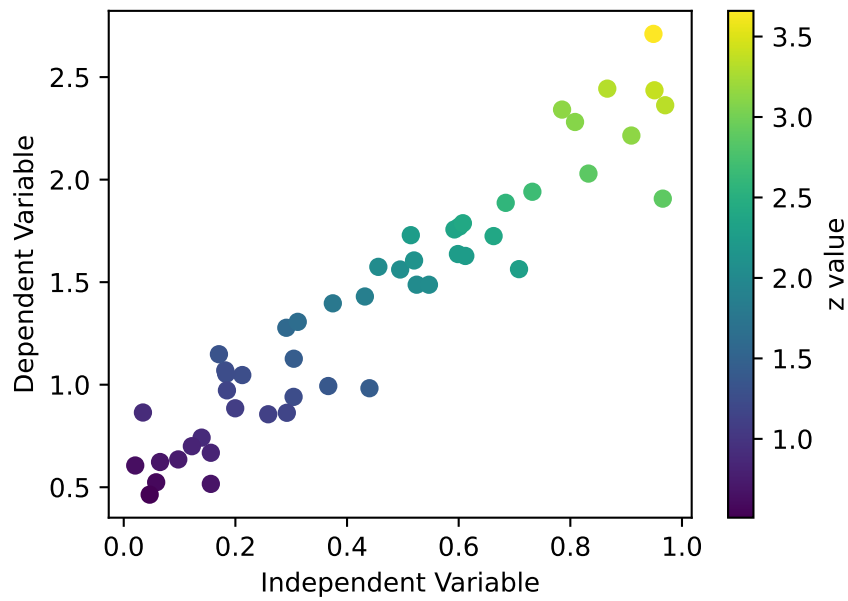


11.3.1. Customizing Markers

```
1 plt.scatter(x, y,  
2             s=50,           # marker size  
3             c="blue",       # color  
4             marker="o",     # marker shape  
5             alpha=0.7,      # transparency  
6             edgecolors="black")
```

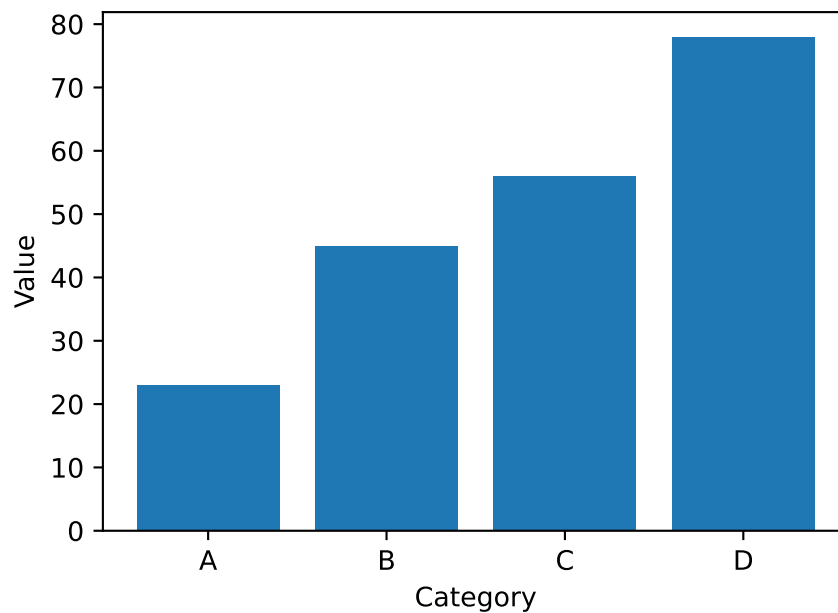
11.3.2. Color by Value

```
1 # Color points by a third variable  
2 z = x + y  
3 plt.scatter(x, y, c=z, cmap="viridis")  
4 plt.colorbar(label="z value")  
5 plt.show()
```



11.4. Bar Charts

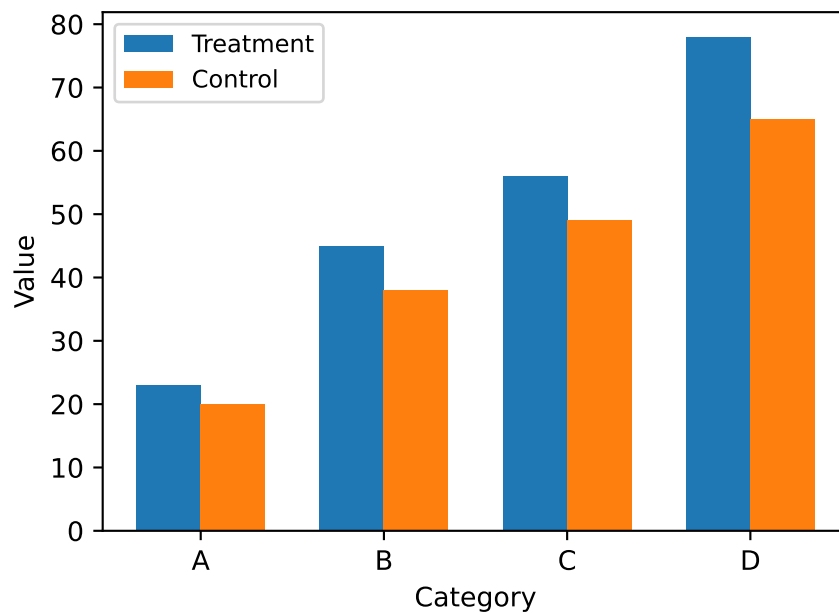
```
1 categories = ["A", "B", "C", "D"]
2 values = [23, 45, 56, 78]
3
4 plt.bar(categories, values)
5 plt.xlabel("Category")
6 plt.ylabel("Value")
7 plt.show()
```



11.4.1. Grouped Bars

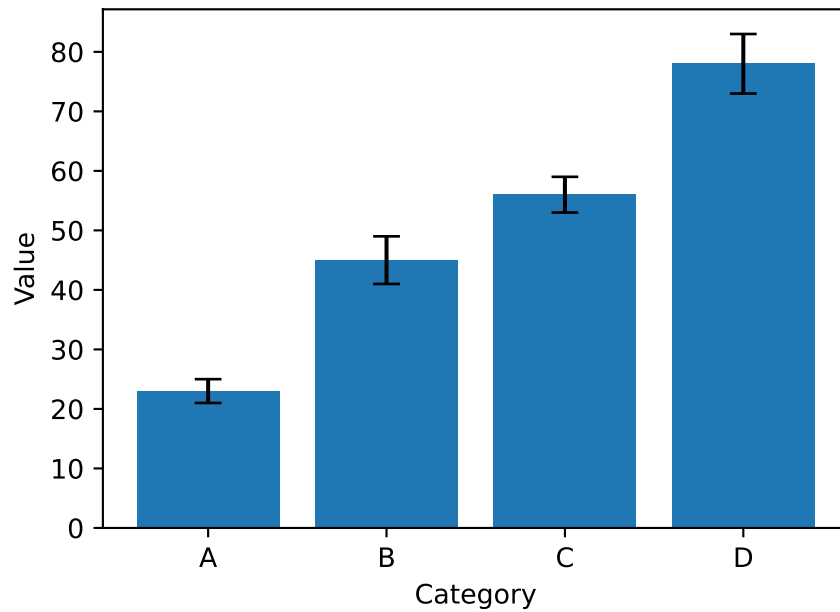
```
1 x = np.arange(4)
2 width = 0.35
```

```
3 treatment = [23, 45, 56, 78]
4 control = [20, 38, 49, 65]
5
6 plt.bar(x - width/2, treatment, width, label="Treatment")
7 plt.bar(x + width/2, control, width, label="Control")
8 plt.xticks(x, ["A", "B", "C", "D"])
9 plt.legend()
10 plt.show()
```



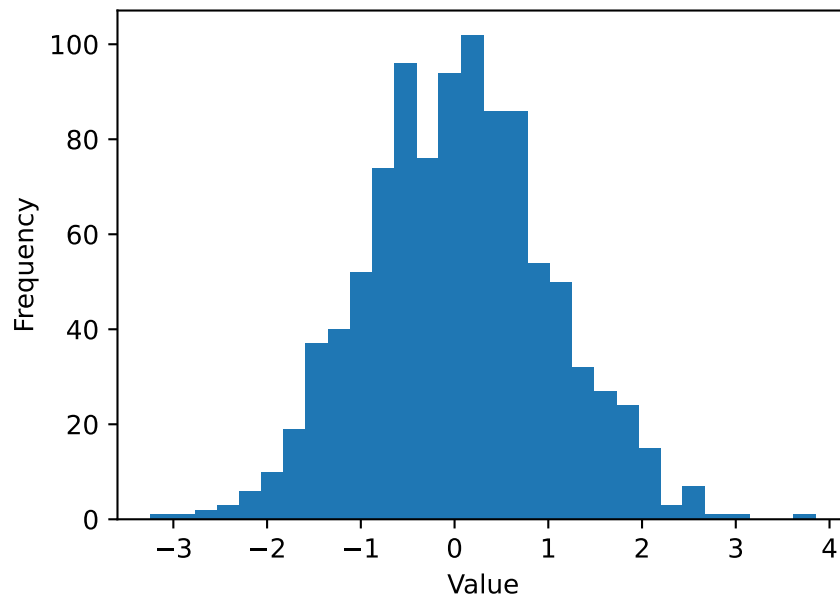
11.4.2. Error Bars

```
1 means = [23, 45, 56, 78]
2 stds = [2, 4, 3, 5]
3
4 plt.bar(categories, means, yerr=stds, capsize=5)
5 plt.ylabel("Value")
6 plt.show()
```



11.5. Histograms

```
1 data = np.random.randn(1000) # 1000 normal random numbers
2
3 plt.hist(data, bins=30)
4 plt.xlabel("Value")
5 plt.ylabel("Frequency")
6 plt.show()
```



11.5.1. Customizing Histograms

```
1 plt.hist(data,
2           bins=30,
3           density=True, # normalize to probability density
```



```

4         alpha=0.7,
5         color="steelblue",
6         edgecolor="black")
7
8 # Overlay multiple histograms
9 plt.hist(data1, bins=30, alpha=0.5, label="Sample 1")
10 plt.hist(data2, bins=30, alpha=0.5, label="Sample 2")
11 plt.legend()

```

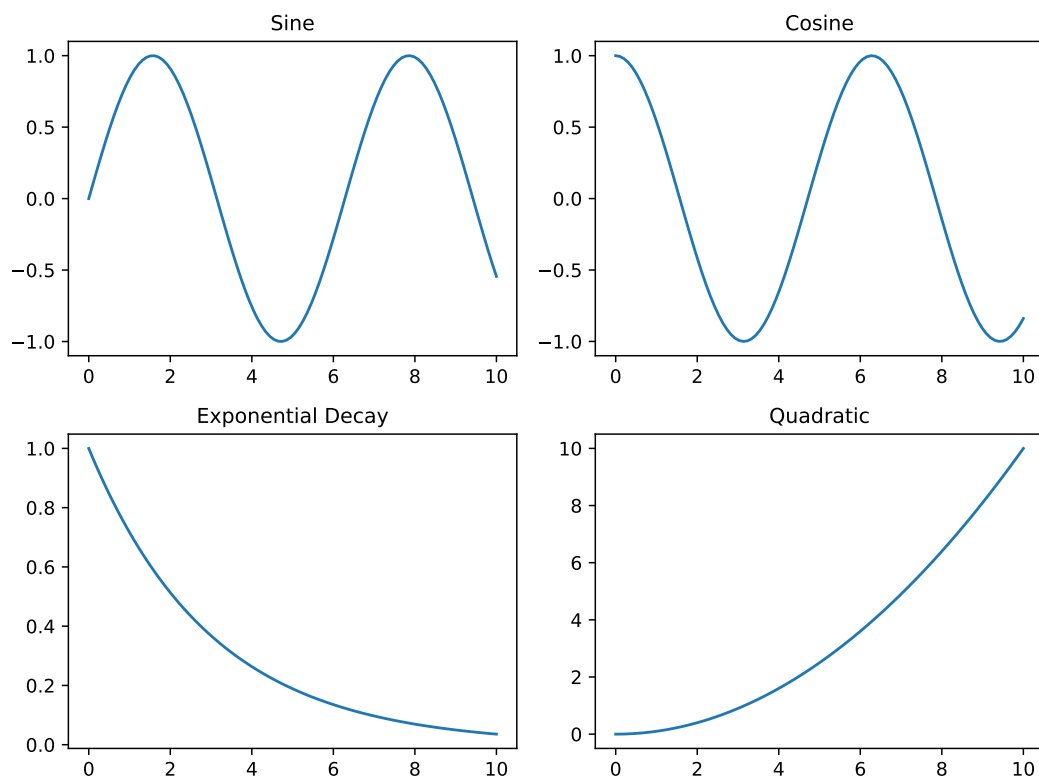
11.6. Subplots

Combine multiple plots in one figure:

```

1 fig, axes = plt.subplots(2, 2, figsize=(10, 8))
2
3 # axes is a 2x2 array of Axes objects
4 axes[0, 0].plot(x, np.sin(x))
5 axes[0, 0].set_title("Sine")
6
7 axes[0, 1].plot(x, np.cos(x))
8 axes[0, 1].set_title("Cosine")
9
10 axes[1, 0].plot(x, np.exp(-x))
11 axes[1, 0].set_title("Exponential Decay")
12
13 axes[1, 1].plot(x, x**2)
14 axes[1, 1].set_title("Quadratic")
15
16 plt.tight_layout() # prevent overlapping
17 plt.savefig("subplots.png")

```



11.6.1. Sharing Axes

```
1 fig, axes = plt.subplots(2, 1, sharex=True)
2
3 axes[0].plot(time, concentration)
4 axes[0].set_ylabel("Concentration (M)")
5
6 axes[1].plot(time, rate)
7 axes[1].set_ylabel("Rate (M/s)")
8 axes[1].set_xlabel("Time (s)")
9
10 plt.tight_layout()
```

11.7. The Object-Oriented Interface

For complex figures, the object-oriented interface gives more control:

```
1 # Create figure and axes explicitly
2 fig, ax = plt.subplots(figsize=(8, 6))
3
4 ax.plot(x, y, "b-", label="Data")
5 ax.set_xlabel("X axis")
6 ax.set_ylabel("Y axis")
7 ax.set_title("My Plot")
8 ax.legend()
9 ax.grid(True, alpha=0.3)
10
11 fig.savefig("plot.png", dpi=300, bbox_inches="tight")
```

Tip

Use `plt.plot()` for quick, simple plots. Use `fig, ax = plt.subplots()` for anything more complex or when you need precise control.

11.8. Customization

11.8.1. Axis Limits and Ticks

```
1 ax.set_xlim(0, 10)
2 ax.set_ylim(-1, 1)
3 ax.set_xticks([0, 2, 4, 6, 8, 10])
4 ax.set_xticklabels(["0", "2", "4", "6", "8", "10 s"])
```

11.8.2. Log Scales

```
1 ax.set_yscale("log")
2 ax.set_xscale("log")
3
4 # Or use specialized plot functions
5 plt.semilogy(x, y)    # log y-axis
6 plt.semilogx(x, y)    # log x-axis
7 plt.loglog(x, y)      # both log
```

11.8.3. Grid

```
1 ax.grid(True)
2 ax.grid(True, which="major", linestyle="-", alpha=0.5)
3 ax.grid(True, which="minor", linestyle=":", alpha=0.3)
4 ax.minorticks_on()
```

11.8.4. Annotations

```
1 ax.annotate("Peak",
2             xy=(peak_x, peak_y),           # point to annotate
3             xytext=(peak_x + 1, peak_y + 0.1), # text position
4             arrowprops=dict(arrowstyle="->"))
```

11.9. Style and Aesthetics

11.9.1. Built-in Styles

```
1 print(plt.style.available) # see all styles
2
3 plt.style.use("seaborn-v0_8-whitegrid")
4 # or
5 plt.style.use("ggplot")
```

11.9.2. Custom RC Parameters

```
1 plt.rcParams["figure.figsize"] = (8, 6)
2 plt.rcParams["font.size"] = 12
3 plt.rcParams["axes.labelsize"] = 14
4 plt.rcParams["lines.linewidth"] = 2
```

11.9.3. Publication-Ready Settings

```
1 # Settings for journal figures
2 plt.rcParams.update({
3     "figure.figsize": (3.5, 2.5), # single column width
4     "font.size": 8,
5     "axes.labelsize": 9,
6     "axes.titlesize": 9,
7     "legend.fontsize": 7,
8     "xtick.labelsize": 7,
9     "ytick.labelsize": 7,
10    "font.family": "sans-serif",
11    "figure.dpi": 300
12 })
```

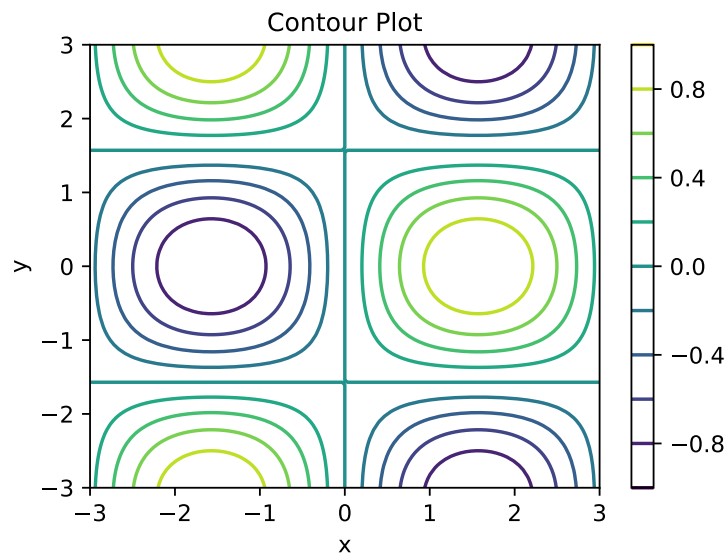
11.10. Specialized Plots

11.10.1. Contour Plots

```

1 x = np.linspace(-3, 3, 100)
2 y = np.linspace(-3, 3, 100)
3 X, Y = np.meshgrid(x, y)
4 Z = np.sin(X) * np.cos(Y)
5
6 plt.contour(X, Y, Z, levels=10)
7 plt.colorbar()

```

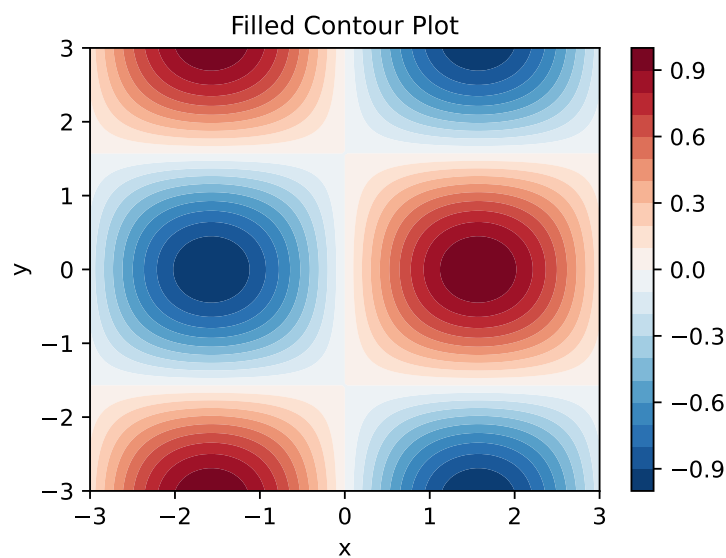


Filled contours use color to show values between contour lines:

```

1 # Filled contours
2 plt.contourf(X, Y, Z, levels=20, cmap="RdBu_r")
3 plt.colorbar()

```

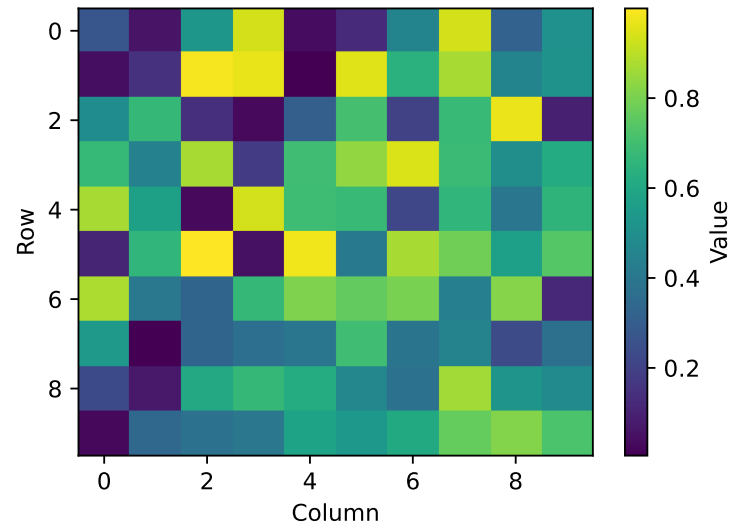


11.10.2. Heatmaps

```

1 data = np.random.rand(10, 10)
2
3 plt.imshow(data, cmap="viridis", aspect="auto")
4 plt.colorbar(label="Value")
5 plt.xlabel("Column")
6 plt.ylabel("Row")

```

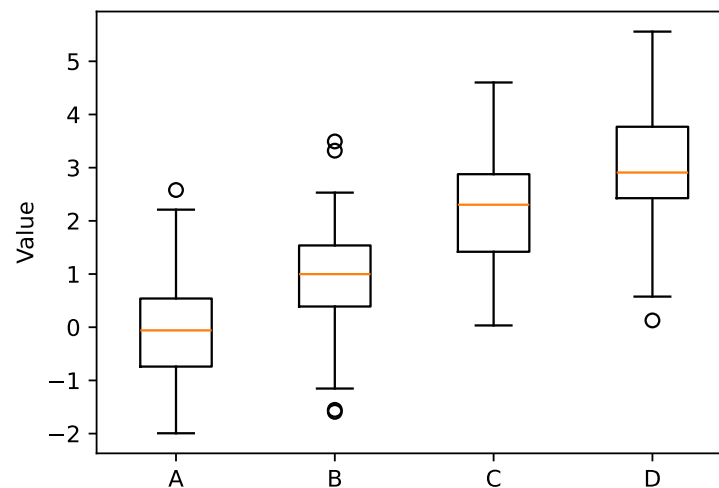


11.10.3. Box Plots

```

1 data = [np.random.randn(100) + i for i in range(4)]
2
3 plt.boxplot(data, labels=["A", "B", "C", "D"])
4 plt.ylabel("Value")

```



11.11. A Complete Figure Example

```

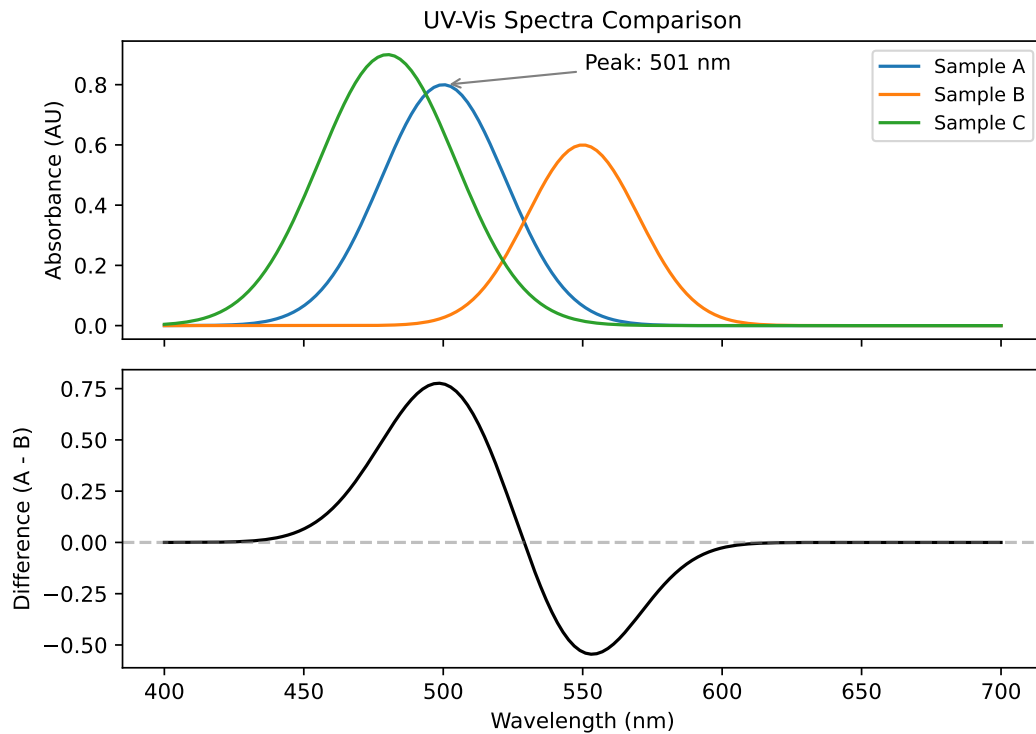
1 import matplotlib.pyplot as plt
2 import numpy as np

```

```

3
4 # Generate sample spectroscopy data
5 wavelengths = np.linspace(400, 700, 150)
6 spectrum1 = np.exp(-((wavelengths - 500)**2) / 1000) * 0.8
7 spectrum2 = np.exp(-((wavelengths - 550)**2) / 800) * 0.6
8 spectrum3 = np.exp(-((wavelengths - 480)**2) / 1200) * 0.9
9
10 # Create figure
11 fig, axes = plt.subplots(2, 1, figsize=(8, 6), sharex=True)
12
13 # Top panel: raw spectra
14 ax1 = axes[0]
15 ax1.plot(wavelengths, spectrum1, label="Sample A")
16 ax1.plot(wavelengths, spectrum2, label="Sample B")
17 ax1.plot(wavelengths, spectrum3, label="Sample C")
18 ax1.set_ylabel("Absorbance (AU)")
19 ax1.legend(loc="upper right")
20 ax1.set_title("UV-Vis Spectra Comparison")
21
22 # Bottom panel: difference spectrum
23 ax2 = axes[1]
24 difference = spectrum1 - spectrum2
25 ax2.plot(wavelengths, difference, "k-")
26 ax2.axhline(y=0, color="gray", linestyle="--", alpha=0.5)
27 ax2.set_ylabel("Difference (A - B)")
28 ax2.set_xlabel("Wavelength (nm)")
29
30 # Add peak annotations
31 peak_idx = np.argmax(spectrum1)
32 ax1.annotate(f"Peak: {wavelengths[peak_idx]:.0f} nm",
33             xy=(wavelengths[peak_idx], spectrum1[peak_idx]),
34             xytext=(wavelengths[peak_idx] + 50, spectrum1[peak_idx] +
35                   0.05),
36             arrowprops=dict(arrowstyle="->", color="gray"))
37
38 plt.tight_layout()
39 plt.savefig("spectra_comparison.pdf", bbox_inches="tight")
40 plt.show()

```



11.12. Chapter Summary

Matplotlib creates publication-quality figures:

- **Basic:** `plt.plot()`, `plt.scatter()`, `plt.bar()`, `plt.hist()`
- **Labels:** `plt.xlabel()`, `plt.ylabel()`, `plt.title()`, `plt.legend()`
- **Subplots:** `fig, axes = plt.subplots(rows, cols)`
- **Customization:** colors, line styles, markers, axes limits
- **Saving:** `plt.savefig("file.pdf", dpi=300)`
- **Styles:** `plt.style.use()`, `plt.rcParams`

Things to Ponder

1. When would you use `plt.plot()` vs `plt.scatter()`?
2. Why save figures as PDF rather than PNG for publications?
3. How would you create a figure with 3 subplots in a row?
4. What's the advantage of the object-oriented interface (`fig, ax = plt.subplots()`)?

Chapter 12.

SciPy: Scientific Computing

SciPy builds on NumPy to provide algorithms for optimization, integration, interpolation, signal processing, and statistics. If NumPy is the foundation, SciPy is the scientific toolkit built on top.

```
1 import numpy as np
2 from scipy import stats, optimize, integrate, interpolate, signal
```

12.1. Statistics

12.1.1. Descriptive Statistics

```
1 from scipy import stats
2
3 data = np.array([23.4, 25.1, 24.8, 26.2, 24.5, 25.8, 23.9])
4
5 # Basic statistics (also available in NumPy)
6 np.mean(data)      # 24.81
7 np.std(data)       # 0.96
8
9 # SciPy extras
10 stats.sem(data)      # standard error of mean
11 stats.gmean(data)    # geometric mean
12 stats.hmean(data)    # harmonic mean
13 stats.mode(data)     # mode (most common value)
14 stats.describe(data) # summary statistics
```

12.1.2. Distributions

SciPy provides many probability distributions:

```
1 from scipy.stats import norm, t, poisson
2
3 # Normal distribution
4 norm.pdf(0, loc=0, scale=1) # probability density at x=0
5 norm.cdf(1.96)             # cumulative probability P(X < 1.96)
6 norm.ppf(0.975)            # inverse CDF (percentile point)
7 norm.rvs(size=100)         # random samples
8
9 # Student's t distribution
10 t.ppf(0.975, df=10)        # critical value, 10 degrees of freedom
11
12 # Generate random samples from any distribution
13 samples = norm.rvs(loc=25, scale=2, size=1000) # mean=25, std=2
```

12.1.3. Statistical Tests

```

1  # t-test: compare means
2  group1 = [23, 25, 24, 26, 25]
3  group2 = [28, 30, 29, 31, 27]
4
5  statistic, pvalue = stats.ttest_ind(group1, group2)
6  print(f"p-value: {pvalue:.4f}")
7
8  # Paired t-test
9  before = [120, 125, 130, 128, 122]
10 after = [115, 120, 125, 121, 118]
11 statistic, pvalue = stats.ttest_rel(before, after)
12
13 # One-sample t-test (compare to known value)
14 statistic, pvalue = stats.ttest_1samp(data, popmean=25)

```

Lab Example

Testing if a treatment has an effect:

```

1  # Control and treatment group yields
2  control = np.array([45.2, 47.1, 44.8, 46.3, 45.9])
3  treatment = np.array([52.1, 49.8, 51.3, 50.7, 53.2])
4
5  # Independent samples t-test
6  t_stat, p_value = stats.ttest_ind(control, treatment)
7
8  print(f"Control mean: {control.mean():.2f}")
9  print(f"Treatment mean: {treatment.mean():.2f}")
10 print(f"t-statistic: {t_stat:.3f}")
11 print(f"p-value: {p_value:.4f}")
12
13 if p_value < 0.05:
14     print("Significant difference (p < 0.05)")
15 else:
16     print("No significant difference")

```

12.1.4. Correlation

```

1  x = np.array([1, 2, 3, 4, 5])
2  y = np.array([2.1, 4.0, 5.8, 8.1, 9.9])
3
4  # Pearson correlation
5  r, p = stats.pearsonr(x, y)
6  print(f"Pearson r: {r:.4f}, p-value: {p:.4f}")
7
8  # Spearman rank correlation
9  rho, p = stats.spearmanr(x, y)

```

12.2. Curve Fitting

12.2.1. Linear Regression

```

1  from scipy.stats import linregress

```

```

2
3 x = np.array([0.1, 0.2, 0.3, 0.4, 0.5])
4 y = np.array([0.12, 0.25, 0.35, 0.48, 0.61])
5
6 result = linregress(x, y)
7 print(f"Slope: {result.slope:.4f}")
8 print(f"Intercept: {result.intercept:.4f}")
9 print(f"R-squared: {result.rvalue**2:.4f}")
10
11 # Predict values
12 y_pred = result.slope * x + result.intercept

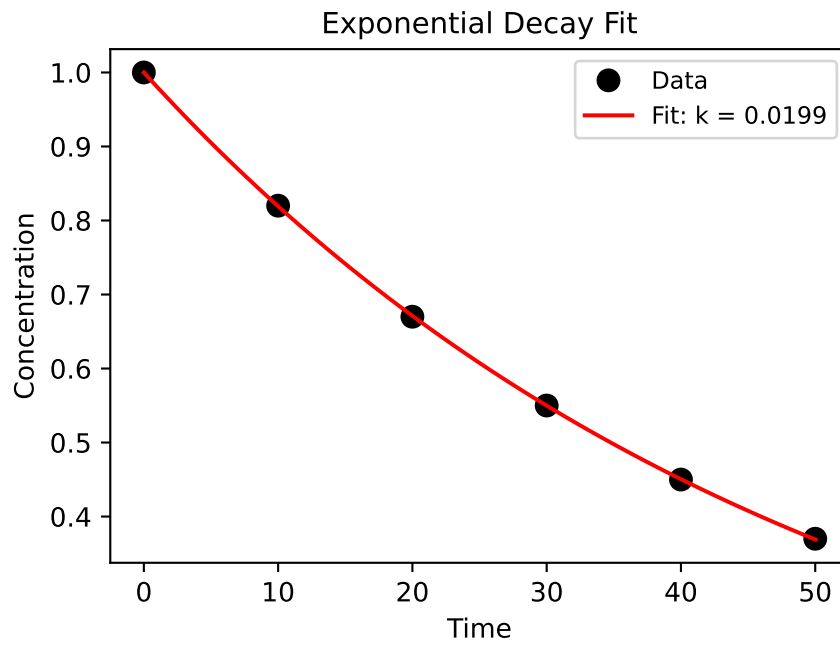
```

12.2.2. Nonlinear Curve Fitting

```

1 from scipy.optimize import curve_fit
2
3 # Define the model function
4 def exponential_decay(t, A, k):
5     return A * np.exp(-k * t)
6
7 # Data
8 t_data = np.array([0, 10, 20, 30, 40, 50])
9 c_data = np.array([1.0, 0.82, 0.67, 0.55, 0.45, 0.37])
10
11 # Fit the curve
12 popt, pcov = curve_fit(exponential_decay, t_data, c_data)
13 A_fit, k_fit = popt
14
15 print(f"Fitted A: {A_fit:.4f}")
16 print(f"Fitted k: {k_fit:.4f}")
17
18 # Parameter uncertainties (standard errors)
19 perr = np.sqrt(np.diag(pcov))
20 print(f"Uncertainty in k: {perr[1]:.4f}")
21
22 # Plot fit
23 t_smooth = np.linspace(0, 50, 100)
24 c_fit = exponential_decay(t_smooth, *popt)

```



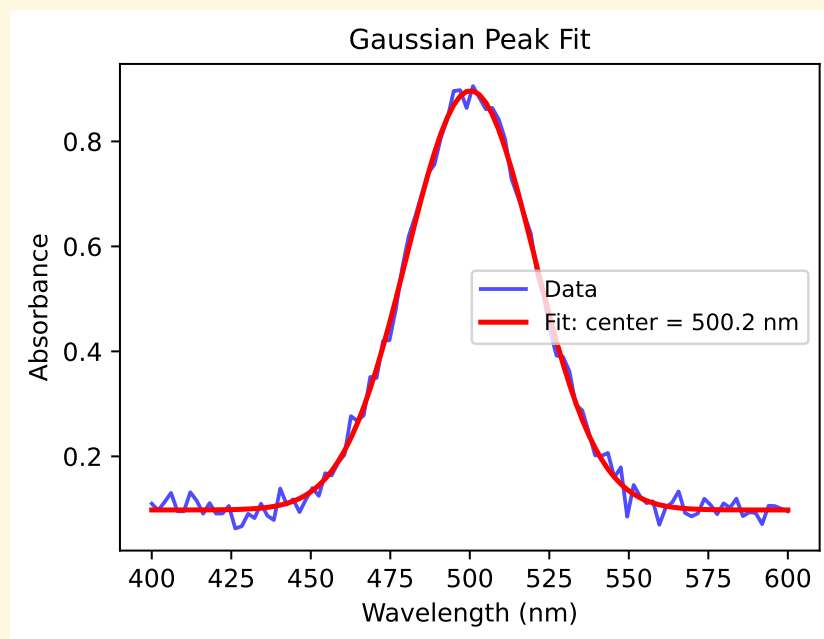
Lab Example

Fitting a Gaussian peak to spectroscopy data:

```

1 from scipy.optimize import curve_fit
2
3 def gaussian(x, amplitude, center, width, baseline):
4     return amplitude * np.exp(-(x - center)**2 / (2 * width**2)) +
5         baseline
6
7 # Simulated spectrum data
8 wavelengths = np.linspace(400, 600, 100)
9 true_peak = gaussian(wavelengths, 0.8, 500, 20, 0.1)
10 noise = np.random.randn(100) * 0.02
11 spectrum = true_peak + noise
12
13 # Fit
14 p0 = [0.7, 495, 25, 0.1] # initial guesses
15 popt, pcov = curve_fit(gaussian, wavelengths, spectrum, p0=p0)
16
17 print(f"Peak center: {popt[1]:.1f} nm")
18 print(f"Peak width (sigma): {popt[2]:.1f} nm")
19 print(f"Peak amplitude: {popt[0]:.3f}")

```



12.3. Optimization

12.3.1. Minimization

```

1 from scipy.optimize import minimize
2
3 # Function to minimize
4 def rosenbrock(x):
5     return (1 - x[0])**2 + 100 * (x[1] - x[0]**2)**2
6
7 # Starting point

```

```

8 x0 = [0, 0]
9
10 # Minimize
11 result = minimize(rosenbrock, x0, method="Nelder-Mead")
12 print(f"Minimum at: {result.x}")
13 print(f"Minimum value: {result.fun}")

```

12.3.2. Root Finding

```

1 from scipy.optimize import root_scalar, fsolve
2
3 # Find where f(x) = 0
4 def f(x):
5     return x**3 - 2*x - 5
6
7 # Scalar root finding
8 result = root_scalar(f, bracket=[2, 3])
9 print(f"Root: {result.root:.6f}")
10
11 # Multiple equations
12 def equations(vars):
13     x, y = vars
14     return [x + y - 4, x * y - 3]
15
16 solution = fsolve(equations, [1, 1])
17 print(f"Solution: x={solution[0]:.4f}, y={solution[1]:.4f}")

```

12.4. Integration

12.4.1. Definite Integrals

```

1 from scipy.integrate import quad
2
3 # Integrate sin(x) from 0 to pi
4 def f(x):
5     return np.sin(x)
6
7 result, error = quad(f, 0, np.pi)
8 print(f"Integral: {result:.6f}") # Should be 2.0
9
10 # With lambda
11 result, error = quad(lambda x: x**2, 0, 1) # integral of x^2

```

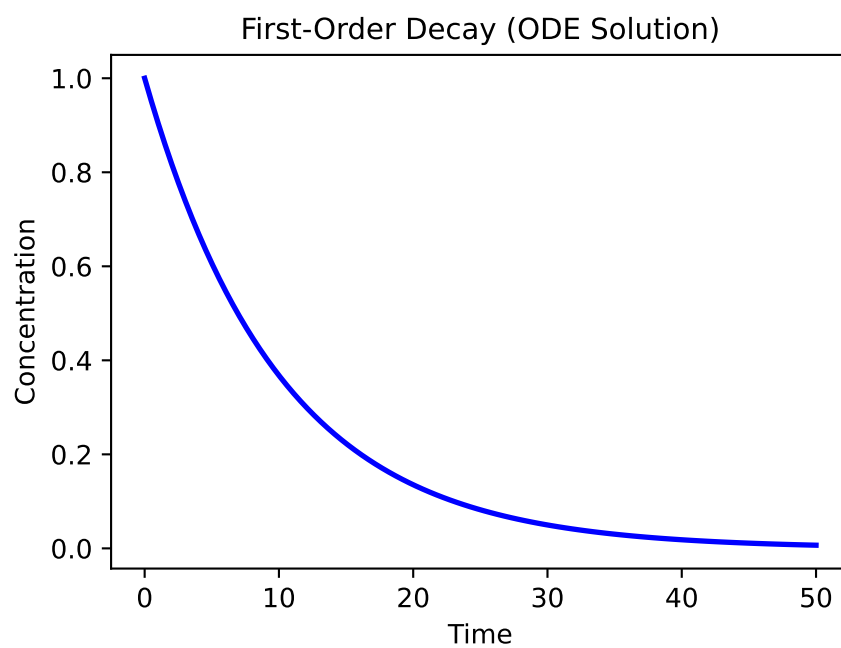
12.4.2. Differential Equations

```

1 from scipy.integrate import odeint
2
3 # First-order ODE: dy/dt = -k*y
4 def decay(y, t, k):
5     return -k * y
6
7 k = 0.1
8 y0 = 1.0
9 t = np.linspace(0, 50, 100)

```

```
10
11 solution = odeint(decay, y0, t, args=(k,))
12
13 import matplotlib.pyplot as plt
14 plt.plot(t, solution)
15 plt.xlabel("Time")
16 plt.ylabel("Concentration")
```



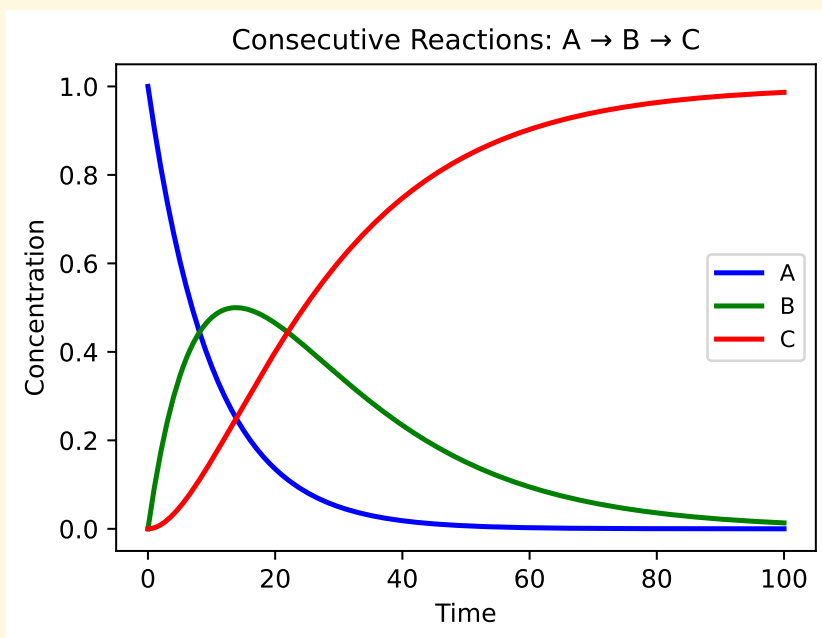
Lab Example

Solving coupled ODEs for a reaction $A \rightarrow B \rightarrow C$:

```

1 from scipy.integrate import odeint
2
3 def reaction_system(y, t, k1, k2):
4     A, B, C = y
5     dAdt = -k1 * A
6     dBdt = k1 * A - k2 * B
7     dCdt = k2 * B
8     return [dAdt, dBdt, dCdt]
9
10 # Parameters
11 k1, k2 = 0.1, 0.05
12 y0 = [1.0, 0.0, 0.0] # initial: all A
13 t = np.linspace(0, 100, 200)
14
15 # Solve
16 solution = odeint(reaction_system, y0, t, args=(k1, k2))
17 A, B, C = solution.T
18
19 # Plot
20 plt.plot(t, A, label="A")
21 plt.plot(t, B, label="B")
22 plt.plot(t, C, label="C")
23 plt.xlabel("Time")
24 plt.ylabel("Concentration")
25 plt.legend()

```



12.5. Interpolation

```

1 from scipy.interpolate import interp1d
2
3 # Sparse data

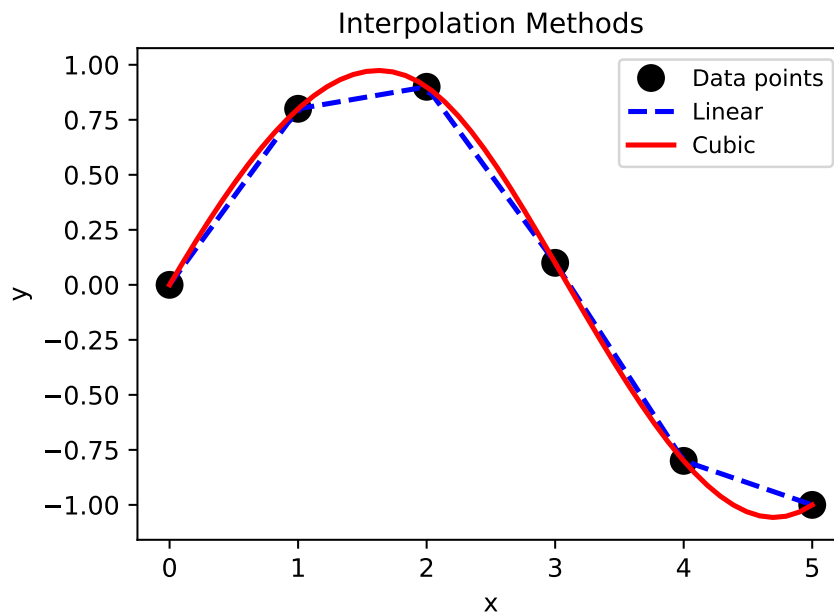
```



```

4 x_data = np.array([0, 1, 2, 3, 4, 5])
5 y_data = np.array([0, 0.8, 0.9, 0.1, -0.8, -1.0])
6
7 # Create interpolation function
8 f_linear = interp1d(x_data, y_data, kind="linear")
9 f_cubic = interp1d(x_data, y_data, kind="cubic")
10
11 # Evaluate at new points
12 x_new = np.linspace(0, 5, 50)
13 y_linear = f_linear(x_new)
14 y_cubic = f_cubic(x_new)

```



Interpolation types:

- "linear": straight lines between points
- "cubic": smooth cubic spline
- "nearest": step function
- "quadratic": quadratic spline

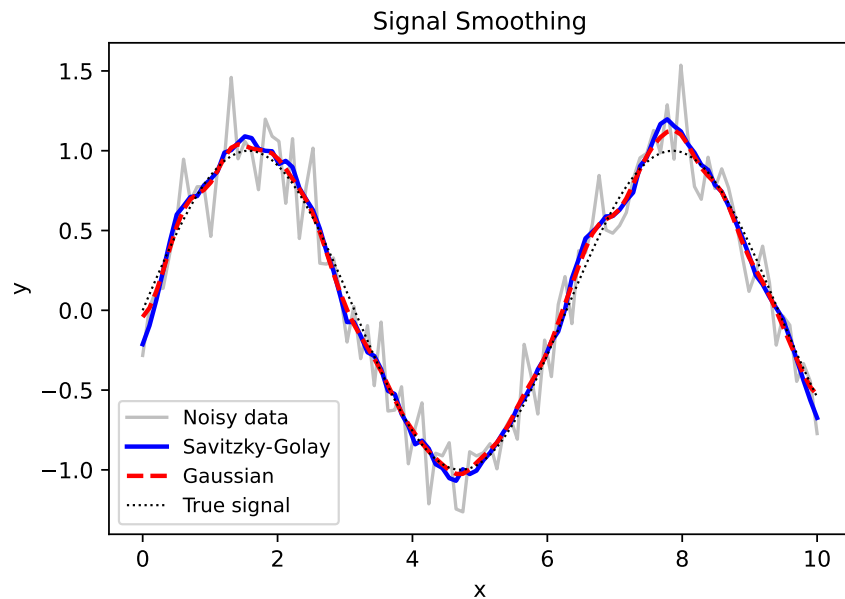
12.6. Signal Processing

12.6.1. Smoothing

```

1 from scipy.signal import savgol_filter
2 from scipy.ndimage import gaussian_filter1d
3
4 # Noisy data
5 x = np.linspace(0, 10, 100)
6 y_clean = np.sin(x)
7 y_noisy = y_clean + np.random.randn(100) * 0.2
8
9 # Savitzky-Golay filter
10 y_smooth = savgol_filter(y_noisy, window_length=11, polyorder=3)
11
12 # Gaussian smoothing
13 y_gauss = gaussian_filter1d(y_noisy, sigma=2)

```



12.6.2. Peak Finding

```

1 from scipy.signal import find_peaks
2
3 # Find peaks
4 peaks, properties = find_peaks(y_data, height=0.5, distance=10)
5 print(f"Peak indices: {peaks}")
6 print(f"Peak heights: {properties['peak_heights']}")

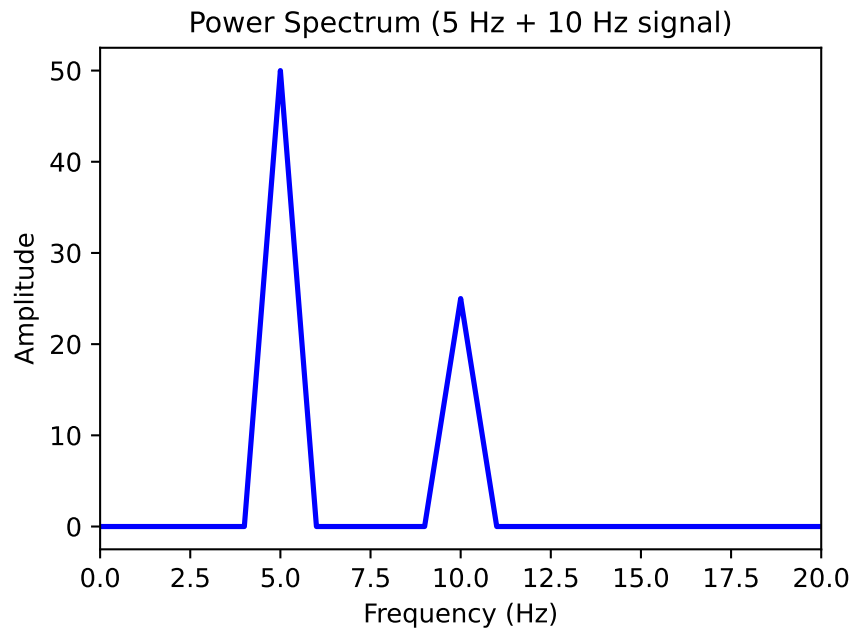
```

12.6.3. Fourier Transform

```

1 from scipy.fft import fft, fftfreq
2
3 # Time domain signal
4 dt = 0.01
5 t = np.arange(0, 1, dt)
6 signal = np.sin(2*np.pi*5*t) + 0.5*np.sin(2*np.pi*10*t)
7
8 # FFT
9 spectrum = fft(signal)
10 freqs = fftfreq(len(t), dt)
11
12 # Plot power spectrum
13 plt.plot(freqs[:len(freqs)//2], np.abs(spectrum[:len(spectrum)//2]))
14 plt.xlabel("Frequency (Hz)")
15 plt.ylabel("Amplitude")

```



12.7. Special Functions

SciPy provides many mathematical special functions:

```

1 from scipy import special
2
3 # Gamma function
4 special.gamma(5)          # 24.0 (= 4!)
5
6 # Error function
7 special.erf(1)            # 0.8427...
8
9 # Bessel functions
10 special.jv(0, 1)          # J_0(1) = 0.7652...
11
12 # Factorial
13 special.factorial(5)      # 120

```

12.8. Physical Constants

```

1 from scipy import constants
2
3 constants.c                # speed of light (m/s)
4 constants.h                # Planck constant (J*s)
5 constants.k                # Boltzmann constant (J/K)
6 constants.R                # gas constant (J/(mol*K))
7 constants.N_A              # Avogadro number
8
9 # Convert units
10 constants.convert_temperature(100, "C", "K") # 373.15
11 constants.calorie          # 1 calorie in Joules

```

12.9. Chapter Summary

SciPy extends NumPy with scientific algorithms:

- **Statistics:** `stats.ttest_ind()`, `stats.linregress()`, `distributions`
- **Curve fitting:** `optimize.curve_fit()`
- **Optimization:** `optimize.minimize()`, `optimize.root_scalar()`
- **Integration:** `integrate.quad()`, `integrate.odeint()`
- **Interpolation:** `interpolate.interp1d()`
- **Signal processing:** `signal.find_peaks()`, `signal.savgol_filter()`
- **Constants:** `constants.R`, `constants.N_A`

Things to Ponder

1. When would you use `curve_fit` vs. `linregress`?
2. What's the difference between interpolation and curve fitting?
3. How do you interpret a p-value from a t-test?
4. Why might you need to provide initial guesses (`p0`) for curve fitting?

Part IV.

Putting It Together

Chapter 13.

Practical Projects

You've learned the pieces. Now let's put them together in complete, realistic projects. Each example demonstrates how the skills from previous chapters combine to solve actual scientific problems.

13.1. Project 1: Calibration Curve Analysis

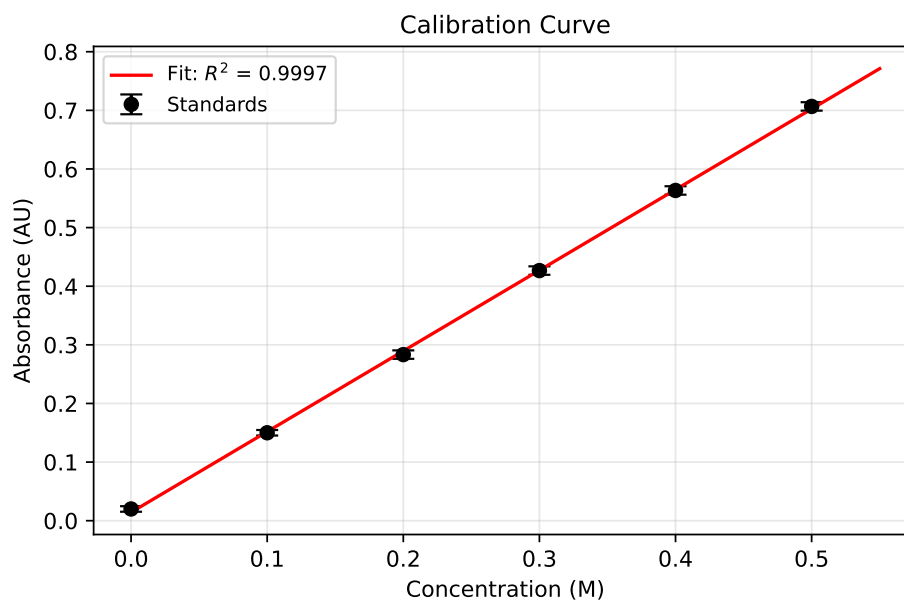
A common task: create a calibration curve from standards and use it to determine concentrations of unknown samples.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy.stats import linregress
5
6 # =====
7 # Load and inspect data
8 # =====
9 standards = pd.read_csv("standards.csv")
10 unknowns = pd.read_csv("unknowns.csv")
11
12 print("Standards:")
13 print(standards.head())
14 print("\nUnknowns:")
15 print(unknowns.head())
16
17 # =====
18 # Calculate mean absorbance for each standard
19 # =====
20 # Standards file has replicates
21 std_summary = standards.groupby("concentration")["absorbance"].agg(
22     ["mean", "std", "count"]
23 ).reset_index()
24 std_summary["sem"] = std_summary["std"] / np.sqrt(std_summary["count"])
25
26 # =====
27 # Linear regression
28 # =====
29 x = std_summary["concentration"].values
30 y = std_summary["mean"].values
31
32 result = linregress(x, y)
33 slope = result.slope
34 intercept = result.intercept
35 r_squared = result.rvalue ** 2
36
37 print(f"\nCalibration equation: A = {slope:.4f} * C + {intercept:.4f}")
38 print(f"R-squared: {r_squared:.6f}")
39
40 # =====
```

```

41 # Create calibration plot
42 # =====
43 fig, ax = plt.subplots(figsize=(8, 6))
44
45 # Plot standards with error bars
46 ax.errorbar(x, y, yerr=std_summary["sem"], fmt="ko",
47             capsize=5, label="Standards")
48
49 # Plot regression line
50 x_line = np.linspace(0, x.max() * 1.1, 100)
51 y_line = slope * x_line + intercept
52 ax.plot(x_line, y_line, "r-", label=f"Fit:  $R^2 = {r_squared:.4f}$ ")
53
54 ax.set_xlabel("Concentration (M)")
55 ax.set_ylabel("Absorbance (AU)")
56 ax.set_title("Calibration Curve")
57 ax.legend()
58 ax.grid(True, alpha=0.3)
59
60 plt.tight_layout()
61 plt.savefig("calibration_curve.pdf")

```



```

1 # =====
2 # Calculate unknown concentrations
3 # =====
4 unknowns["concentration"] = (unknowns["absorbance"] - intercept) / slope
5
6 # Check if within calibration range
7 min_std = x.min()
8 max_std = x.max()
9 unknowns["in_range"] = (unknowns["concentration"] >= min_std) & \
10                        (unknowns["concentration"] <= max_std)
11
12 # Flag samples outside range
13 outside_range = unknowns[~unknowns["in_range"]]
14 if len(outside_range) > 0:

```



```

15     print(f"\nWarning: {len(outside_range)} samples outside calibration
      range")
16
17 # =====
18 # Save results
19 # =====
20 unknowns.to_csv("unknown_concentrations.csv", index=False)
21 print(f"\nResults saved. Processed {len(unknowns)} samples.")

```

13.2. Project 2: Kinetics Data Analysis

Analyze reaction kinetics data to determine rate constants and reaction order.

```

1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  from scipy.optimize import curve_fit
5
6  # =====
7  # Load kinetics data
8  # =====
9  data = pd.read_csv("kinetics_data.csv")
10 time = data["time_s"].values
11 concentration = data["concentration_M"].values
12
13 # =====
14 # Define kinetic models
15 # =====
16 def first_order(t, C0, k):
17     """First-order decay:  $C = C0 * \exp(-kt)$ """
18     return C0 * np.exp(-k * t)
19
20 def second_order(t, C0, k):
21     """Second-order decay:  $1/C = 1/C0 + kt$ """
22     return C0 / (1 + C0 * k * t)
23
24 # =====
25 # Fit both models
26 # =====
27 # First order
28 try:
29     popt1, pcov1 = curve_fit(first_order, time, concentration)
30     C0_1, k1 = popt1
31     y_fit1 = first_order(time, *popt1)
32     ss_res1 = np.sum((concentration - y_fit1)**2)
33     ss_tot = np.sum((concentration - np.mean(concentration))**2)
34     r2_1 = 1 - ss_res1/ss_tot
35     print(f"First-order: k = {k1:.4f} s-1, R2 = {r2_1:.4f}")
36 except:
37     print("First-order fit failed")
38
39 # Second order
40 try:
41     popt2, pcov2 = curve_fit(second_order, time, concentration)
42     C0_2, k2 = popt2
43     y_fit2 = second_order(time, *popt2)
44     ss_res2 = np.sum((concentration - y_fit2)**2)

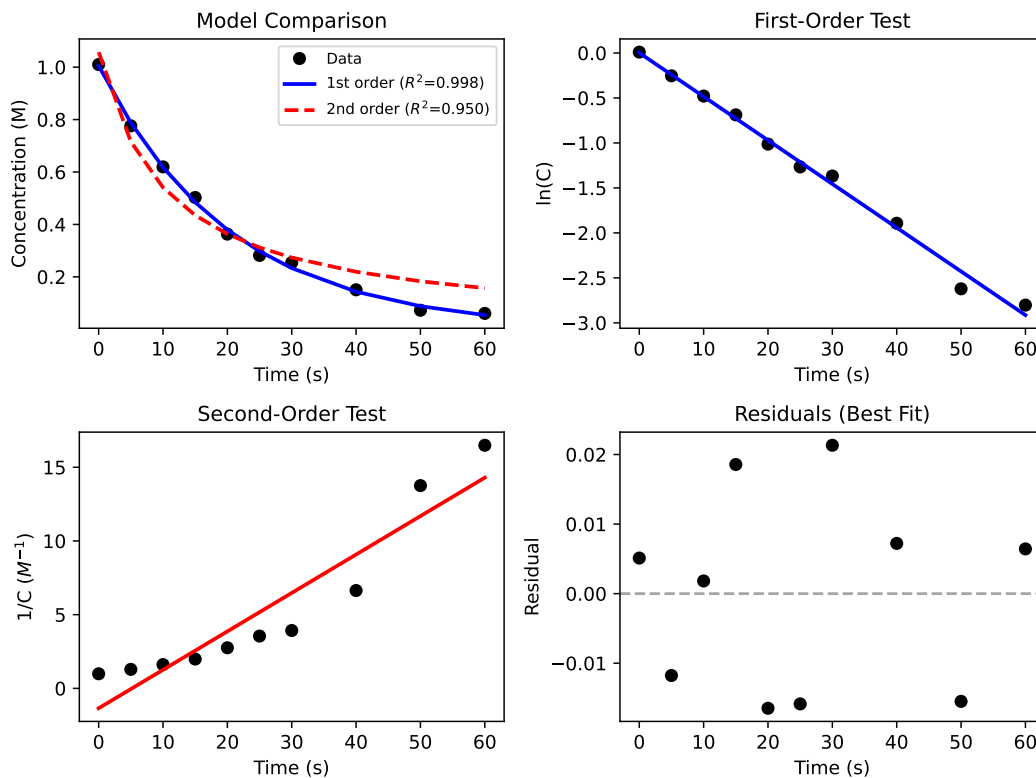
```

```

45     r2_2 = 1 - ss_res2/ss_tot
46     print(f"Second-order: k = {k2:.4f} M-1s-1, R2 = {r2_2:.4f}")
47 except:
48     print("Second-order fit failed")
49
50 # =====
51 # Determine best model
52 # =====
53 if r2_1 > r2_2:
54     best_model = "First-order"
55     best_k = k1
56     best_fit = y_fit1
57 else:
58     best_model = "Second-order"
59     best_k = k2
60     best_fit = y_fit2
61
62 print(f"\nBest fit: {best_model}")
63
64 # =====
65 # Create diagnostic plots
66 # =====
67 fig, axes = plt.subplots(2, 2, figsize=(10, 8))
68
69 # Raw data with fits
70 ax1 = axes[0, 0]
71 ax1.plot(time, concentration, "ko", label="Data")
72 ax1.plot(time, y_fit1, "b-", label=f"1st order (R2={r2_1:.3f})")
73 ax1.plot(time, y_fit2, "r--", label=f"2nd order (R2={r2_2:.3f})")
74 ax1.set_xlabel("Time (s)")
75 ax1.set_ylabel("Concentration (M)")
76 ax1.legend()
77 ax1.set_title("Model Comparison")
78
79 # First-order linearization: ln(C) vs t
80 ax2 = axes[0, 1]
81 ax2.plot(time, np.log(concentration), "ko")
82 ax2.set_xlabel("Time (s)")
83 ax2.set_ylabel("ln(C)")
84 ax2.set_title("First-Order Test")
85
86 # Second-order linearization: 1/C vs t
87 ax3 = axes[1, 0]
88 ax3.plot(time, 1/concentration, "ko")
89 ax3.set_xlabel("Time (s)")
90 ax3.set_ylabel("1/C (M-1)")
91 ax3.set_title("Second-Order Test")
92
93 # Residuals for best fit
94 ax4 = axes[1, 1]
95 residuals = concentration - best_fit
96 ax4.plot(time, residuals, "ko")
97 ax4.axhline(y=0, color="r", linestyle="--")
98 ax4.set_xlabel("Time (s)")
99 ax4.set_ylabel("Residual")
100 ax4.set_title(f"Residuals ({best_model})")
101
102 plt.tight_layout()

```

```
103 plt.savefig("kinetics_analysis.pdf")
```



```

1  # =====
2  # Calculate half-life
3  # =====
4  if best_model == "First-order":
5      half_life = np.log(2) / best_k
6  else:
7      half_life = 1 / (best_k * concentration[0])
8
9  print(f"Half-life: {half_life:.2f} s")

```

13.3. Project 3: Batch Processing Spectra

Process multiple spectrum files: baseline correction, peak finding, and summary report.

```

1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  from pathlib import Path
5  from scipy.signal import find_peaks, savgol_filter
6
7  # =====
8  # Setup
9  # =====
10 data_folder = Path("spectra")
11 output_folder = Path("processed")
12 output_folder.mkdir(exist_ok=True)
13
14 results = []

```

```

15
16 # =====
17 # Processing functions
18 # =====
19 def load_spectrum(filepath):
20     """Load spectrum from CSV file."""
21     df = pd.read_csv(filepath)
22     return df["wavelength"].values, df["absorbance"].values
23
24 def baseline_correct(absorbance, percentile=5):
25     """Subtract baseline estimated from lowest values."""
26     baseline = np.percentile(absorbance, percentile)
27     return absorbance - baseline
28
29 def find_spectrum_peaks(wavelengths, absorbance, min_height=0.1):
30     """Find peaks in spectrum."""
31     peak_indices, properties = find_peaks(absorbance, height=min_height)
32     peak_wavelengths = wavelengths[peak_indices]
33     peak_heights = properties["peak_heights"]
34     return list(zip(peak_wavelengths, peak_heights))
35
36 # =====
37 # Process each file
38 # =====
39 for filepath in sorted(data_folder.glob("*.csv")):
40     print(f"Processing {filepath.name}...")
41
42     # Load
43     wavelengths, absorbance = load_spectrum(filepath)
44
45     # Smooth
46     abs_smooth = savgol_filter(absorbance, window_length=11, polyorder
47                                =3)
48
49     # Baseline correct
50     abs_corrected = baseline_correct(abs_smooth)
51
52     # Find peaks
53     peaks = find_spectrum_peaks(wavelengths, abs_corrected, min_height
54                                =0.05)
55
56     # Record results
57     sample_result = {
58         "filename": filepath.name,
59         "max_absorbance": np.max(abs_corrected),
60         "num_peaks": len(peaks),
61     }
62
63     # Add first 3 peaks
64     for i, (wl, height) in enumerate(peaks[:3]):
65         sample_result[f"peak{i+1}_wavelength"] = wl
66         sample_result[f"peak{i+1}_height"] = height
67
68     results.append(sample_result)
69
70     # Save processed spectrum
71     processed = pd.DataFrame({
72         "wavelength": wavelengths,

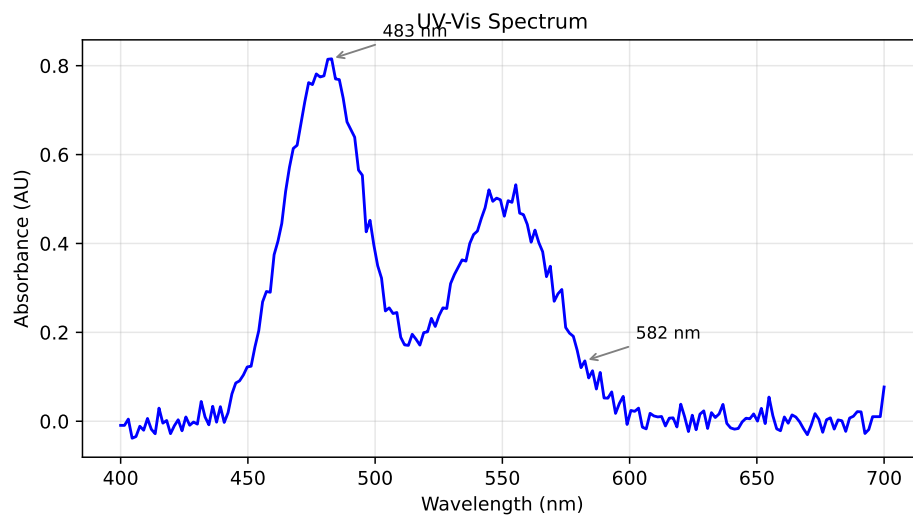
```

```

71         "raw": absorbance,
72         "smoothed": abs_smooth,
73         "corrected": abs_corrected
74     })
75     processed.to_csv(output_folder / f"processed_{filepath.name}", index
76                     =False)
77
78     # Create figure
79     fig, ax = plt.subplots(figsize=(8, 5))
80     ax.plot(wavelengths, absorbance, "gray", alpha=0.5, label="Raw")
81     ax.plot(wavelengths, abs_corrected, "b-", label="Corrected")
82
83     # Mark peaks
84     for wl, height in peaks:
85         ax.annotate(f"{wl:.0f}", xy=(wl, height), fontsize=8,
86                   ha="center", va="bottom")
87
88     ax.set_xlabel("Wavelength (nm)")
89     ax.set_ylabel("Absorbance (AU)")
90     ax.set_title(filepath.stem)
91     ax.legend()
92
93     plt.tight_layout()
94     plt.savefig(output_folder / f"{filepath.stem}.png", dpi=150)
95     plt.close()
96
97     # =====
98     # Create summary
99     # =====
100    summary = pd.DataFrame(results)
101    summary.to_csv(output_folder / "summary.csv", index=False)
102
103    print(f"\nProcessed {len(results)} files")
104    print(f"Results saved to {output_folder}")
105    print("\nSummary:")
106    print(summary[["filename", "max_absorbance", "num_peaks"]].to_string())

```

An example of processed output:



13.4. Project 4: Interactive Data Dashboard

Create a simple interactive analysis script with user input.

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 def load_data(filepath):
6     """Load data with error handling."""
7     try:
8         return pd.read_csv(filepath)
9     except FileNotFoundError:
10        print(f"Error: File '{filepath}' not found")
11        return None
12    except Exception as e:
13        print(f"Error loading file: {e}")
14        return None
15
16 def main():
17     print("=" * 50)
18     print("Data Analysis Dashboard")
19     print("=" * 50)
20
21     # Get filename
22     filepath = input("\nEnter data file path: ").strip()
23     data = load_data(filepath)
24
25     if data is None:
26         return
27
28     print(f"\nLoaded {len(data)} rows, {len(data.columns)} columns")
29     print(f"Columns: {list(data.columns)}")
30
31     while True:
32         print("\n" + "-" * 30)
33         print("Options:")
34         print("  1. View data summary")
35         print("  2. Plot column")
36         print("  3. Calculate statistics")
37         print("  4. Filter data")
38         print("  5. Export filtered data")
39         print("  6. Exit")
40
41         choice = input("\nSelect option (1-6): ").strip()
42
43         if choice == "1":
44             print("\nData Summary:")
45             print(data.describe())
46
47         elif choice == "2":
48             print(f"\nAvailable columns: {list(data.columns)}")
49             x_col = input("X column: ").strip()
50             y_col = input("Y column: ").strip()
51
52             try:
53                 plt.figure(figsize=(8, 6))
54                 plt.plot(data[x_col], data[y_col], "o-")
55                 plt.xlabel(x_col)

```

```

56         plt.ylabel(y_col)
57         plt.title(f"{y_col} vs {x_col}")
58         plt.tight_layout()
59         plt.show()
60     except KeyError as e:
61         print(f"Column not found: {e}")
62
63     elif choice == "3":
64         print(f"\nAvailable columns: {list(data.columns)}")
65         col = input("Column to analyze: ").strip()
66
67         try:
68             values = data[col].dropna()
69             print(f"\nStatistics for '{col}':")
70             print(f"    Count: {len(values)}")
71             print(f"    Mean: {values.mean():.4f}")
72             print(f"    Std: {values.std():.4f}")
73             print(f"    Min: {values.min():.4f}")
74             print(f"    Max: {values.max():.4f}")
75         except (KeyError, TypeError) as e:
76             print(f"Error: {e}")
77
78     elif choice == "4":
79         print(f"\nAvailable columns: {list(data.columns)}")
80         col = input("Column to filter: ").strip()
81         op = input("Operator (>, <, ==): ").strip()
82         val = input("Value: ").strip()
83
84         try:
85             val = float(val)
86             if op == ">":
87                 data = data[data[col] > val]
88             elif op == "<":
89                 data = data[data[col] < val]
90             elif op == "==":
91                 data = data[data[col] == val]
92             print(f"Filtered to {len(data)} rows")
93         except Exception as e:
94             print(f"Error filtering: {e}")
95
96     elif choice == "5":
97         outpath = input("Output filename: ").strip()
98         data.to_csv(outpath, index=False)
99         print(f"Saved to {outpath}")
100
101     elif choice == "6":
102         print("Goodbye!")
103         break
104
105     else:
106         print("Invalid option")
107
108 if __name__ == "__main__":
109     main()

```

13.5. Tips for Real Projects

13.5.1. Project Organization

```

1 my_project/
2   data/
3     raw/           # original data (never modify!)
4     processed/     # cleaned/processed data
5   scripts/
6     analyze.py
7     visualize.py
8     utils.py       # helper functions
9   results/
10    figures/
11    tables/
12  README.md        # project description
13  main.py           # Your main script which you run
14  requirements.txt  # dependencies

```

Then, from your main script you can call functions from the scripts folder. For example, say `analyze.py` contains functions `process_data()`, `load_data()` and `visualize.py` contains `plot_results()` then perhaps your main script looks something like this:

```

1 from scripts.analyze import load_data, process_data
2 from scripts.visualize import plot_results
3
4 my_data_path = "path/to/my/data.csv"
5 data = load_data(my_data_path)
6 processed_data = process_data(data)
7 plot_results(processed_data)

```

13.5.2. Reproducibility Checklist

- Keep raw data separate and unmodified
- Record which script version produced each output
- Use `requirements.txt` to track package versions
- Add comments explaining non-obvious decisions
- Save figures as both PDF (for papers) and PNG (for presentations)

13.5.3. Common Patterns

```

1 # Pattern: Process multiple files
2 from pathlib import Path
3
4 for filepath in Path("data").glob("*.csv"):
5     data = pd.read_csv(filepath)
6     result = process(data)
7     result.to_csv(f"results/{filepath.stem}_processed.csv")
8
9 # Pattern: Save figure with data
10 fig, ax = plt.subplots()
11 ax.plot(x, y)
12 fig.savefig("plot.pdf")
13 pd.DataFrame({"x": x, "y": y}).to_csv("plot_data.csv")
14
15 # Pattern: Progress indicator

```



```
16 for i, filepath in enumerate(files):  
17     print(f"Processing {i+1}/{len(files)}: {filepath.name}")  
18     # process file
```

Tip

If progress bars end up being important to your project, check out the package `tqdm` for polished progress bars with duration estimates built in.

Things to Ponder

1. What repetitive tasks in your research could Python automate?
2. How would you modify the calibration project to handle non-linear calibration curves?
3. Design a project to analyze data from your own research.
4. What error handling would you add to make these scripts more robust?

Part V.

Closing Remarks

13.6. Where to Go From Here

This book gave you the essentials. The real learning happens when you start using Python for your own research. Here are resources to help you continue.

13.6.1. Packages to Explore

These libraries extend your toolkit for specific tasks. With what you’ve learned, you can teach yourself any of them from their documentation:

- **seaborn** — Statistical visualization built on Matplotlib. Less code, beautiful plots.
- **scikit-learn** — Machine learning.
- **statsmodels** — Statistical modeling: linear models, time series, hypothesis tests.
- **sympy** — Symbolic mathematics: algebra, calculus, equation solving (Mathematica like).
- **lmfit** — Curve fitting with parameter constraints, confidence intervals, & model evaluation.
- **uncertainties** — Automatic error propagation through calculations.
- **pint** — Physical units handling (never confuse meters and feet again).

13.6.2. Domain-Specific Libraries

Search for Python packages in your field—they almost certainly exist:

- **Chemistry:** RDKit, Open Babel, cclib, PyMOL
- **Biology:** Biopython, scikit-bio, scanpy
- **Physics/Astronomy:** Astropy, QuTiP, PyMC
- **Geoscience:** xarray, cartopy, MetPy
- **Image Analysis:** scikit-image, OpenCV, Pillow

13.6.3. Improve Your Workflow

As your projects grow, these practices will save you time and headaches:

- Learn Git for version control
- Write tests for your code
- Create reusable packages from your scripts
- Explore Jupyter notebooks for interactive analysis

13.6.4. Practice

The best way to learn is to use Python for something you actually care about. Pick a dataset from your research. Automate something tedious. Recreate a figure from a paper. Break things, fix them, and learn by doing.

You now have enough foundation to teach yourself the rest.

13.6.5. AI in the Scientific Workflow

Note

Large Language Models (LLMs) are powerful assistive technologies, but they reward prepared minds disproportionately. This section offers suggestions for integrating them into your learning without undermining the skill development that makes you an independent researcher.

As you progress, you will inevitably encounter LLMs (AI chatbots) in your coding workflow. When used developmentally, they can accelerate your transition from syntax memorization to architectural thinking. Used prematurely, they can create an illusion of competence that fragments under scrutiny. The goal is to harness the former while avoiding the latter.

Understanding the Tool

Think of an LLM as an *ontological cartographer*—a system that maps relationships between concepts based on statistical patterns across text, rather than experiential understanding.

You understand a “reaction mechanism” through laboratory experience, which carries a lot of information we never care to encode in language. The model understands “reaction mechanism” only in so far as can be derived from what we cared to capture using language. This is not a limitation to resent but a translation layer to manage. The model translates between linguistic representations of concepts; you must provide the physical-world verification.

Things to Ponder

Think about how much is left unsaid in literature methods sections, or lab manuals. How could you expect a thing which lives in a world defined entirely by these incomplete captures to match your depth of comprehension of the topics?

Warning

An LLM cannot improve the quality of your question—it can only elaborate upon it. If you formulate a concept’s question representation vaguely, that vagueness is a part of the concept you are invoking. So, when the LLM translates the concept to its answer representation, quality predictably suffers. To win, you must understand the domain well enough to formulate good questions and evaluate the answers. For automated script writing, AI is most helpful when you are already proficient, not when you are a complete novice.

A Developmental Framework

Rather than viewing AI as forbidden or universally permitted, consider this progression:

1. Foundation Phase (Avoid generation, embrace explanation)

When learning a new library (e.g., your first RDKit script or lmfit model), do not ask the AI to write the code. Instead, write your own attempt, then use AI to:

- Explain error tracebacks in plain language
- Compare your approach to idiomatic patterns
- Suggest documentation sections you may have missed

2. Apprenticeship Phase (Pair programming)

Once you can write working but inelegant code, use AI as a collaborative reviewer:

- Refactor working scripts for readability
- Vectorize slow loops you have already prototyped
- Generate unit tests for functions you have written

3. Independence Phase (Managed delegation)

When you could write the code yourself but time is constrained, delegate with oversight:

- Generate boilerplate for familiar patterns, then audit line-by-line
- Prototype alternative algorithms you already understand conceptually
- Document code you have written (reverse the typical workflow)

Tip**The explain-it-back Rule**

Before submitting any AI-assisted code, apply the laboratory notebook standard: could you explain this snippet to a labmate during group meeting? If not, treat it as a learning opportunity, not a submission. Open a separate file, break the code into chunks, and annotate each line until the logic is transparent.

Mentorship and Cognitive Load

Novice code written by humans has predictable error patterns—syntax mistakes, logical gaps, misconception signatures. These are diagnostically valuable; a mentor can identify the underlying confusion quickly, turning debugging into targeted teaching.

AI-generated novice code lacks these tell-tale patterns. It appears plausible but fails in subtle ways (e.g., using deprecated matplotlib arguments that still run but produce wrong chemical visualisations). This creates asymmetric debugging: you learn nothing, while your mentor expends disproportionate effort uncovering the error.

The solution is transparency, not avoidance. When seeking help with AI-assisted code:

- Present your original attempt alongside the AI version
- Highlight specific lines you do not fully understand
- Ask: “What conceptual gaps would cause this type of error?” rather than just “Fix this.”

This transforms the interaction from resentment-generating noise into a diagnostic dialogue about your reasoning.

Speed vs. Velocity

It is tempting to view AI as a productivity multiplier, but in skill acquisition, velocity matters more than speed. Velocity implies direction; speed merely measures motion.

Writing a script manually might take four hours; with AI, forty minutes. But if the manual process taught you to recognise array broadcasting errors, you have gained velocity—the next similar task takes thirty minutes regardless of AI assistance. If you skipped the struggle, you remain dependent on the tool for identical tasks forever.

Slow is smooth, and smooth is fast. Master the concept at a pace that allows comprehension, then use AI to accelerate implementation of what you genuinely understand.

Warning**Reputation and Ownership**

In professional contexts—industry R&D, graduate research groups, collaborative publications—your code represents your scientific rigour. Submitting opaque AI-generated scripts that fail under review signals unreliable judgement. Conversely, demonstrating fluency in explaining, verifying, and modifying AI-assisted workflows signals mature computational literacy. The tool is neutral; your stewardship of it is not.

13.6.6. Where to Get Help

If you do get stuck (and you will be—everyone does) in order of quality:

- **Official Documentation** — NumPy, Pandas, and Matplotlib all have excellent tutorials and example galleries.
- **Your colleagues**—They will often know more about the specific problems you’re trying to solve than most others.

- **Stack Overflow** — Search before asking. Your error message has almost certainly been answered.
- **GitHub Issues** — For package-specific bugs or feature questions.
- **Reddit** — r/learnpython is friendly to beginners; r/Python for general discussion.
- **Chat-bots**—Undeniably useful, even if these can be problematic. Use with caution, and don't accept code you don't understand. These are a double-edged blade, and you WILL get cut using these.

Happy Coding!